

Listas lineares

Estruturas de Dados

Prof. Dr. Franklin César Flores

Departamento de Informática

Universidade Estadual de Maringá

Listas lineares

- Este conjunto de slides foi utilizado durante as aulas das disciplinas:
- 6884 – Estruturas de Dados – Turma 02 (Ciência da Computação)
- 9893 – Estruturas de Dados – Turma 31 (Informática)
- Maringá, 2º semestre letivo de 2024

Roteiro

- Listas lineares
 - Tipo abstrato de dados
- Pilhas, filas e filas duplas (deques)
 - Tipos abstratos de dados
 - Implementações com alocação estática
 - Implementações com alocação dinâmica
- Outras implementações de listas ligadas
 - Listas ligadas com cabeça de lista
 - Listas circulares
 - Listas duplamente ligadas
- Exercícios
- Bibliografia

Listas lineares

- Uma **lista linear** é uma sequência de n elementos $a_1, a_2, a_3, \dots, a_n$ dispostos em uma determinada ordem
- As posições dos elementos na lista linear definem a ordem da disposição destes elementos
- Dizemos que o número de elementos de uma lista linear ($n \geq 0$) define o seu comprimento. Também dizemos que n é o tamanho da lista
- A lista linear é uma lista vazia se $n = 0$



Listas lineares

- Uma **lista linear** é uma sequência de n elementos $a_1, a_2, a_3, \dots, a_n$ dispostos em uma determinada ordem
- A lista linear é uma lista vazia se $n = 0$
- Se $n > 0$,
 - a_1 é o primeiro elemento da lista linear
 - a_n é o último elemento
 - a_i é precedido por a_{i-1} , $\forall 1 < i \leq n$
 - a_i é sucedido por a_{i+1} , $\forall 1 \leq i < n$



Listas lineares

- Uma **lista linear** é uma sequência de n elementos $a_1, a_2, a_3, \dots, a_n$ dispostos em uma determinada ordem
- Uma lista linear pode ser armazenada em alocação estática ou alocação dinâmica



Listas lineares

- Algumas operações com o tipo abstrato de dados **lista linear**:

1. Criar uma lista linear

Listas lineares

- Algumas operações com o tipo abstrato de dados **lista linear**:

2. Localizar o i -ésimo elemento de uma lista linear



Listas lineares

- Algumas operações com o tipo abstrato de dados **lista linear**:

3. Inserir um elemento na i -ésima posição



Listas lineares

- Algumas operações com o tipo abstrato de dados **lista linear**:

4. Remover o i -ésimo elemento da lista linear



Listas lineares

- Algumas operações com o tipo abstrato de dados **lista linear**:

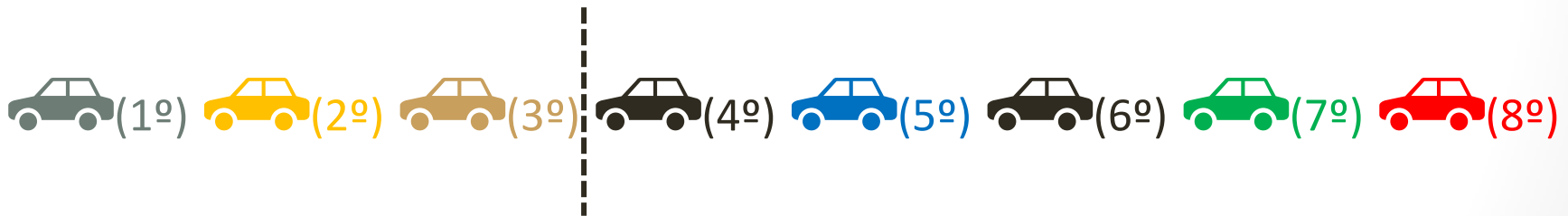
5. Combinar duas ou mais listas lineares em uma única lista



Listas lineares

- Algumas operações com o tipo abstrato de dados **lista linear**:

6. Separar uma lista linear em duas ou mais listas



Listas lineares

- Algumas operações com o tipo abstrato de dados **lista linear**:

7. Criar uma cópia dos elementos de uma lista linear
(gerando uma outra lista linear)



Listas lineares

- Algumas operações com o tipo abstrato de dados **lista linear**:

8. Informar o número de elementos de uma lista linear



Listas lineares

- Algumas operações com o tipo abstrato de dados **lista linear**:

9. Pesquisar por um elemento em uma lista linear



Listas lineares

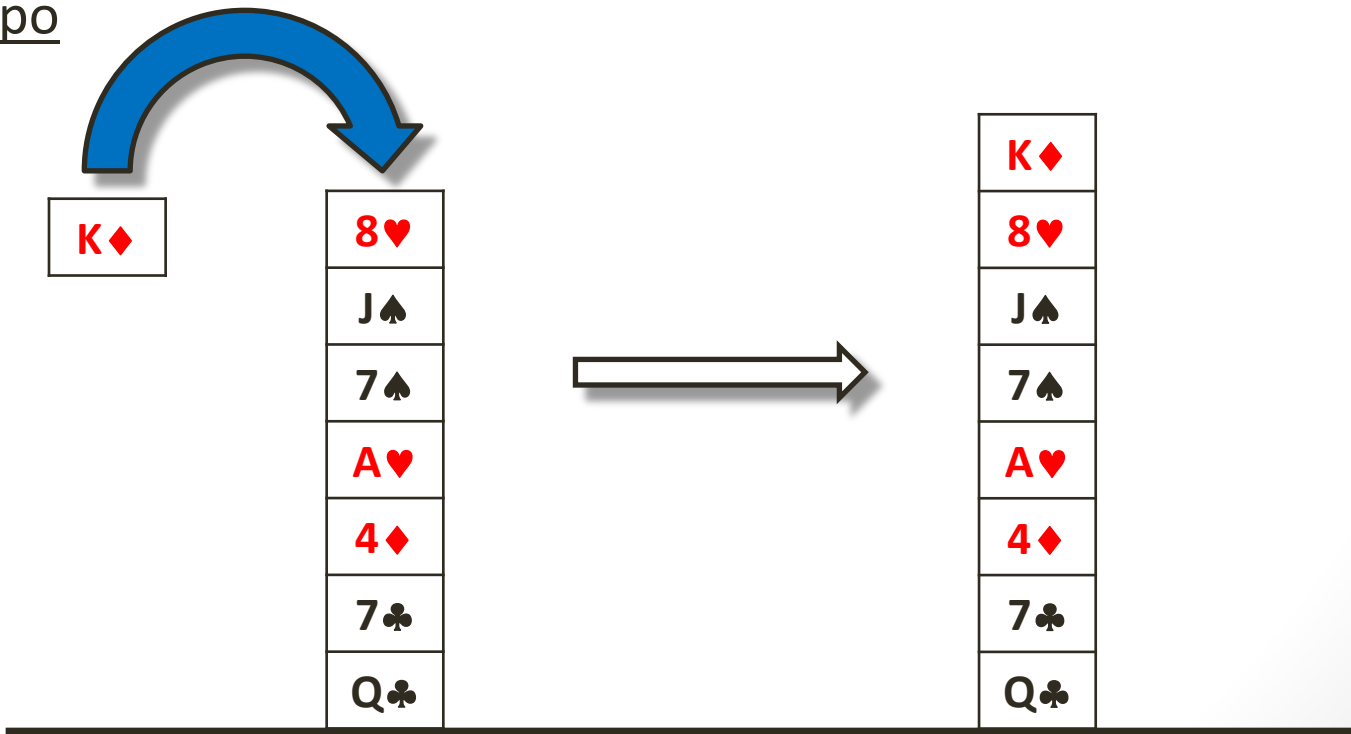
- Algumas operações com o tipo abstrato de dados **lista linear**:

10. Ordenar uma lista linear de acordo com alguma característica de seus elementos
(ordem crescente ou decrescente)



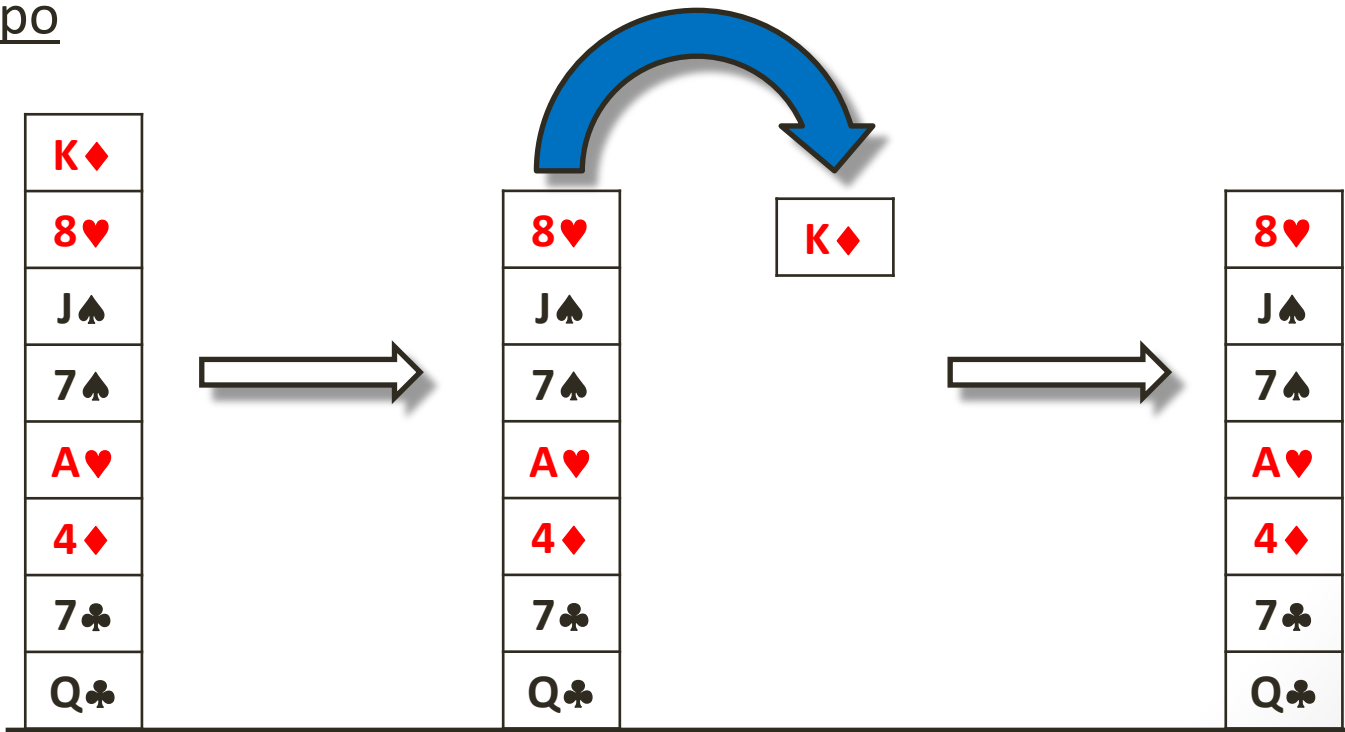
Listas lineares

- A **pilha** é uma lista linear onde as inserções, remoções e demais acessos são realizados somente em uma das extremidades da lista
- Esta extremidade onde ocorrem os acessos é chamada de topo



Listas lineares

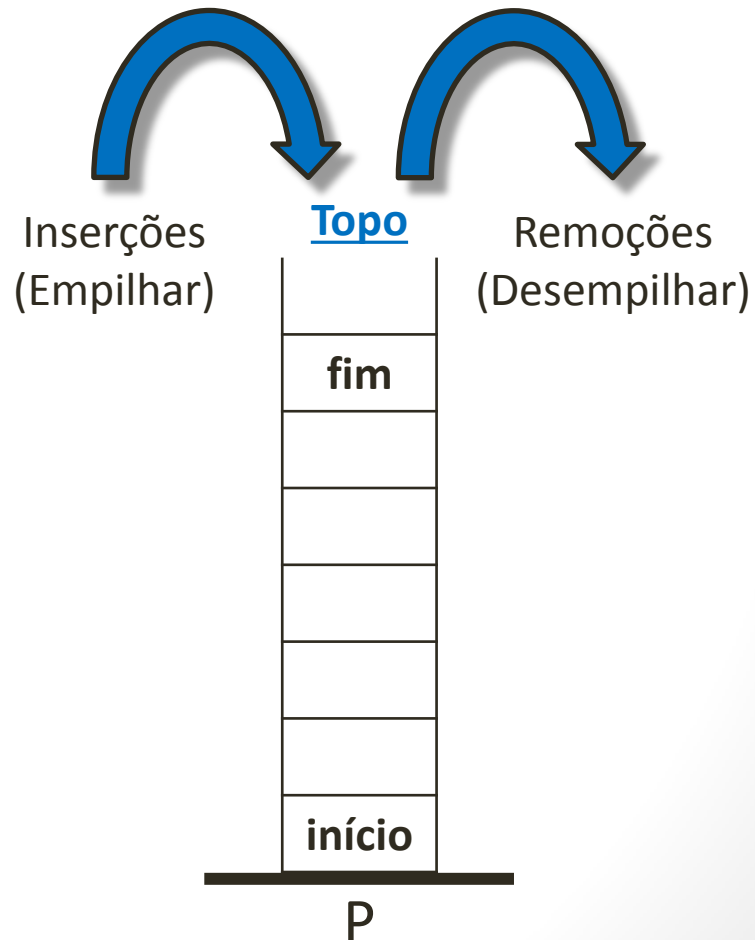
- A **pilha** é uma lista linear onde as inserções, remoções e demais acessos são realizados somente em uma das extremidades da lista
- Esta extremidade onde ocorrem os acessos é chamada de topo



Listas lineares

- Propriedade da pilha:
- O último elemento que foi inserido em uma pilha é o primeiro que poderá ser removido

- *stack*
- LIFO
- Last **I**n, First **O**ut



Listas lineares

- Algumas operações com o tipo abstrato de dados **pilha**:

1. InicializaPilha(P);

- Faz a pilha P ficar vazia

Topo

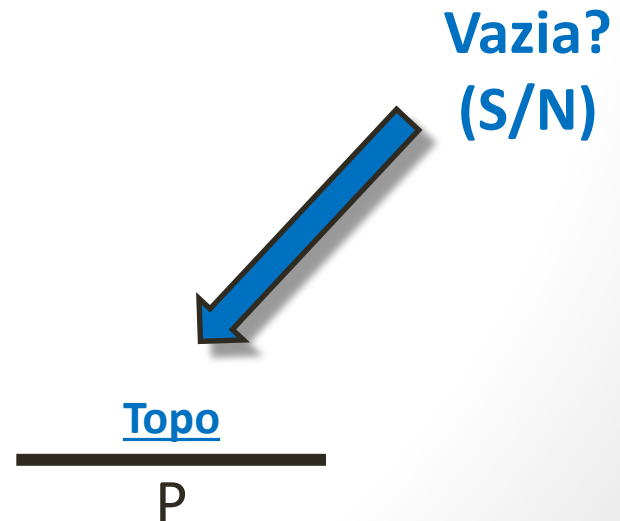
P

Listas lineares

- Algumas operações com o tipo abstrato de dados **pilha**:

2. PilhaVazia(P);

- Retorna VERDADEIRO se a pilha P estiver vazia
- Retorna FALSO, caso contrário

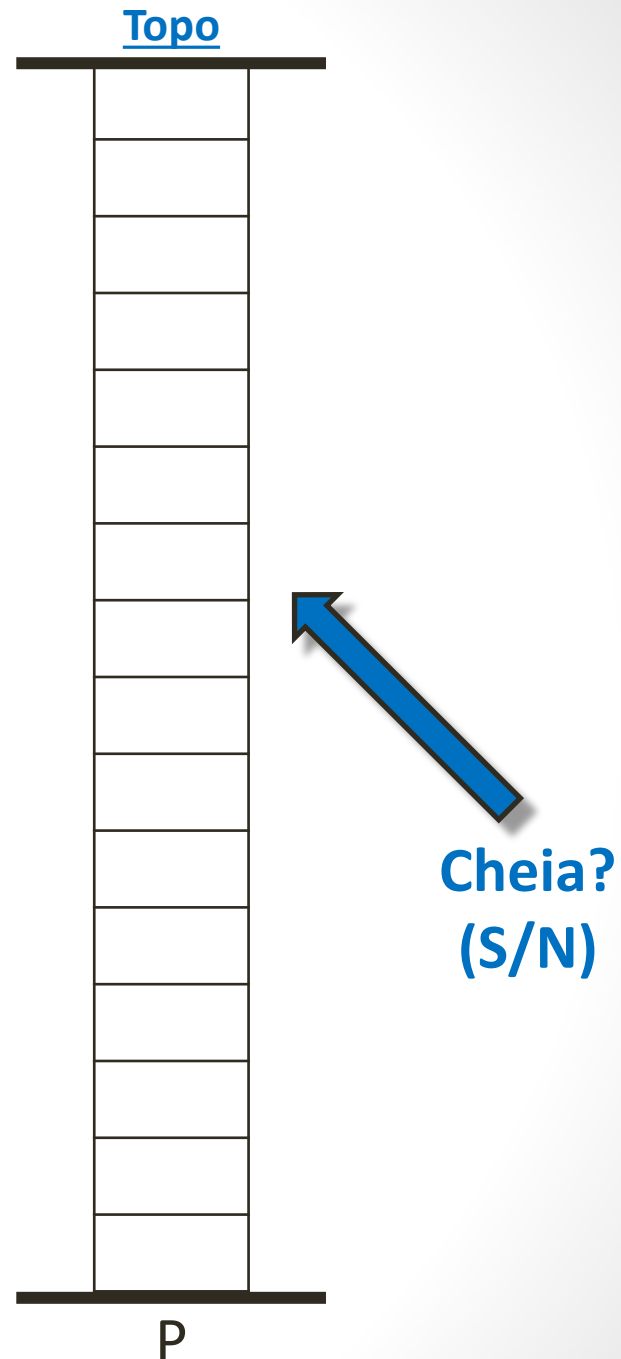


Listas lineares

- Algumas operações com o tipo abstrato de dados **pilha**:

3. PilhaCheia(P);

- Retorna VERDADEIRO se a pilha P estiver cheia
- Retorna FALSO, caso contrário

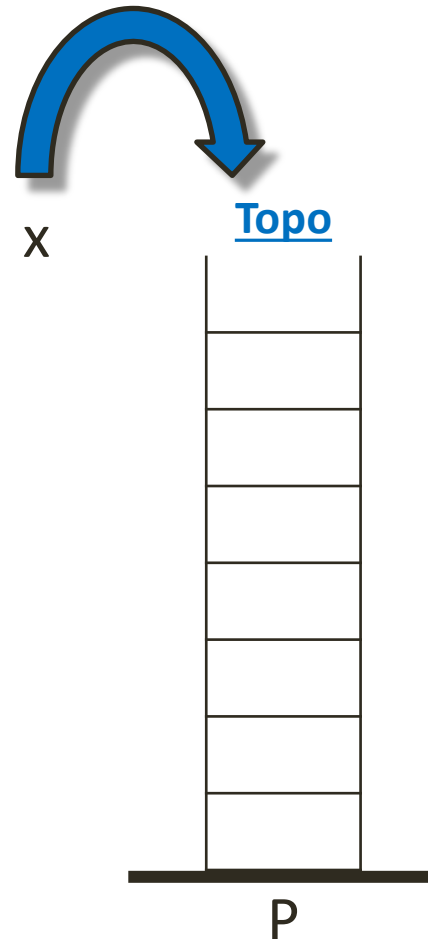


Listas lineares

- Algumas operações com o tipo abstrato de dados **pilha**:

4. Empilha(P, x);

- Insere o elemento x no topo da pilha P

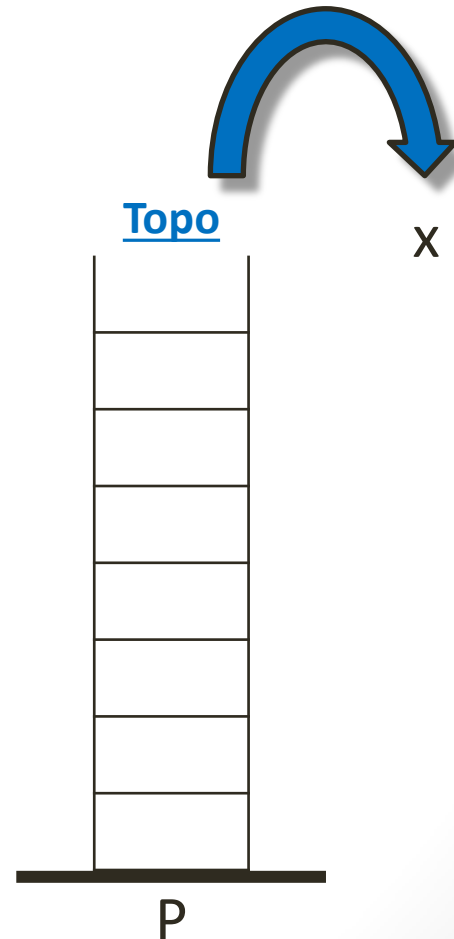


Listas lineares

- Algumas operações com o tipo abstrato de dados **pilha**:

5. Desempilha(P, x);

- Remove o elemento do topo da pilha P e o devolve em x

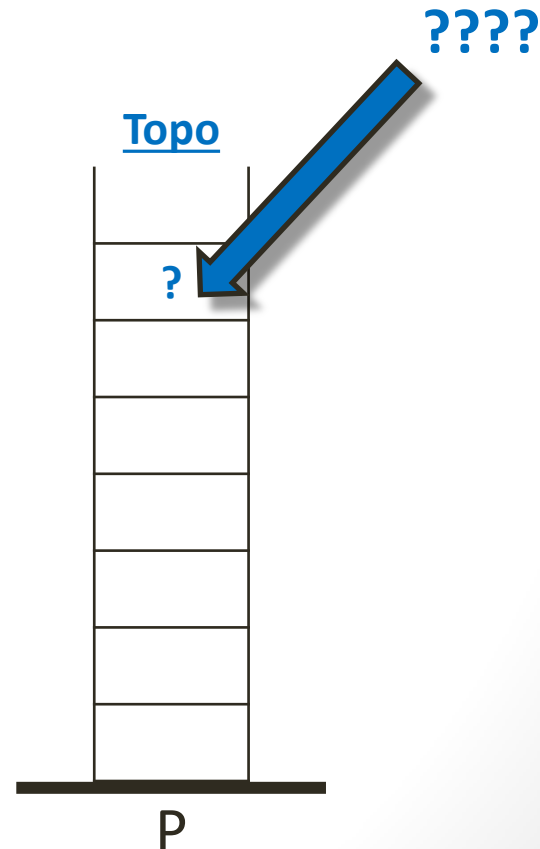


Listas lineares

- Algumas operações com o tipo abstrato de dados **pilha**:

6. `ElementoDoTopo(P);`

- Retorna o elemento que está no topo da pilha P
(sem removê-lo da estrutura)

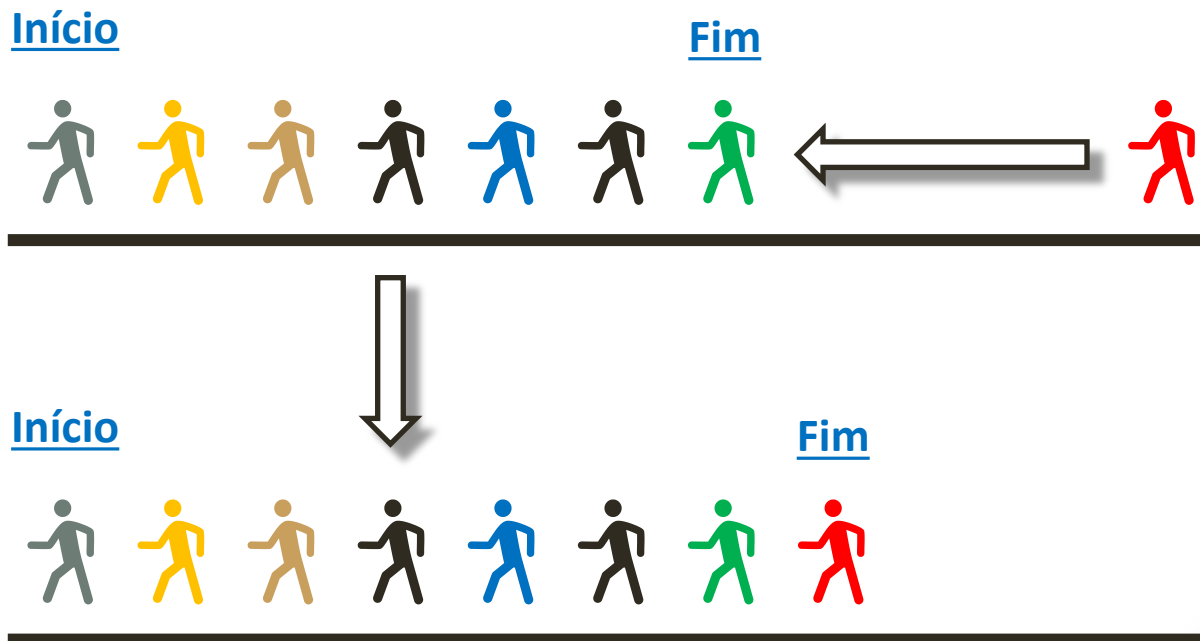


Listas lineares

- Alguns exemplos de aplicações de **pilha**:
 - Avaliação de notação posfixa
 - Verificação de parentização
 - Recursão
 - Backtracking
 - Grafos (busca em profundidade)
 - Gerenciamento de memória

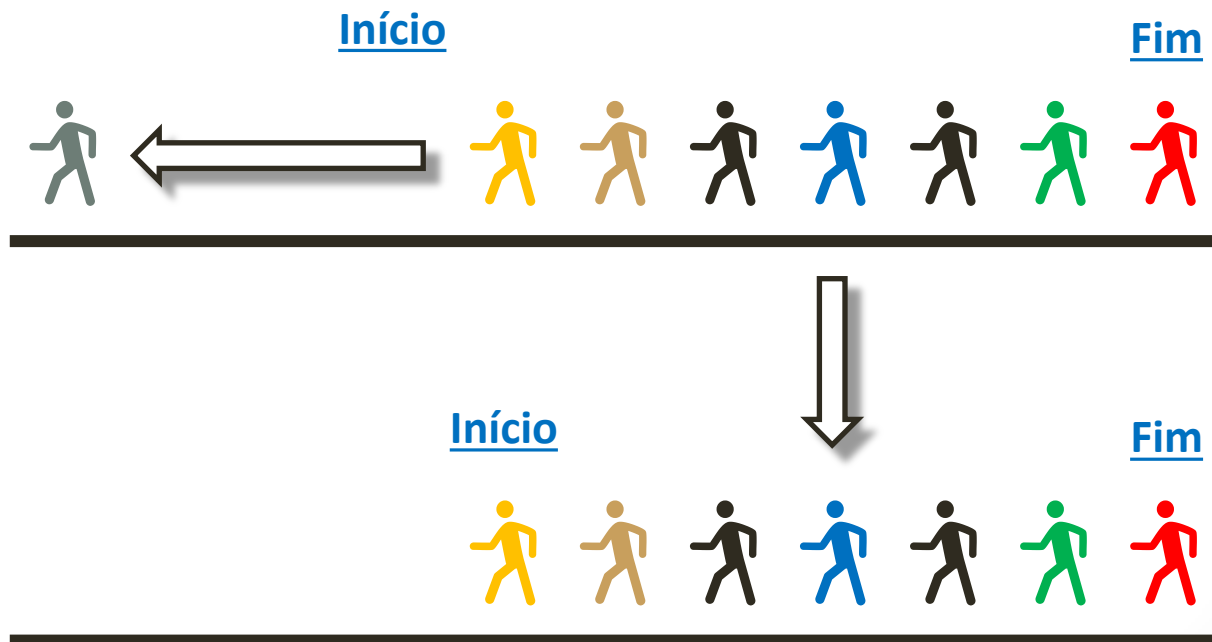
Listas lineares

- A **fila** é uma lista linear onde as inserções são realizadas em uma das extremidades, que é chamada de fim
- As remoções e demais acessos são realizados na outra extremidade, que é chamada de início



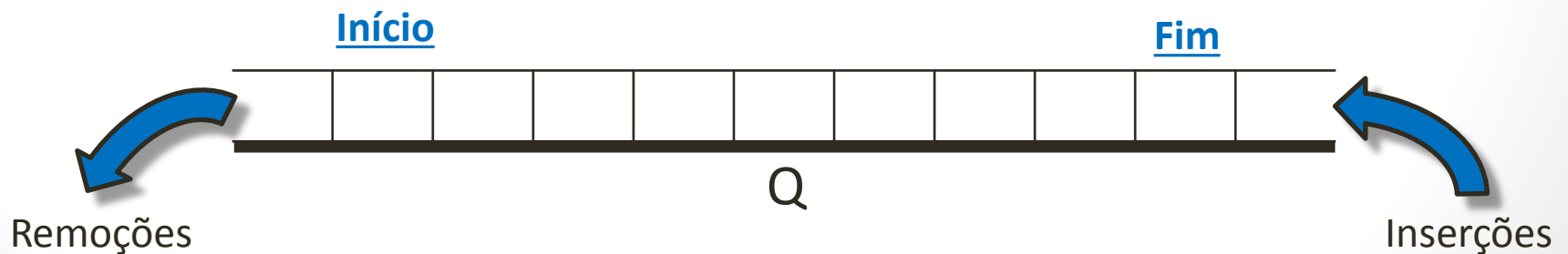
Listas lineares

- A **fila** é uma lista linear onde as inserções são realizadas em uma das extremidades, que é chamada de fim
- As remoções e demais acessos são realizados na outra extremidade, que é chamada de início



Listas lineares

- Propriedade da fila:
- O primeiro elemento que foi inserido em uma fila é o primeiro que poderá ser removido
- *queue*
- FIFO
- First **I**n, First **O**ut



Listas lineares

- Algumas operações com o tipo abstrato de dados **fila**:

1. InicializaFila(Q);

- Faz a fila Q ficar vazia

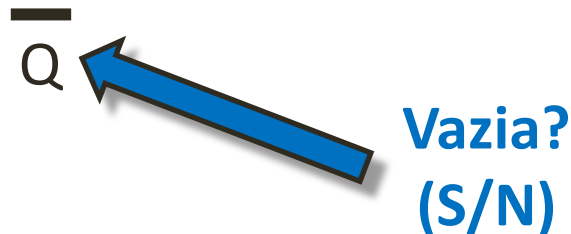
Q

Listas lineares

- Algumas operações com o tipo abstrato de dados **fila**:

2. FilaVazia(Q);

- Retorna VERDADEIRO se a fila Q estiver vazia
- Retorna FALSO, caso contrário

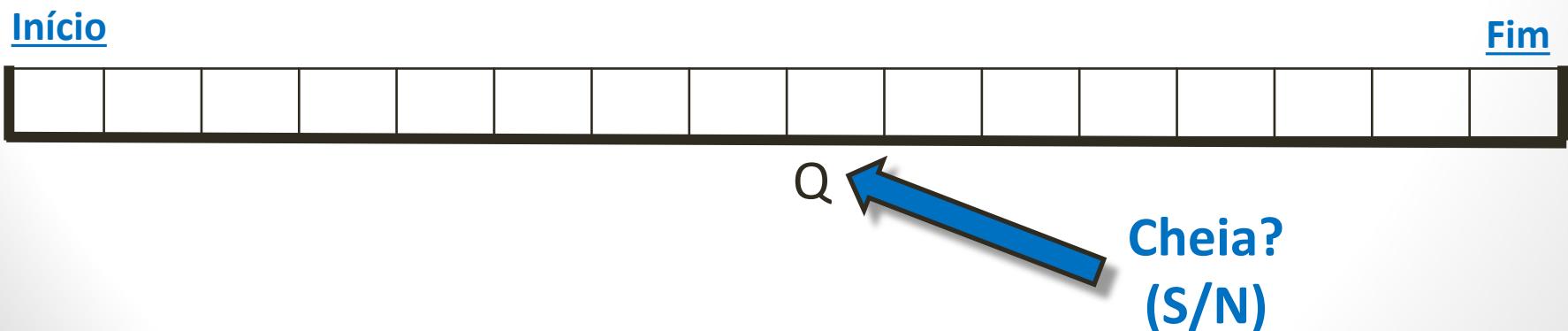


Listas lineares

- Algumas operações com o tipo abstrato de dados **fila**:

3. FilaCheia(Q);

- Retorna VERDADEIRO se a fila Q estiver cheia
- Retorna FALSO, caso contrário

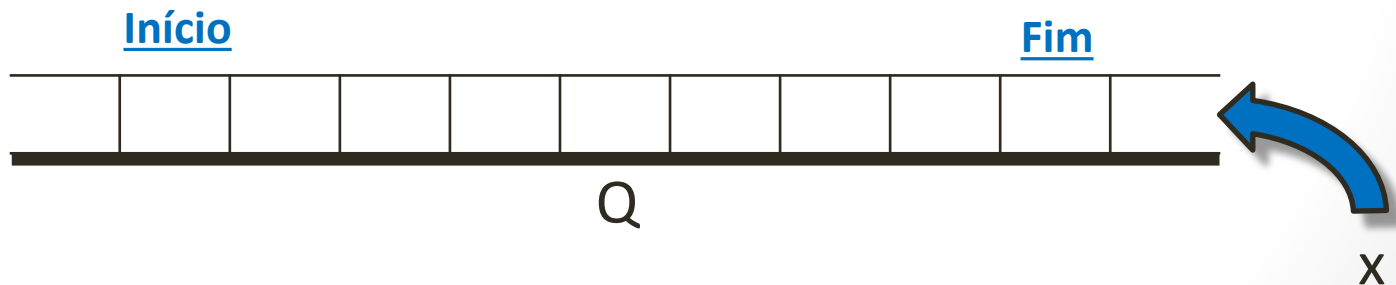


Listas lineares

- Algumas operações com o tipo abstrato de dados **fila**:

4. `InsereFila(Q, x);`

- Insere o elemento x após o último elemento da fila Q

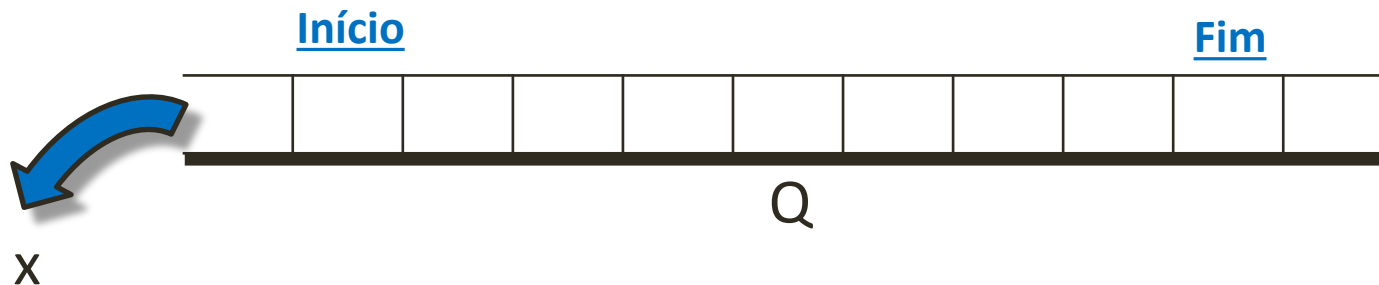


Listas lineares

- Algumas operações com o tipo abstrato de dados **fila**:

5. RemoveFila(Q, x);

- Remove o primeiro elemento da fila Q e o devolve em x

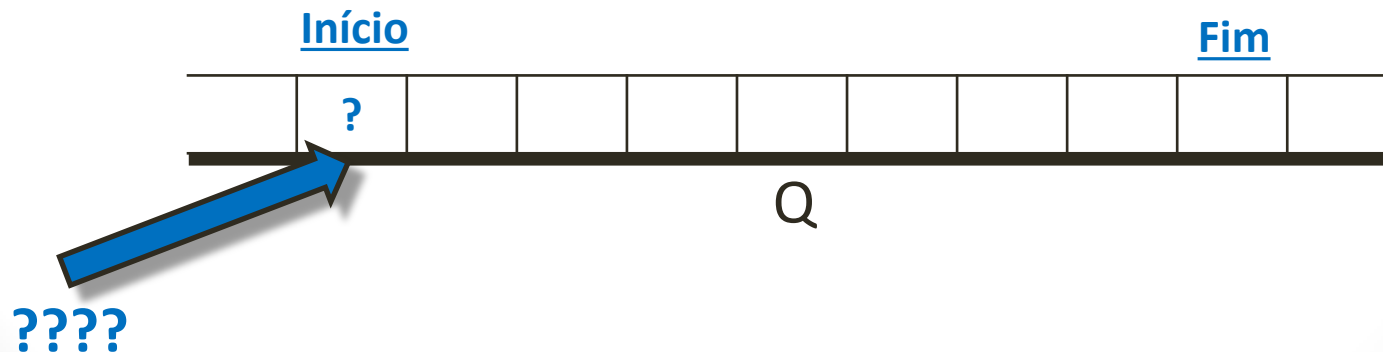


Listas lineares

- Algumas operações com o tipo abstrato de dados **fila**:

6. PrimeiroDaFila(Q);

- Retorna o primeiro elemento (sem removê-lo da estrutura) que está no início da fila Q

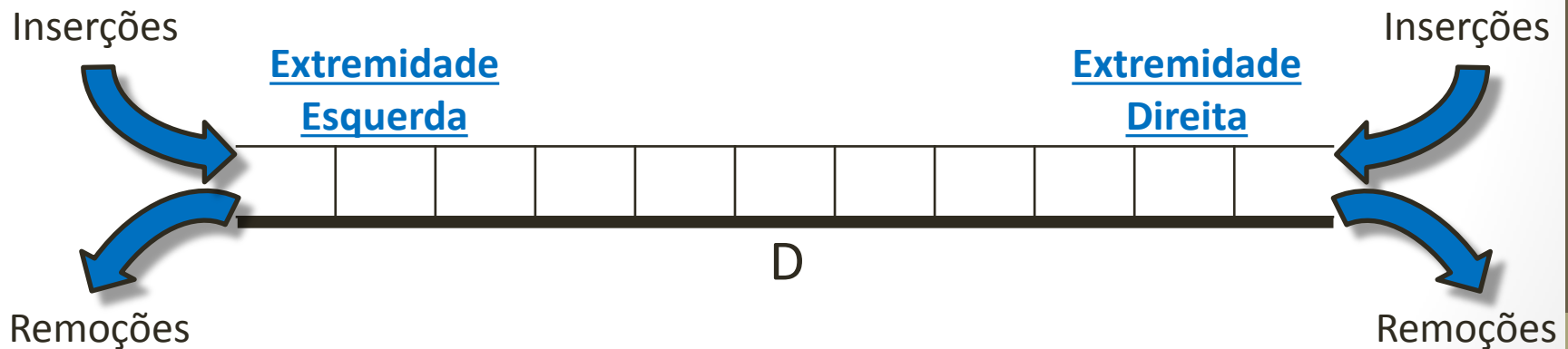


Listas lineares

- Alguns exemplos de aplicações de **fila**:
- Gerenciamento de processos
- Gerenciamento de recursos compartilhados
- Grafos (busca em largura)
- Buffer de dados
- Aplicações de filas em geral

Listas lineares

- A **fila dupla** é uma lista linear onde os elementos podem ser inseridos, removidos e acessados somente nas extremidades da lista (esquerda e direita)
- Também é conhecida como *deque* (*double-ended queue*)



Listas lineares

- Algumas operações com o tipo abstrato de dados **fila dupla**:

1. InicializaFilaDupla(D);

- Faz a fila dupla D ficar vazia

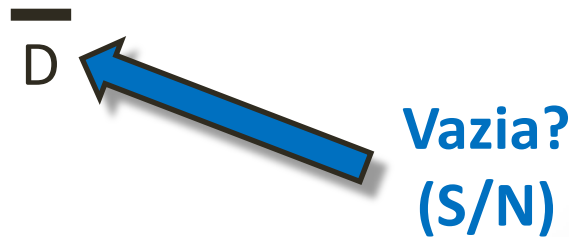
D

Listas lineares

- Algumas operações com o tipo abstrato de dados **fila dupla**:

2. FilaDuplaVazia(D);

- Retorna VERDADEIRO se a fila dupla D estiver vazia
- Retorna FALSO, caso contrário



Listas lineares

- Algumas operações com o tipo abstrato de dados **fila dupla**:

3. FilaDuplaCheia(D);

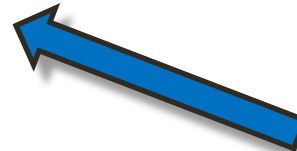
- Retorna VERDADEIRO se a fila dupla D estiver cheia
- Retorna FALSO, caso contrário

Extremidade
Esquerda

Extremidade
Direita



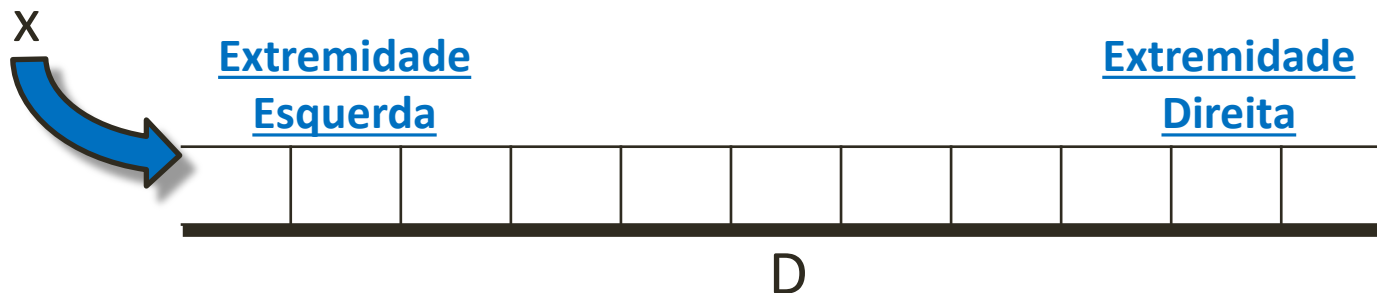
D



Cheia?
(S/N)

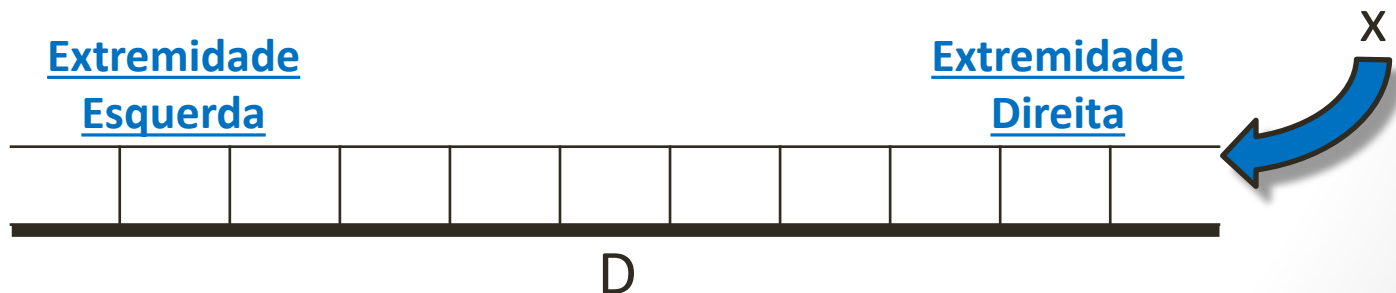
Listas lineares

- Algumas operações com o tipo abstrato de dados **fila dupla**:
4. `InserEsquerdaFilaDupla(D, x);`
- Insere o elemento `x` na extremidade esquerda da fila dupla `D`



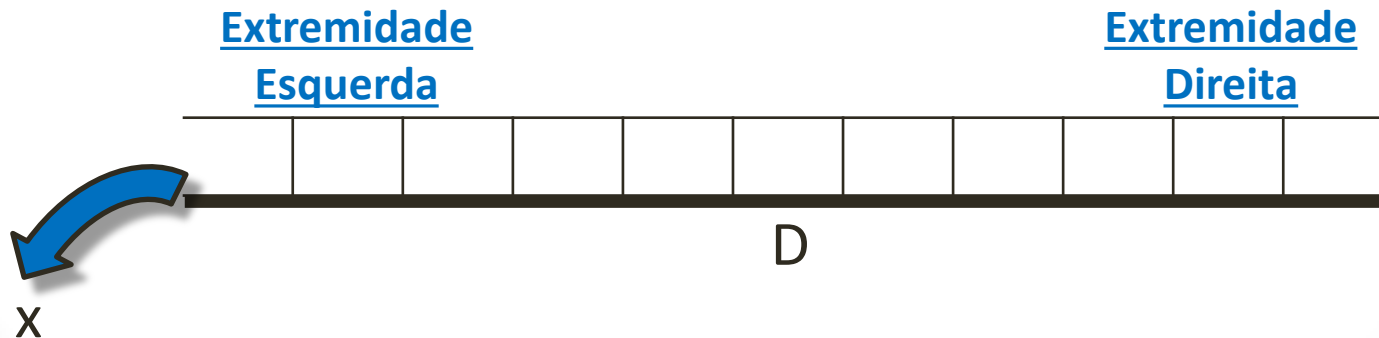
Listas lineares

- Algumas operações com o tipo abstrato de dados **fila dupla**:
5. `InserDireitaFilaDupla(D, x);`
- Insere o elemento x na extremidade direita da fila dupla D



Listas lineares

- Algumas operações com o tipo abstrato de dados **fila dupla**:
6. RemoveEsquerdaFilaDupla(D, x);
- Remove o elemento na extremidade esquerda da fila dupla D e o devolve em x

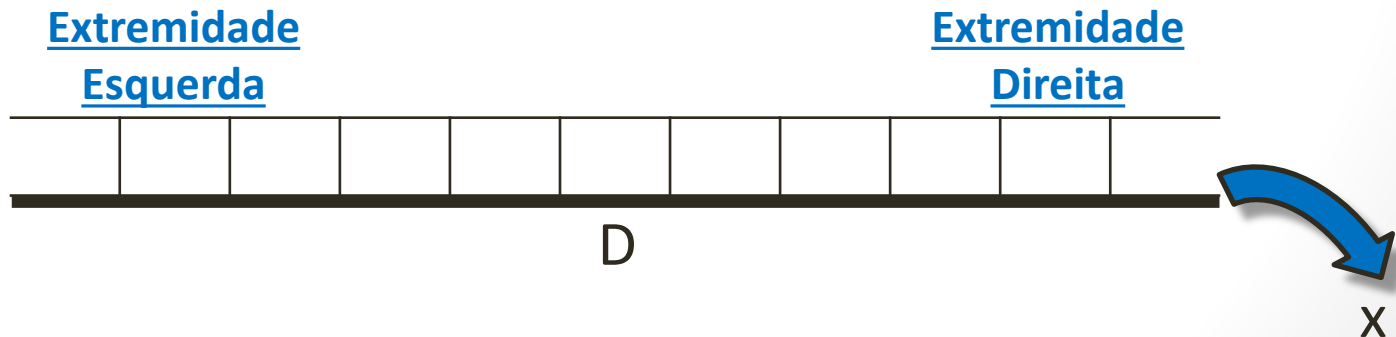


Listas lineares

- Algumas operações com o tipo abstrato de dados **fila dupla**:

7. RemoveDireitaFilaDupla(D, x);

- Remove o elemento na extremidade direita da fila dupla D e o devolve em x

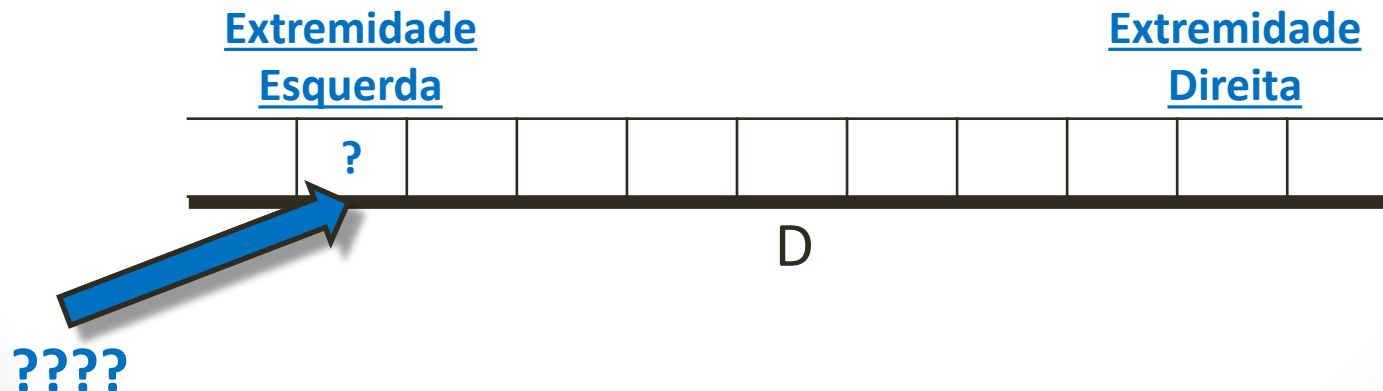


Listas lineares

- Algumas operações com o tipo abstrato de dados **fila dupla**:

8. ElementoEsquerdaFilaDupla(D);

- Retorna o elemento que está na extremidade esquerda da fila dupla D (sem removê-lo da estrutura)

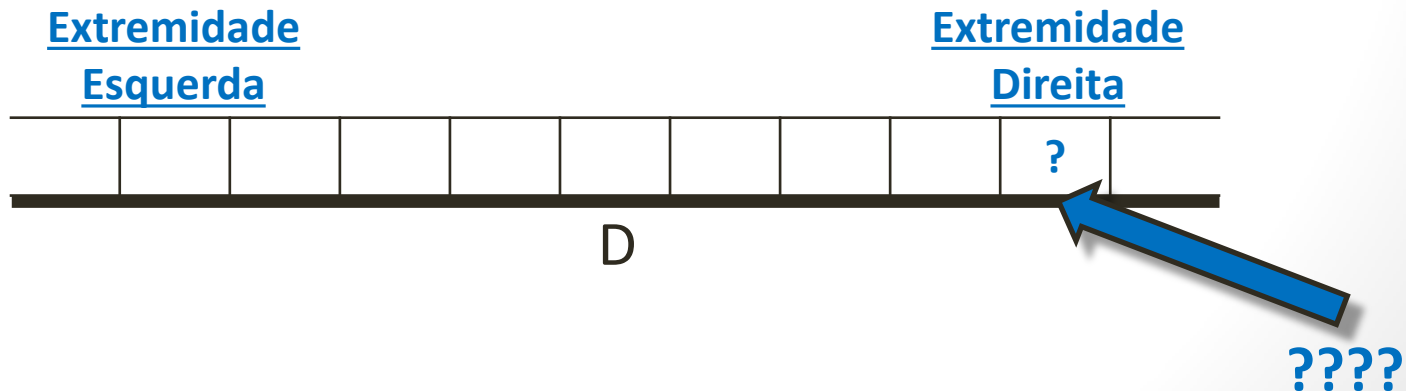


Listas lineares

- Algumas operações com o tipo abstrato de dados **fila dupla**:

9. ElementoDireitaFilaDupla(D);

- Retorna o elemento que está na extremidade direita da fila dupla D (sem removê-lo da estrutura)



Listas lineares

- Alguns exemplos de aplicações de **fila dupla**:
 - Pilhas e filas
 - Histórico de browser
 - Checagem de palíndromos
 - Gerenciamento de processos

Listas lineares em alocação estática

Pilha (alocação estática)

tipo TpPilha = registro

início

	0	1	2	3			TamMax
Elem :					...		

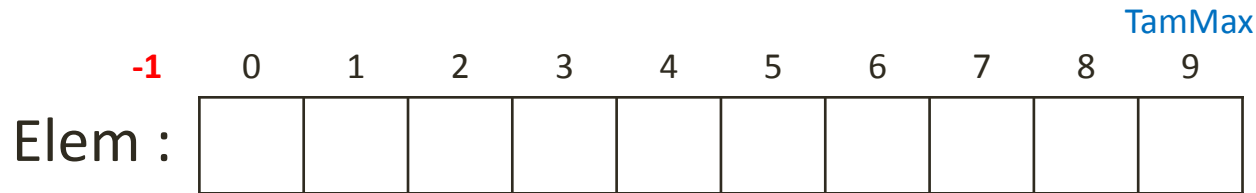
Topo : inteiro;

fim;

onde:

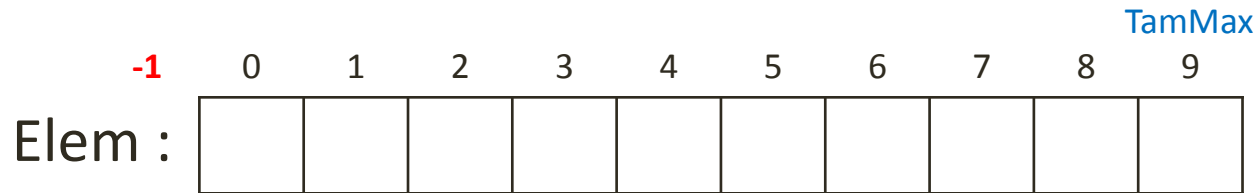
- Elem : vetor onde os elementos da pilha serão armazenados
- Topo : posição do último elemento na pilha
- TamMax é uma constante que define o tamanho físico do vetor Elem (que armazenará, no máximo, $\text{TamMax} + 1$ elementos)

Pilha (alocação estática)



- InicializaPilha(P);
- PilhaVazia(P);
- PilhaCheia(P);
- Empilha(P, x);
- Desempilha(P, x);
- ElementoDoTopo(P);

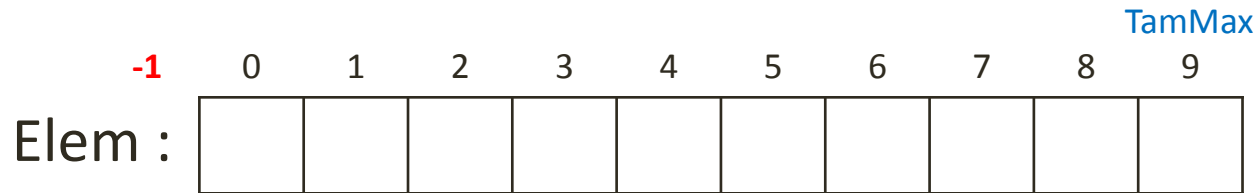
Pilha (alocação estática)



- InicializaPilha(P);

Complexidade:

Pilha (alocação estática)



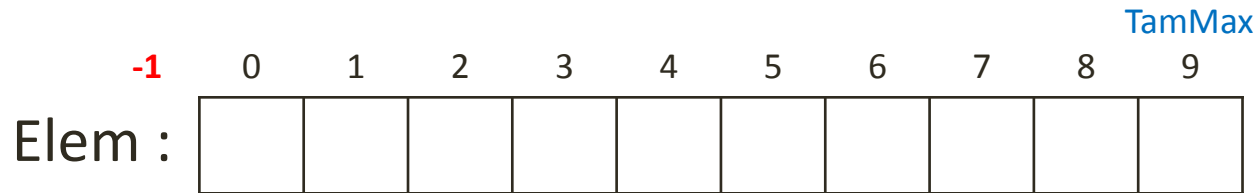
InicializaPilha(P);

P.Topo $\leftarrow$ -1;

Complexidade:

$\Theta(1)$

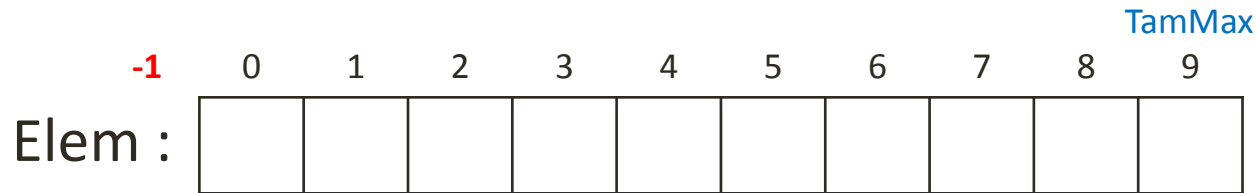
Pilha (alocação estática)



- PilhaVazia(P);

Complexidade:

Pilha (alocação estática)



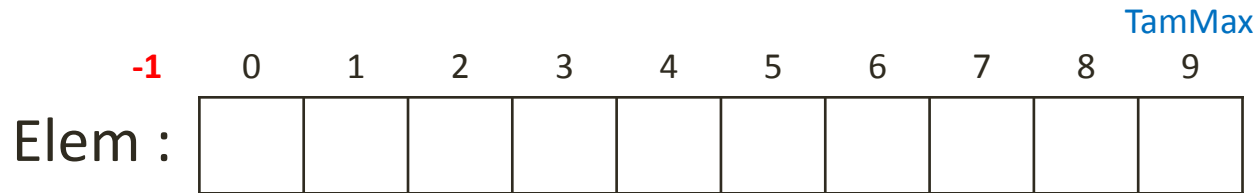
PilhaVazia(P);

PilhaVazia $\leftarrow$ P.Topo = -1;

Complexidade:

$\Theta(1)$

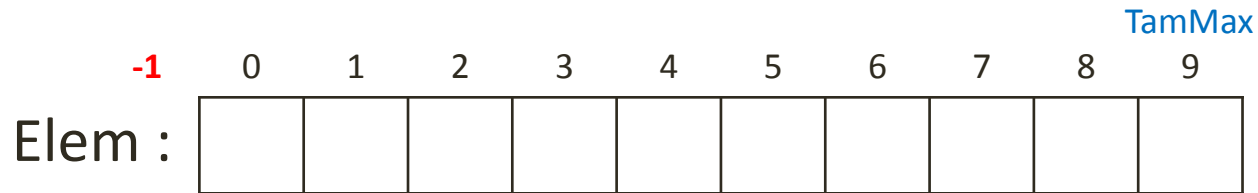
Pilha (alocação estática)



- PilhaCheia(P);

Complexidade:

Pilha (alocação estática)



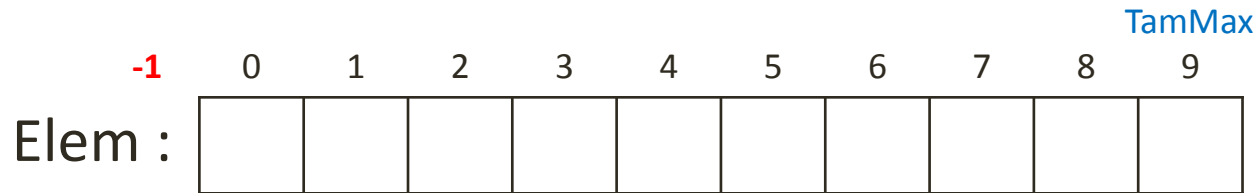
PilhaCheia(P);

PilhaCheia $\leftarrow$ P.Topo = TamMax;

Complexidade:

$\Theta(1)$

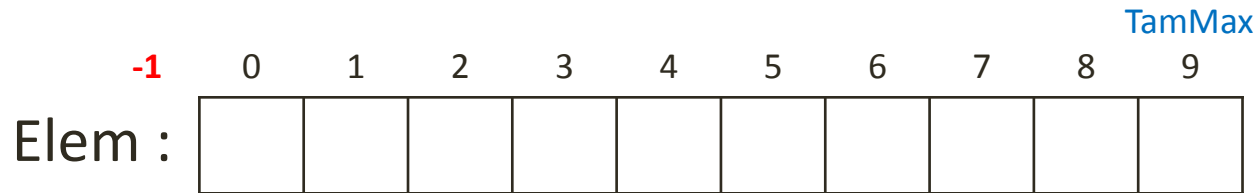
Pilha (alocação estática)



- Empilha(P, x);

Complexidade:

Pilha (alocação estática)



Empilha(P, x);

se NÃO PilhaCheia(P)

então

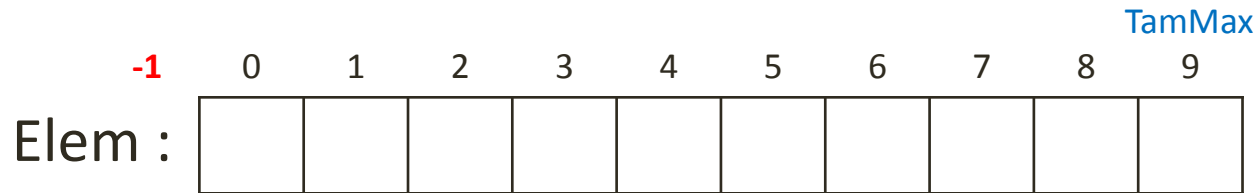
P.Topo $\leftarrow$ P.Topo + 1;

P.Elem[P.Topo] $\leftarrow$ x;

Complexidade:

$\Theta(1)$

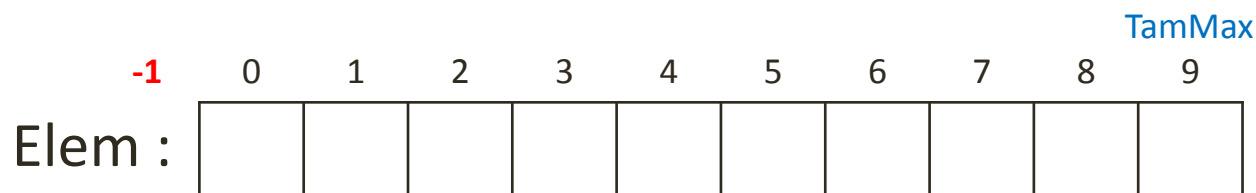
Pilha (alocação estática)



- Desempilha(P, x);

Complexidade:

Pilha (alocação estática)



Desempilha(P, x);

se NÃO PilhaVazia(P)

então

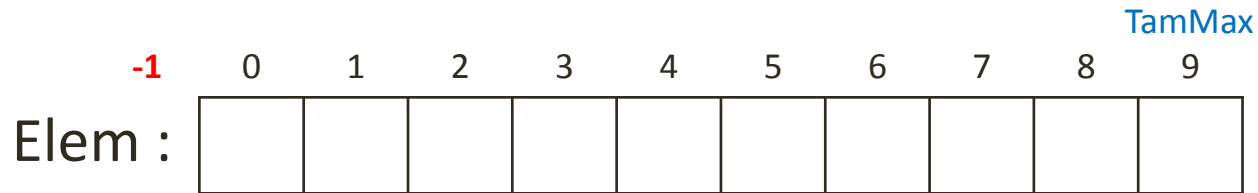
$x \leftarrow P.\text{Elem}[P.\text{Topo}];$

$P.\text{Topo} \leftarrow P.\text{Topo} - 1;$

Complexidade:

$\Theta(1)$

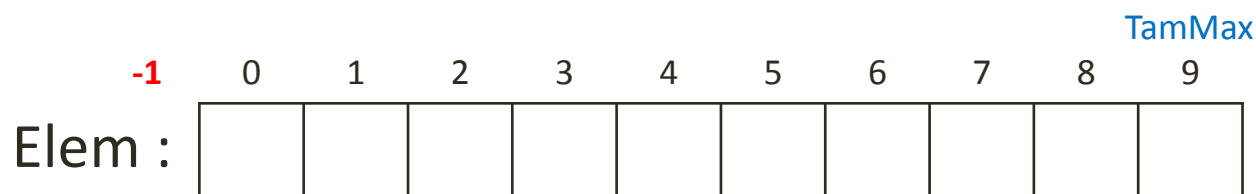
Pilha (alocação estática)



- ElementoDoTopo(P);

Complexidade:

Pilha (alocação estática)



ElementoDoTopo(P);

se NÃO PilhaVazia(P)

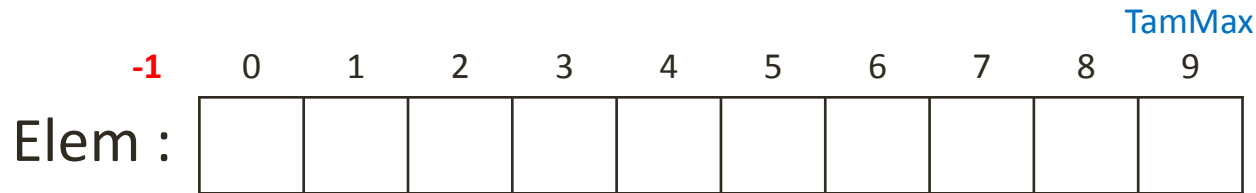
então | ElementoDoTopo $\leftarrow$ P.Elem[P.Topo];

senão | <erro>

Complexidade:

$\Theta(1)$

Pilha (alocação estática)



Função	Complexidade
InicializaPilha(P);	$\Theta(1)$
PilhaVazia(P);	$\Theta(1)$
PilhaCheia(P);	$\Theta(1)$
Empilha(P, x);	$\Theta(1)$
Desempilha(P, x);	$\Theta(1)$
ElementoDoTopo(P);	$\Theta(1)$

Fila (alocação estática)

tipo TpFila = registro

início

	0	1	2	3			TamMax
Elem :					...		

Início : inteiro;

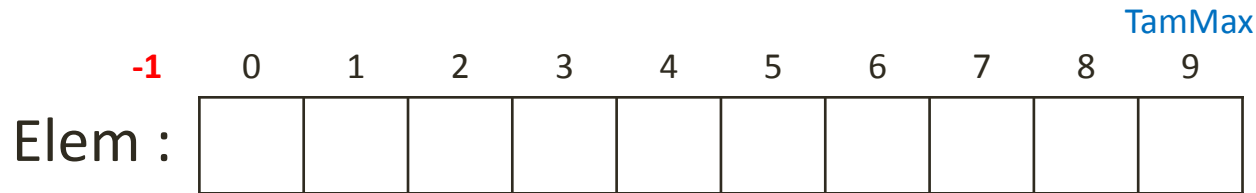
Fim : inteiro;

fim;

onde:

- Elem : vetor onde os elementos da fila serão armazenados
- Início : posição do último elemento removido da fila
- Fim : posição do último elemento inserido na fila

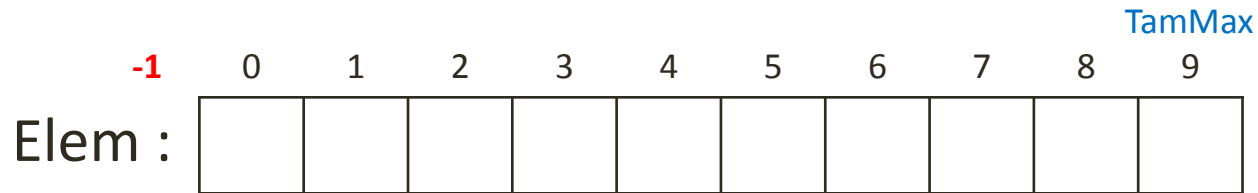
Fila (alocação estática)



- InicializaFila(Q);
- FilaVazia(Q);
- FilaCheia(Q);
- InsereFila(Q, x);
- RemoveFila(Q, x);
- PrimeiroDaFila(Q);

DeslocaFila(Q);

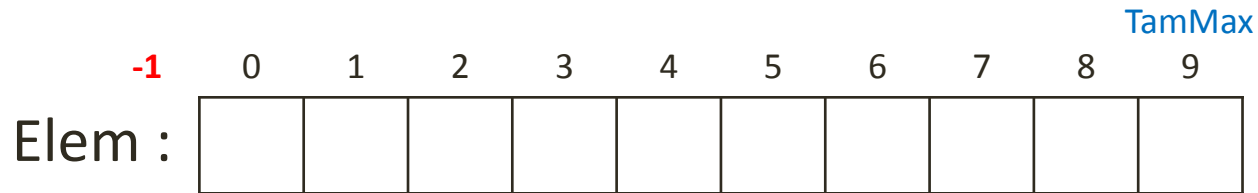
Fila (alocação estática)



- InicializaFila(Q);

Complexidade:

Fila (alocação estática)



InicializaFila(Q);

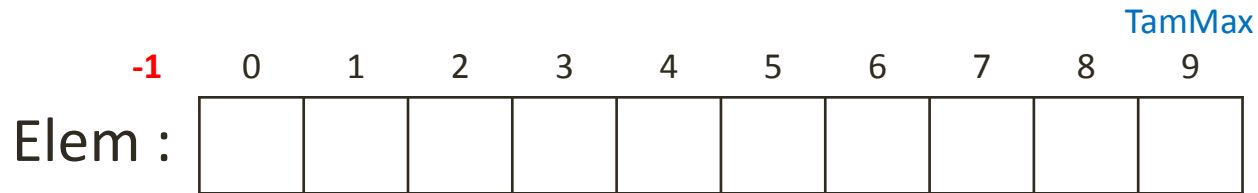
Q.Início $\leftarrow$ -1;

Q.Fim $\leftarrow$ -1;

Complexidade:

$\Theta(1)$

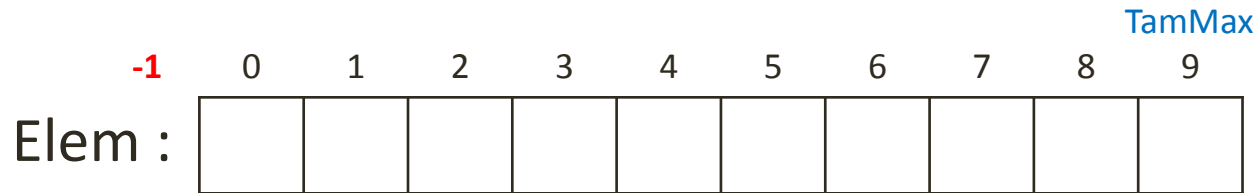
Fila (alocação estática)



- FilaVazia(Q);

Complexidade:

Fila (alocação estática)



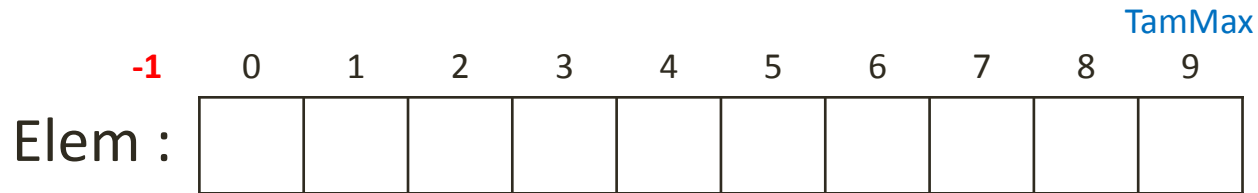
FilaVazia(Q);

FilaVazia $\leftarrow$ Q.Início = Q.Fim;

Complexidade:

$\Theta(1)$

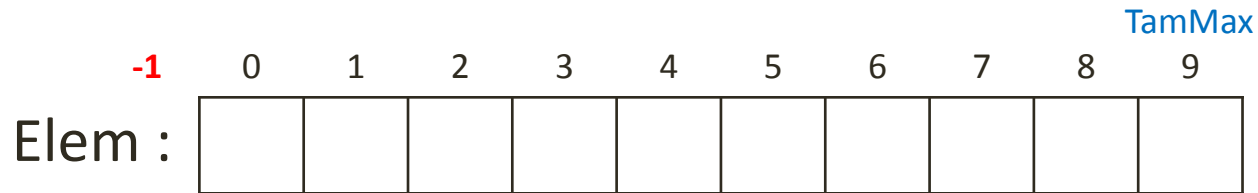
Fila (alocação estática)



- FilaCheia(Q);

Complexidade:

Fila (alocação estática)



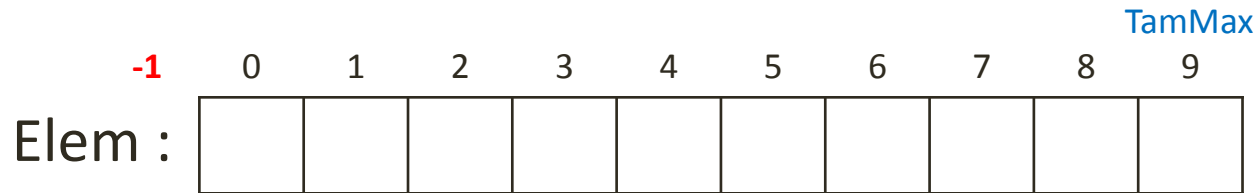
FilaCheia(Q);

FilaCheia $\leftarrow$ (Q.Início = -1) $\underline{\text{E}}$ (Q.Fim = TamMax);

Complexidade:

$\Theta(1)$

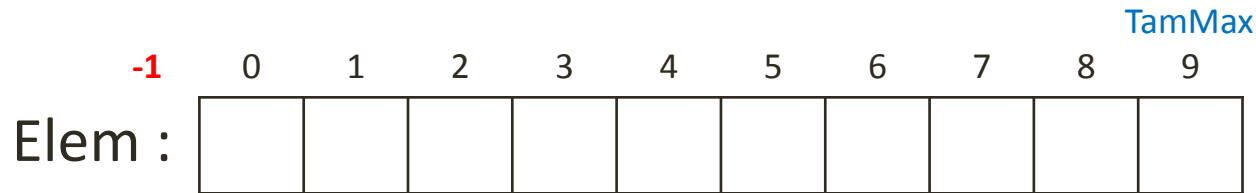
Fila (alocação estática)



- DeslocaFila(Q);

Complexidade:

Fila (alocação estática)



DeslocaFila(Q);

desloc $\leftarrow$ Q.Início + 1;

para i $\leftarrow$ (Q.Início + 1) até Q.Fim

faça | Q.Elem[i - desloc] $\leftarrow$ Q.Elem[i];

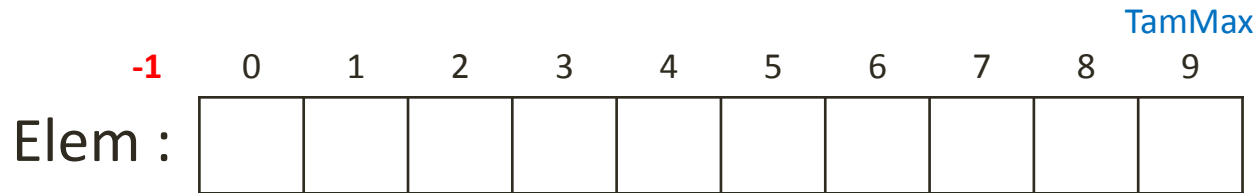
Q.Início $\leftarrow$ -1;

Q.Fim $\leftarrow$ Q.Fim - desloc;

Complexidade:

$O(n)$

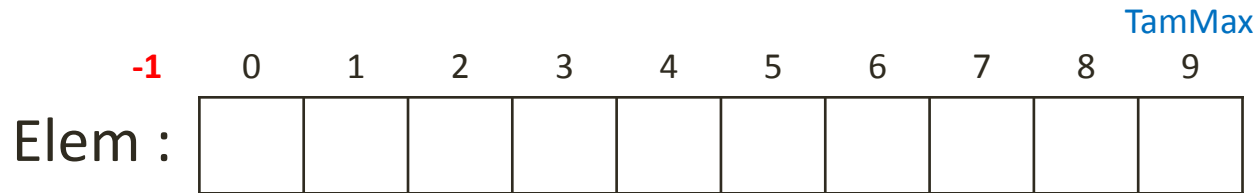
Fila (alocação estática)



- `InsereFila(Q, x);`

Complexidade:

Fila (alocação estática)



InsereFila(Q, x);

se NÃO FilaCheia(Q)

então

se Q.Fim = TamMax

então | DeslocaFila(Q);

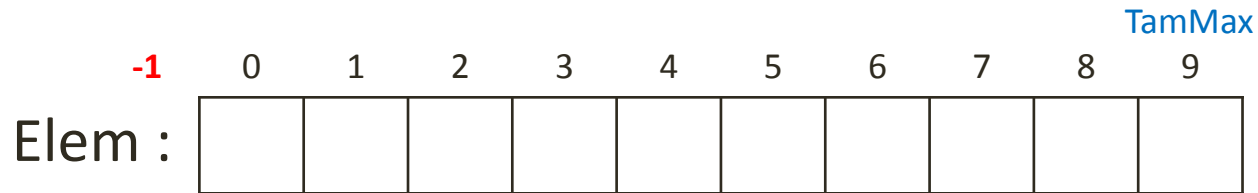
Q.Fim $\leftarrow$ Q.Fim + 1;

Q.Elem[Q.Fim] $\leftarrow$ x;

Complexidade:

$O(n)$

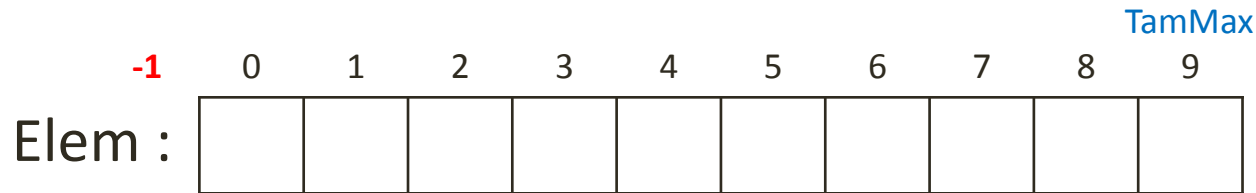
Fila (alocação estática)



- RemoveFila(Q, x);

Complexidade:

Fila (alocação estática)



RemoveFila(Q, x);

se NÃO FilaVazia(Q)

então

Q.Início $\leftarrow$ Q.Início + 1;

x $\leftarrow$ Q.Elem[Q.Início]

se FilaVazia(Q)

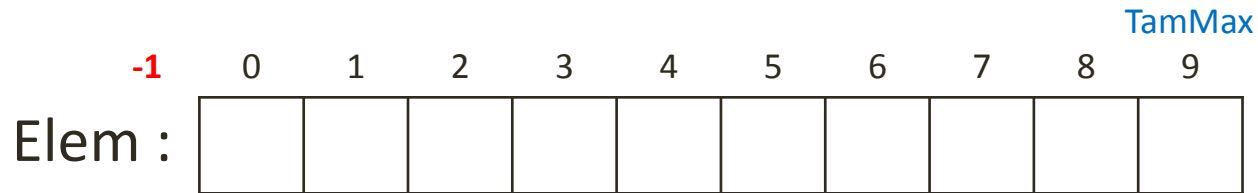
então

InicializaFila(Q);

Complexidade:

$\Theta(1)$

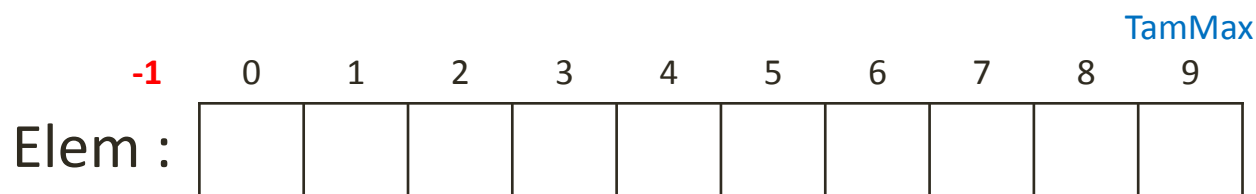
Fila (alocação estática)



- PrimeiroDaFila(Q);

Complexidade:

Fila (alocação estática)



PrimeiroDaFila(Q);

se NÃO FilaVazia(Q)

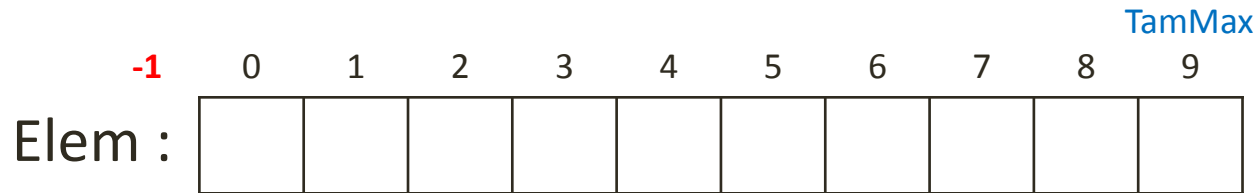
então | PrimeiroDaFila $\leftarrow$ Q.Elem[Q.Início+1];

senão | <erro>

Complexidade:

$\Theta(1)$

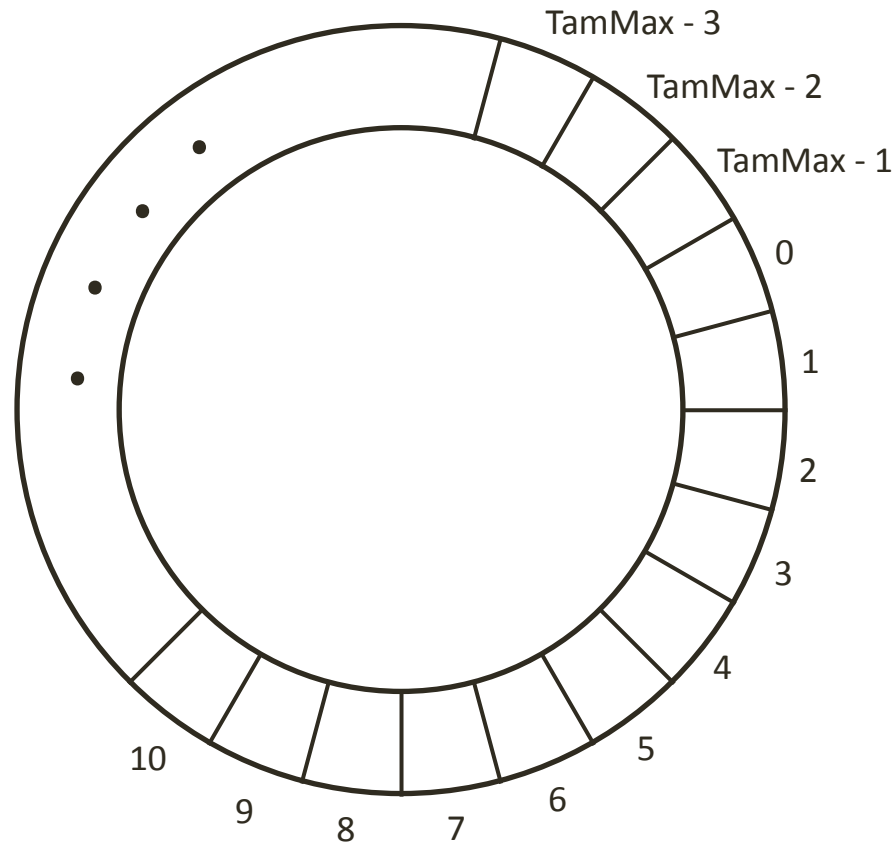
Fila (alocação estática)



Função	Complexidade
InicializaFila(Q);	$\Theta(1)$
FilaVazia(Q);	$\Theta(1)$
FilaCheia(Q);	$\Theta(1)$
DeslocaFila(Q);	$O(n)$
InsereFila(Q, x);	$O(n)$
RemoveFila(Q, x);	$\Theta(1)$
PrimeiroDaFila(Q);	$\Theta(1)$

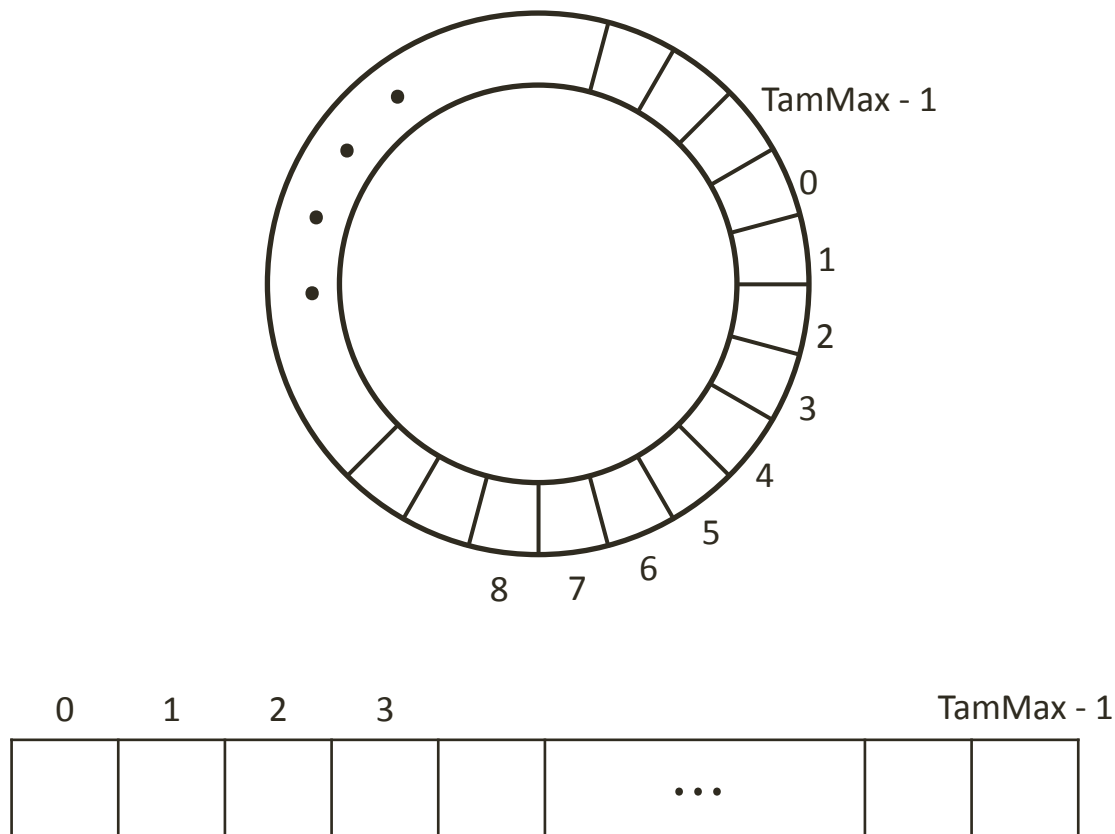
Fila (alocação estática)

- Considere a utilização de um vetor circular



Fila (alocação estática)

- Considere a utilização de um vetor circular



Fila (alocação estática)

tipo TpFilaCirc = registro

início

	0	1	2	3			TamMax - 1
Elem :						...	

Início : inteiro;

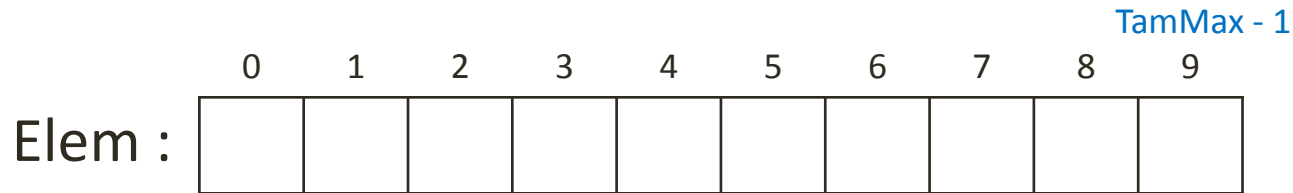
Fim : inteiro;

fim;

onde:

- Elem : vetor onde os elementos da fila serão armazenados
- Início : posição do último elemento removido da fila
- Fim : posição do último elemento inserido na fila

Fila (alocação estática)



- InicializaFilaCirc(Q);
- FilaCircVazia(Q);
- FilaCircCheia(Q);
- InsereFilaCirc(Q, x);
- RemoveFilaCirc(Q, x);
- PrimeiroDaFilaCirc(Q);

Fila (alocação estática)

Elem :

0	1	2	3	4	5	6	7	8	9

TamMax - 1

- Considere um número inteiro i que represente um índice para acessar os elementos no vetor Elem
- O índice i é um valor contido no intervalo $[0 \dots \text{TamMax} - 1]$

i	$i + 1$	
0	1	
1	2	
2	3	
3	4	
4	5	
5	6	
6	7	
7	8	
8	9	
9	<u>10</u>	

Fila (alocação estática)

Elem :

0	1	2	3	4	5	6	7	8	9

TamMax - 1

- Função resto
- Considere uma função resto(a , b) que calcula o resto da divisão de a por b

i	$i + 1$	<u>resto</u> ($i + 1, \text{TamMax}$)
0	1	1
1	2	2
2	3	3
3	4	4
4	5	5
5	6	6
6	7	7
7	8	8
8	9	9
9	<u>10</u>	<u>0</u>

Fila (alocação estática)

	0	1	2	3	4	5	6	7	8	9	TamMax - 1
Elem :											

- Função resto

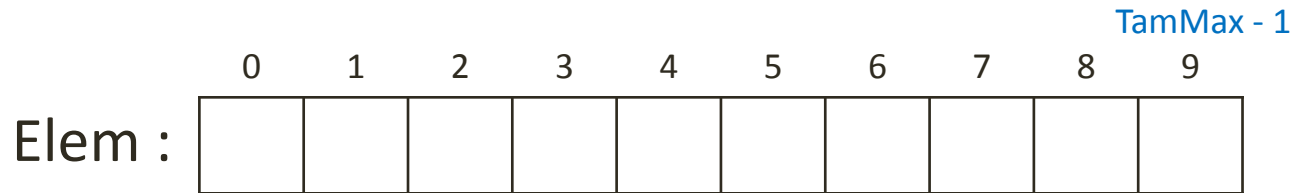
- resto($i + 1$, TamMax)

mapeia o incremento de i
no intervalo

[0 ... TamMax - 1]

i	$i + 1$	<u>resto</u> ($i + 1$, TamMax)
0	1	1
1	2	2
2	3	3
3	4	4
4	5	5
5	6	6
6	7	7
7	8	8
8	9	9
9	<u>10</u>	<u>0</u>

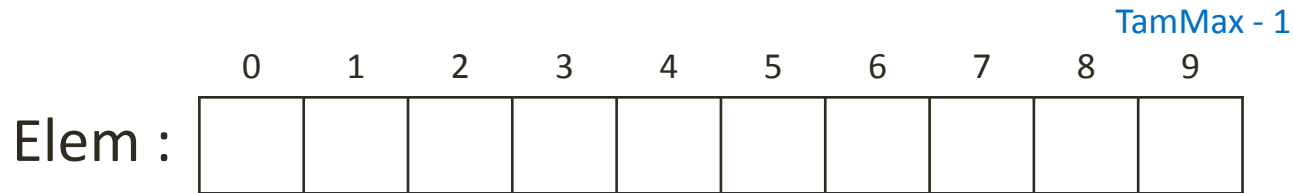
Fila (alocação estática)



- InicializaFilaCirc(Q);

Complexidade:

Fila (alocação estática)



InicializaFilaCirc(Q);

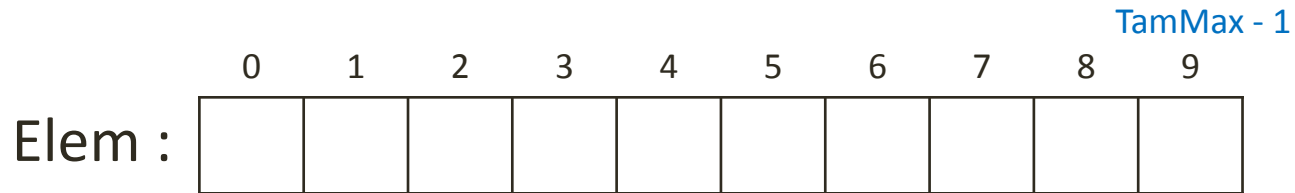
Q.Início $\leftarrow$ 0;

Q.Fim $\leftarrow$ 0;

Complexidade:

$\Theta(1)$

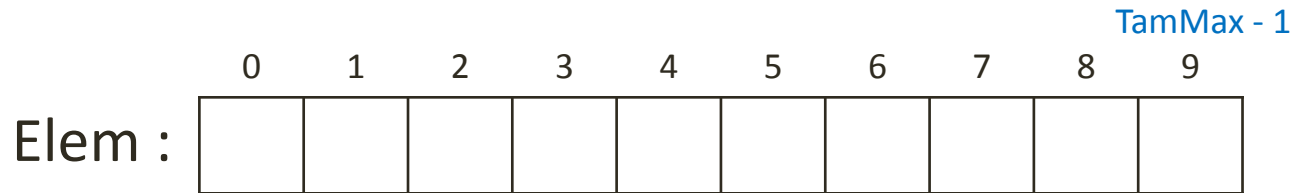
Fila (alocação estática)



- FilaCircVazia(Q);

Complexidade:

Fila (alocação estática)



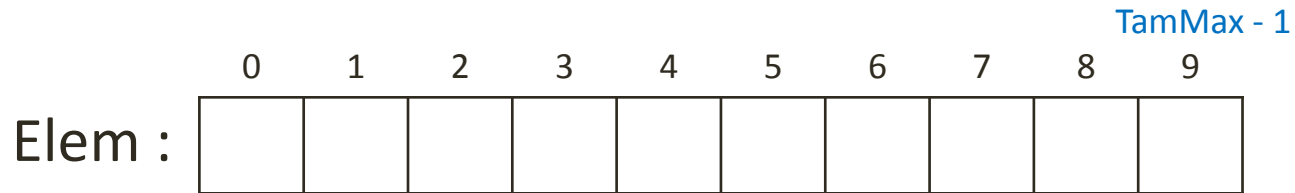
FilaCircVazia(Q);

FilaCircVazia $\leftarrow$ Q.Início = Q.Fim;

Complexidade:

$\Theta(1)$

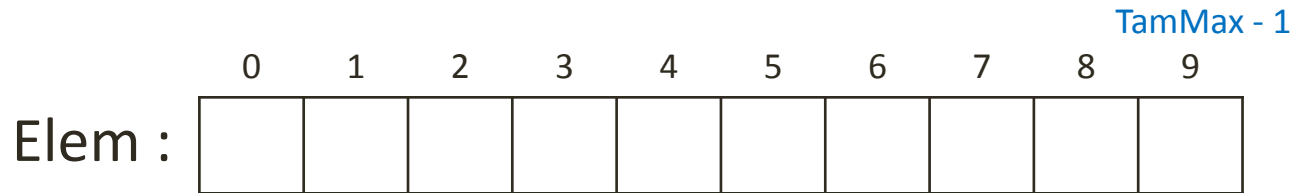
Fila (alocação estática)



- FilaCircCheia(Q);

Complexidade:

Fila (alocação estática)



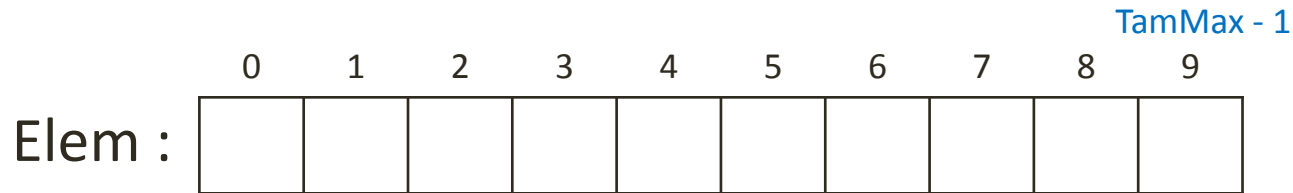
FilaCircCheia(Q);

FilaCircCheia $\leftarrow$ resto((Q.Fim + 1), TamMax) = Q.Início;

Complexidade:

$\Theta(1)$

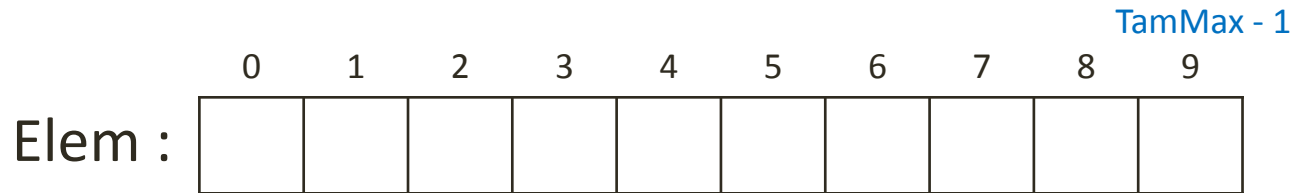
Fila (alocação estática)



- `InsereFilaCirc(Q, x);`

Complexidade:

Fila (alocação estática)



InsereFilaCirc(Q, x);

se NÃO FilaCircCheia(Q)

então

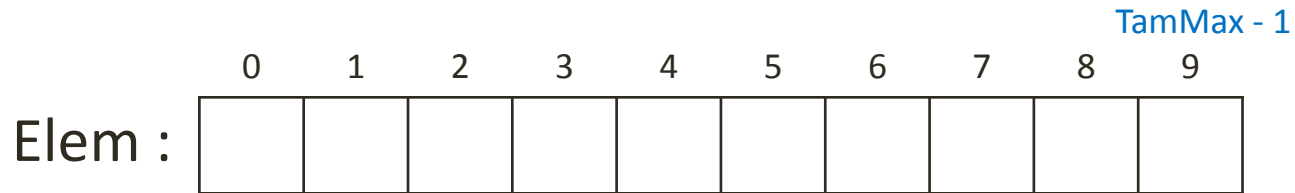
Q.Fim $\leftarrow$ resto((Q.Fim + 1), TamMax);

Q.Elem[Q.Fim] $\leftarrow$ x;

Complexidade:

$\Theta(1)$

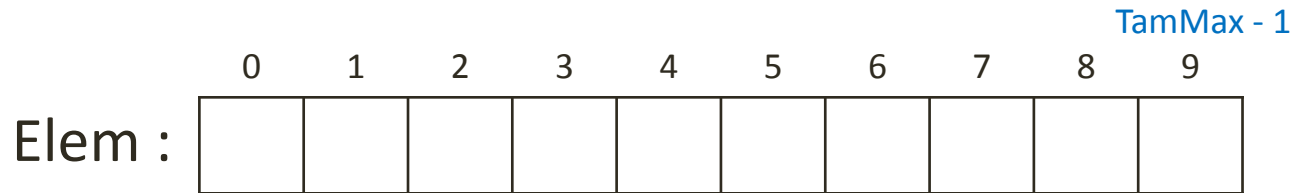
Fila (alocação estática)



- RemoveFilaCirc(Q, x);

Complexidade:

Fila (alocação estática)



RemoveFilaCirc(Q, x);

se NÃO FilaCircVazia(Q)

então

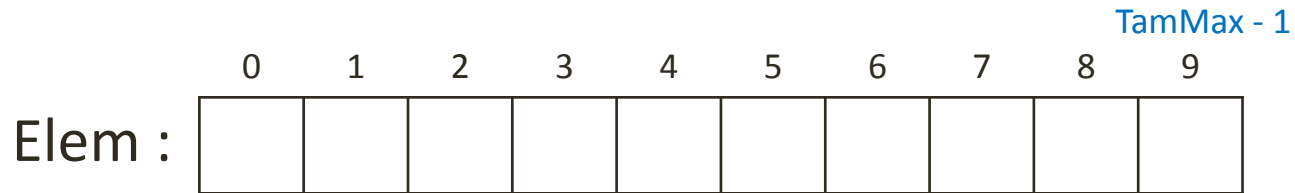
Q.Início $\leftarrow$ resto((Q.Início + 1), TamMax);

x $\leftarrow$ Q.Elem[Q.Início];

Complexidade:

$\Theta(1)$

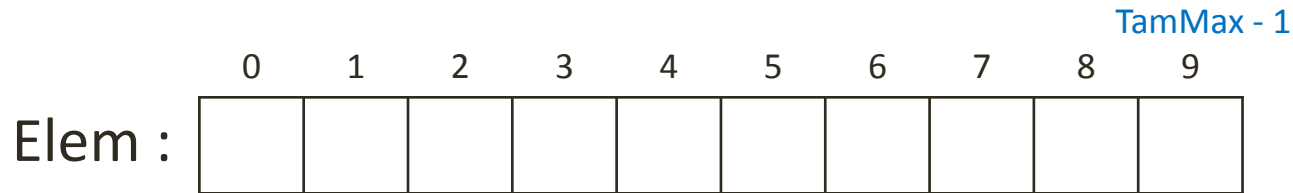
Fila (alocação estática)



- PrimeiroDaFilaCirc(Q);

Complexidade:

Fila (alocação estática)



PrimeiroDaFilaCirc(Q);

se NÃO FilaCircVazia(Q)

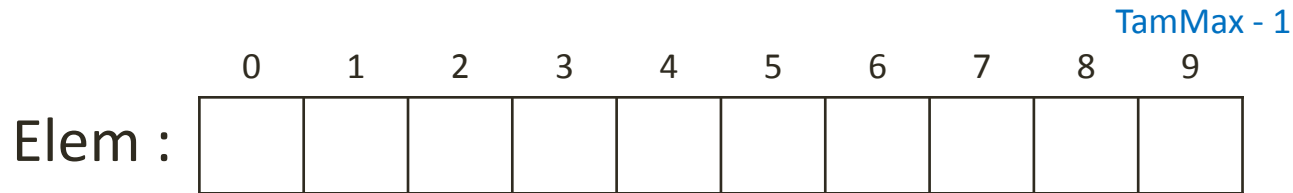
então | PrimeiroDaFilaCirc $\leftarrow$ Q.Elem[resto((Q.Início + 1), TamMax)];

senão | <erro>

Complexidade:

$\Theta(1)$

Fila (alocação estática)



Função	Complexidade
InicializaFilaCirc(Q);	$\Theta(1)$
FilaCircVazia(Q);	$\Theta(1)$
FilaCircCheia(Q);	$\Theta(1)$
InsererFilaCirc(Q, x);	$\Theta(1)$
RemoveFilaCirc(Q, x);	$\Theta(1)$
PrimeiroDaFilaCirc(Q);	$\Theta(1)$

Fila dupla (alocação estática)

tipo TpFilaDupla = registro

início

	0	1	2	3			TamMax
Elem :					...		

Esq : inteiro;

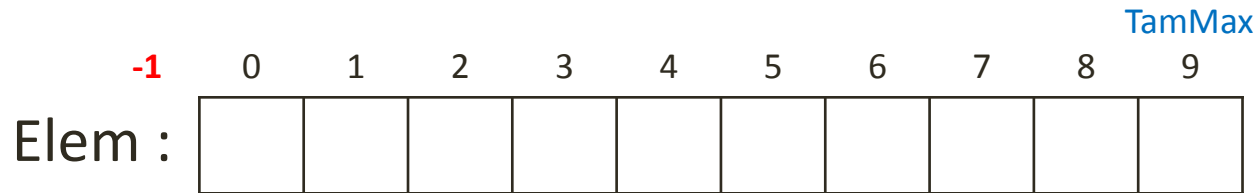
Dir : inteiro;

fim;

onde:

- Elem : vetor onde os elementos da fila dupla serão armazenados
- Esq : posição anterior ao elemento na extremidade esquerda
- Dir : posição do elemento na extremidade direita

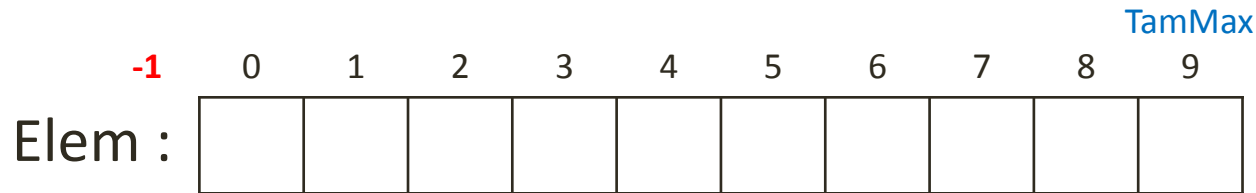
Fila dupla (alocação estática)



- InicializaFilaDupla(D);
- FilaDuplaVazia(D);
- FilaDuplaCheia(D);
- InsereEsquerdaFilaDupla(D, x);
- InsereDireitaFilaDupla(D, x);

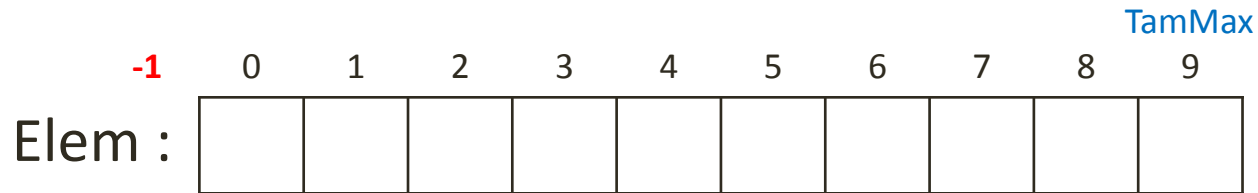
DeslocaFilaDupla(D);

Fila dupla (alocação estática)



- RemoveEsquerdaFilaDupla(D, x);
- RemoveDireitaFilaDupla(D, x);
- ElementoEsquerdaFilaDupla(D);
- ElementoDireitaFilaDupla(D);

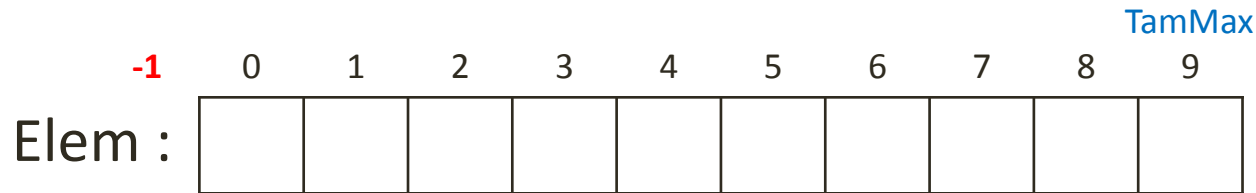
Fila dupla (alocação estática)



- InicializaFilaDupla(D);

Complexidade:

Fila dupla (alocação estática)



InicializaFilaDupla(D);

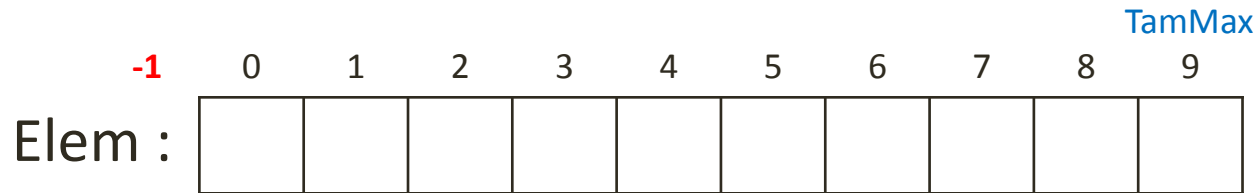
$D.Esq \leftarrow \lfloor TamMax / 2 \rfloor;$

$D.Dir \leftarrow D.Esq;$

Complexidade:

$\Theta(1)$

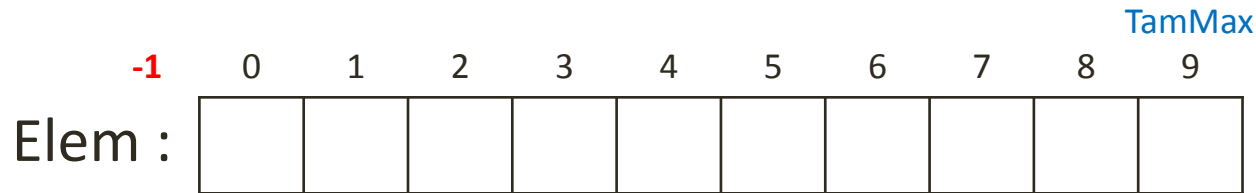
Fila dupla (alocação estática)



- FilaDuplaVazia(D);

Complexidade:

Fila dupla (alocação estática)



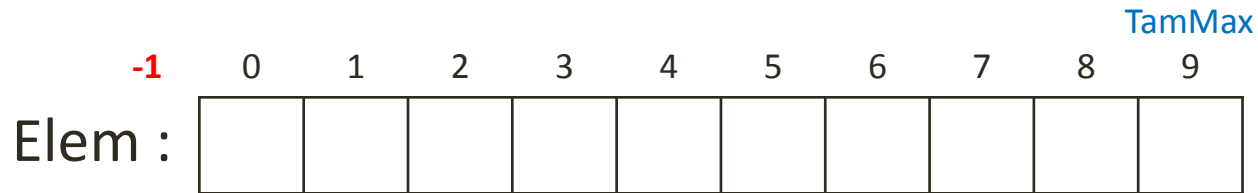
FilaDuplaVazia(D);

FilaDuplaVazia $\leftarrow$ D.Esq = D.Dir;

Complexidade:

$\Theta(1)$

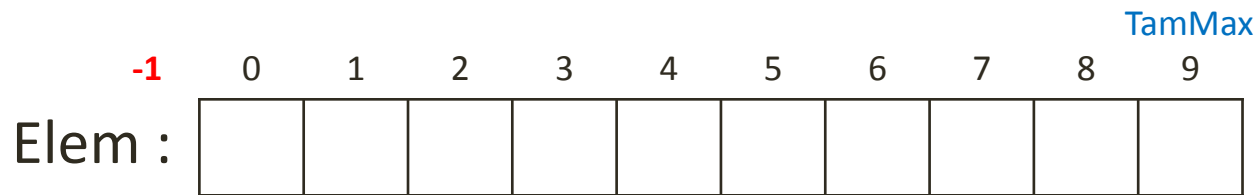
Fila dupla (alocação estática)



- FilaDuplaCheia(D);

Complexidade:

Fila dupla (alocação estática)



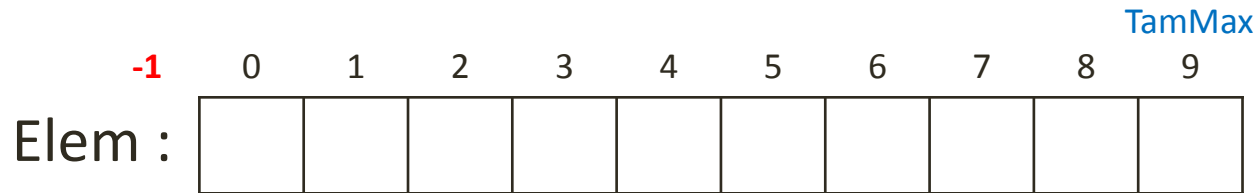
FilaDuplaCheia(D);

FilaDuplaCheia $\leftarrow$ (D.Esq = -1) $\underline{\text{E}}$ (D.Dir = TamMax);

Complexidade:

$\Theta(1)$

Fila dupla (alocação estática)



- DeslocaFilaDupla(D);

Complexidade:

Fila dupla (alocação estática)

DeslocaFilaDupla(D);

se D.Dir = TamMax

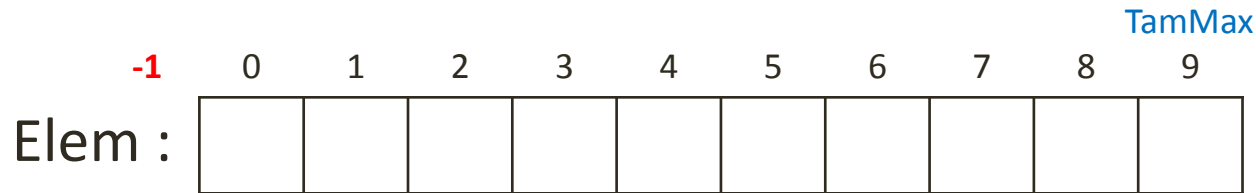
então | desloc $\leftarrow \lceil (D.Esq + 1) / 2 \rceil$;
|
| para i $\leftarrow (D.Esq + 1)$ até D.Dir
| faça | D.Elem[i - desloc] $\leftarrow$ D.Elem[i];
| D.Esq $\leftarrow$ D.Esq - desloc;
| D.Dir $\leftarrow$ D.Dir - desloc;

senão | desloc $\leftarrow \lceil (TamMax - D.Dir) / 2 \rceil$;
| para i $\leftarrow$ D.Dir até (D.Esq + 1) passo -1
| faça | D.Elem[i + desloc] $\leftarrow$ D.Elem[i];
| D.Esq $\leftarrow$ D.Esq + desloc;
| D.Dir $\leftarrow$ D.Dir + desloc;

Complexidade:

$O(n)$

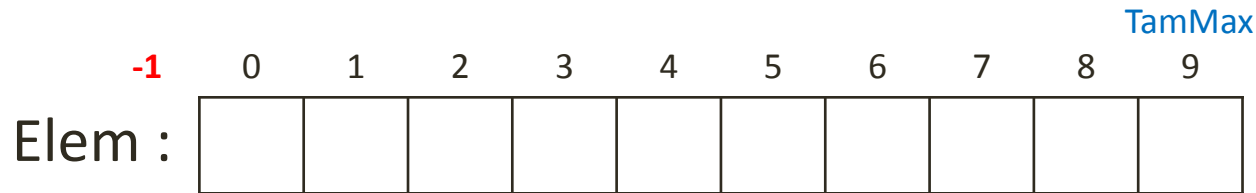
Fila dupla (alocação estática)



- `InseresquerdaFilaDupla(D, x);`

Complexidade:

Fila dupla (alocação estática)



`InserEsquerdaFilaDupla(D, x);`

`se NÃO FilaDuplaCheia(D)`

`então`

`se D.Esq = -1`

`então DeslocaFilaDupla(D);`

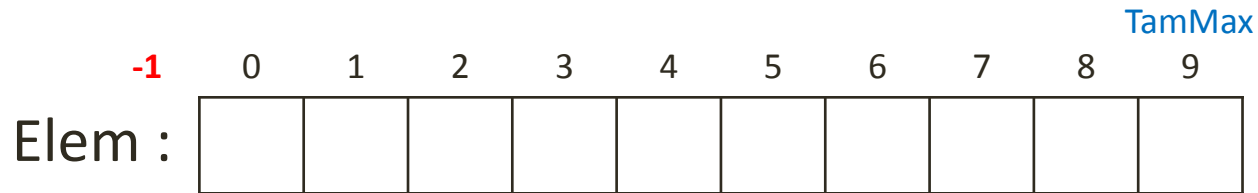
`D.Elem[D.Esq] ← x;`

`D.Esq ← D.Esq - 1;`

Complexidade:

$O(n)$

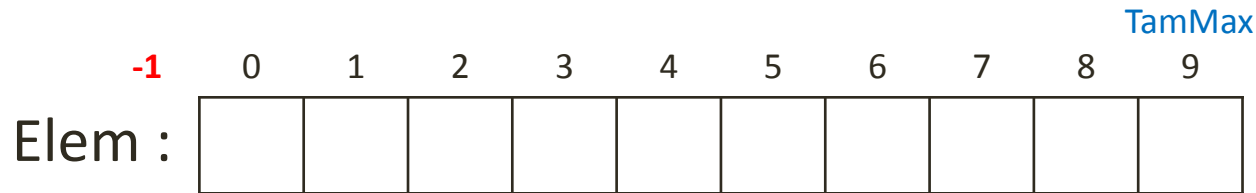
Fila dupla (alocação estática)



- InsereDireitaFilaDupla(D, x);

Complexidade:

Fila dupla (alocação estática)



InserDireitaFilaDupla(D, x);

se NÃO FilaDuplaCheia(D)

então

se D.Dir = TamMax

então | DeslocaFilaDupla(D);

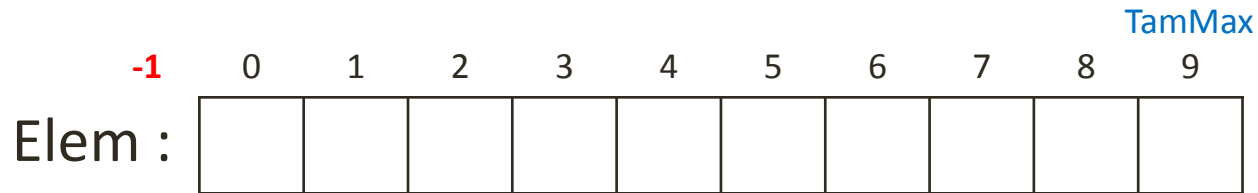
D.Dir $\leftarrow$ D.Dir + 1;

D.Elem[D.Dir] $\leftarrow$ x;

Complexidade:

$O(n)$

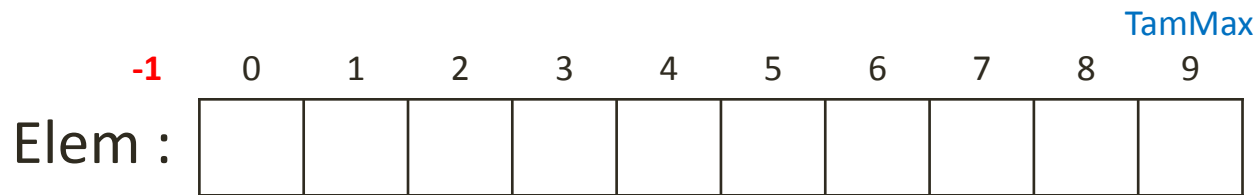
Fila dupla (alocação estática)



- RemoveEsquerdaFilaDupla(D, x);

Complexidade:

Fila dupla (alocação estática)



RemoveEsquerdaFilaDupla(D, x);

se NÃO FilaDuplaVazia(D)

então

D.Esq $\leftarrow$ D.Esq + 1;

x $\leftarrow$ D.Elem[D.Esq];

se FilaDuplaVazia(D)

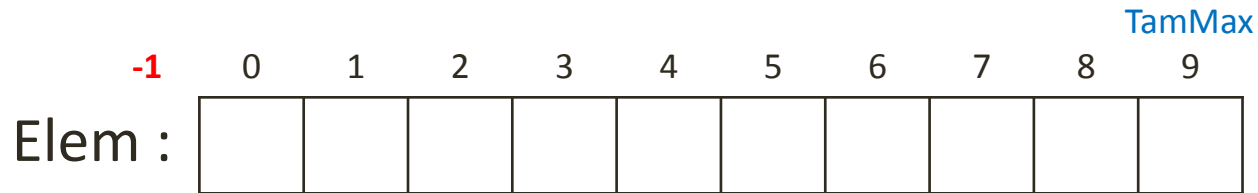
então

InicializaFilaDupla(D);

Complexidade:

$\Theta(1)$

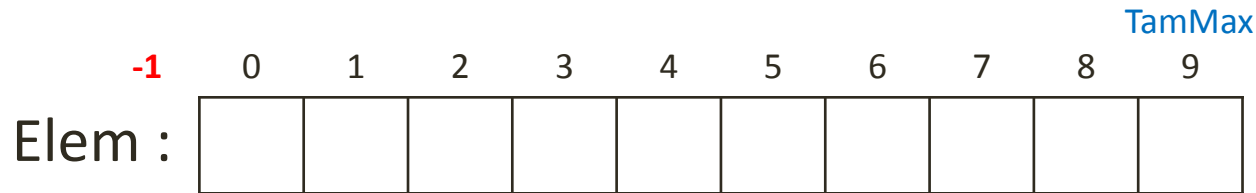
Fila dupla (alocação estática)



- RemoveDireitaFilaDupla(D, x);

Complexidade:

Fila dupla (alocação estática)



RemoveDireitaFilaDupla(D, x);

se NÃO FilaDuplaVazia(D)

então

$x \leftarrow D.Elem[D.Dir];$

$D.Dir \leftarrow D.Dir - 1;$

se FilaDuplaVazia(D)

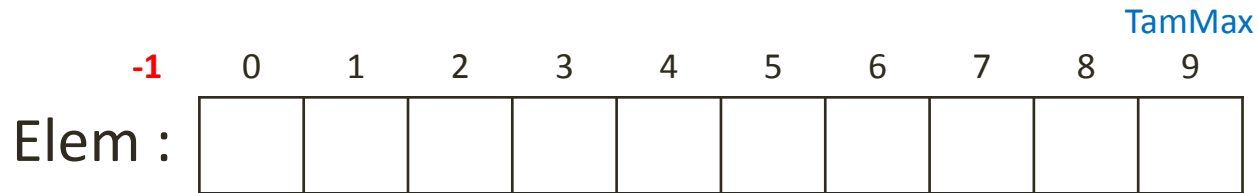
então

InicializaFilaDupla(D);

Complexidade:

$\Theta(1)$

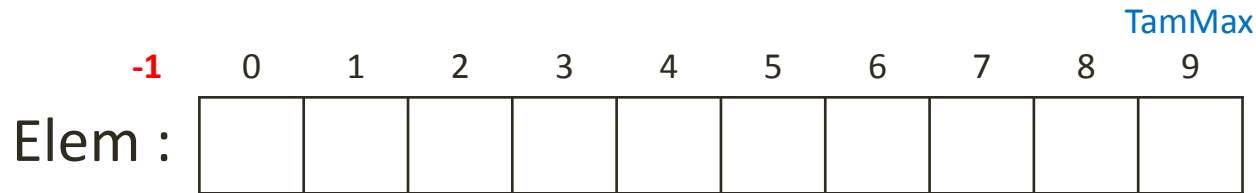
Fila dupla (alocação estática)



- ElementoEsquerdaFilaDupla(D);

Complexidade:

Fila dupla (alocação estática)



ElementoEsquerdaFilaDupla(D);

se NÃO FilaDuplaVazia(D)

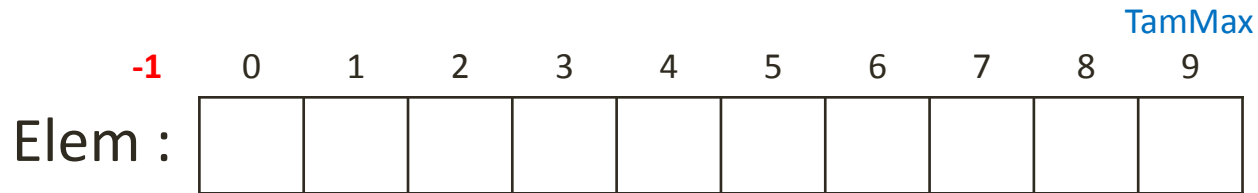
então | ElementoEsquerdaFilaDupla $\leftarrow$ D.Elem[D.Esq+1];

senão | <erro>

Complexidade:

$\Theta(1)$

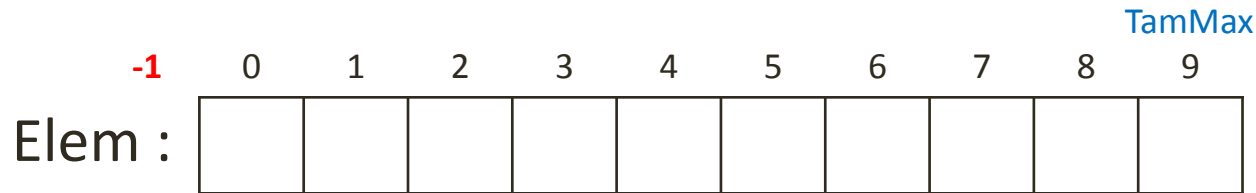
Fila dupla (alocação estática)



- ElementoDireitaFilaDupla(D);

Complexidade:

Fila dupla (alocação estática)



ElementoDireitaFilaDupla(D);

se NÃO FilaDuplaVazia(D)

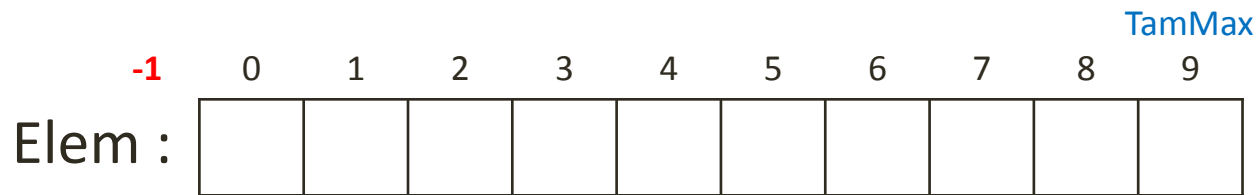
então | ElementoDireitaFilaDupla ← D.Elem[D.Dir];

senão | <erro>

Complexidade:

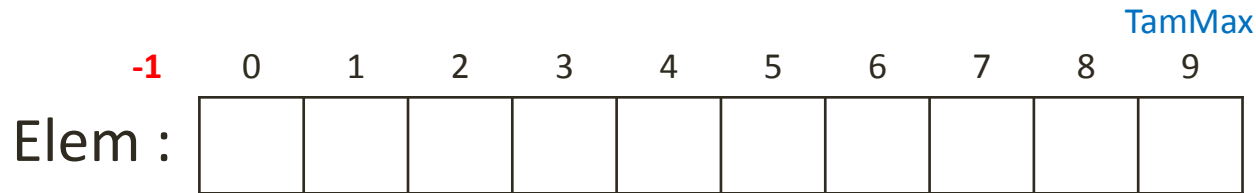
$\Theta(1)$

Fila dupla (alocação estática)



Função	Complexidade
InicializaFilaDupla(D);	$\Theta(1)$
FilaDuplaVazia(D);	$\Theta(1)$
FilaDuplaCheia(D);	$\Theta(1)$
DeslocaFilaDupla(D);	$O(n)$
InseresEsquerdaFilaDupla(D, x);	$O(n)$
InseresDireitaFilaDupla(D, x);	$O(n)$

Fila dupla (alocação estática)



Função	Complexidade
RemoveEsquerdaFilaDupla(D, x);	$\Theta(1)$
RemoveDireitaFilaDupla(D, x);	$\Theta(1)$
ElementoEsquerdaFilaDupla(D, x);	$\Theta(1)$
ElementoDireitaFilaDupla(D, x);	$\Theta(1)$

Listas lineares em alocação dinâmica

Listas lineares em alocação dinâmica

- Registro

tipo TpNó = registro
início

info : tipo;

prox : ^TpNó;

fim;



onde:

- info : campo de informação (pode ter um ou mais campos)
- prox : é um ponteiro do tipo TpNó

Pilha (alocação dinâmica)

tipo TpPilha = registro

início

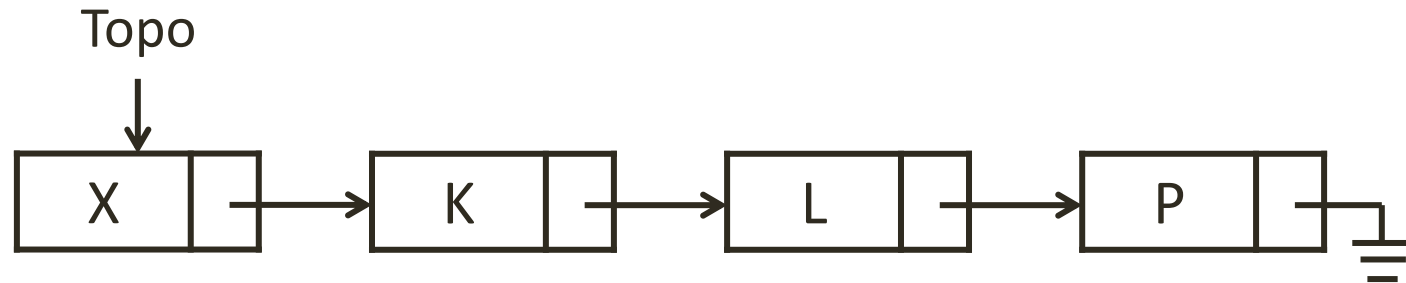
Topo : ^TpNó;

fim;

onde:

- Topo : ponteiro que aponta para o elemento que está no topo da pilha

Pilha (alocação dinâmica)



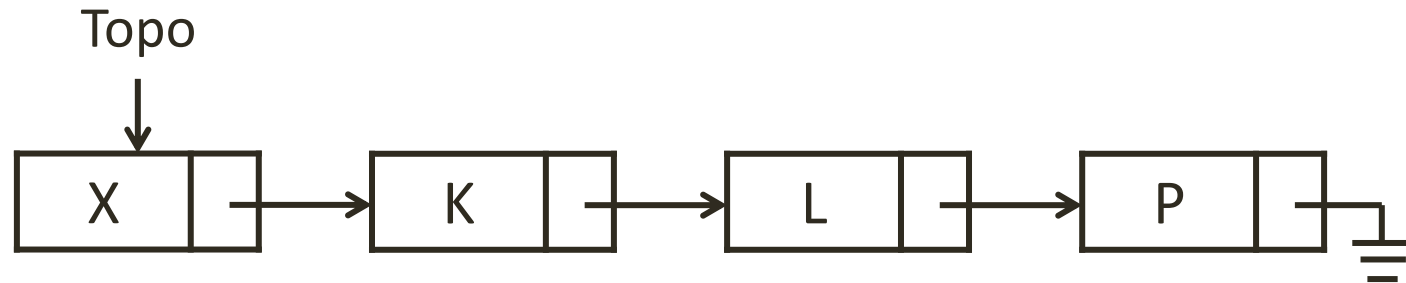
- InicializaPilha(P);
- PilhaVazia(P);
- Empilha(P, x);
- Desempilha(P, x);
- ElementoDoTopo(P);

Pilha (alocação dinâmica)

- InicializaPilha(P);

Complexidade:

Pilha (alocação dinâmica)



InicializaPilha(P);

P.Topo $\leftarrow$ nil;

Complexidade:

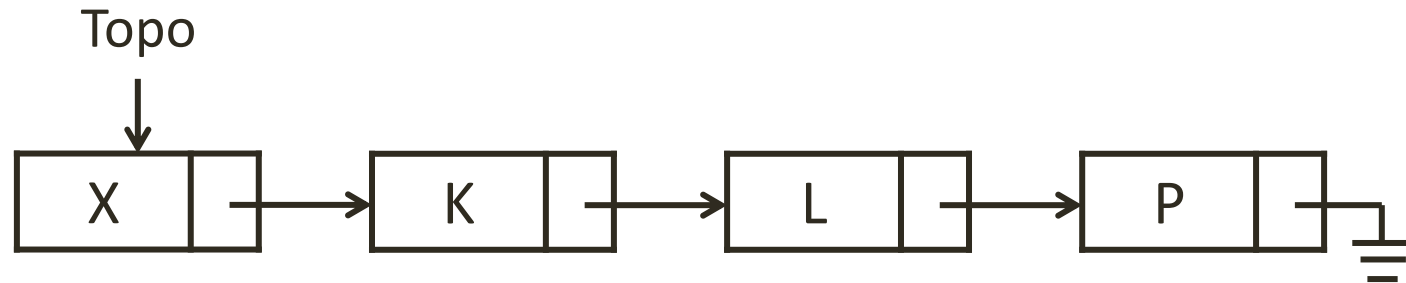
$\Theta(1)$

Pilha (alocação dinâmica)

- PilhaVazia(P);

Complexidade:

Pilha (alocação dinâmica)



`PilhaVazia(P);`

`PilhaVazia ← P.Topo = nil;`

Complexidade:

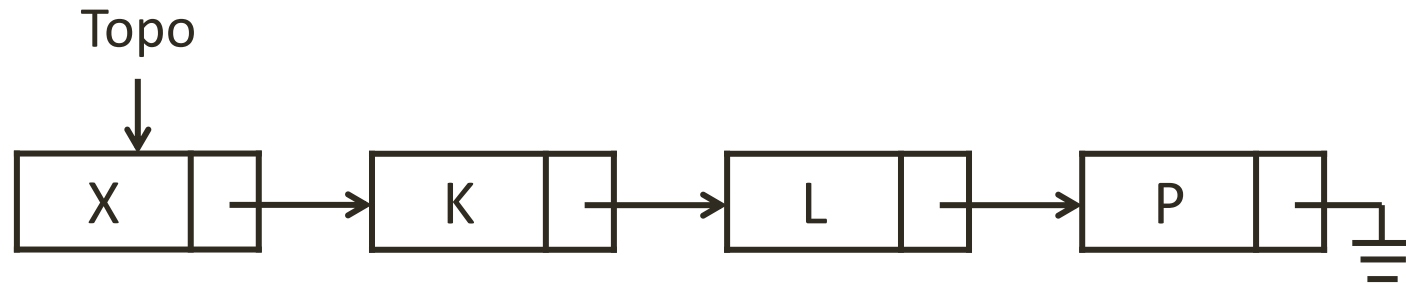
$\Theta(1)$

Pilha (alocação dinâmica)

- Empilha(P, x);

Complexidade:

Pilha (alocação dinâmica)



Empilha(P, x);

novo(Q);

Q^.info $\leftarrow$ x;

Q^.prox $\leftarrow$ P.Topo;

P.Topo $\leftarrow$ Q;

Complexidade:

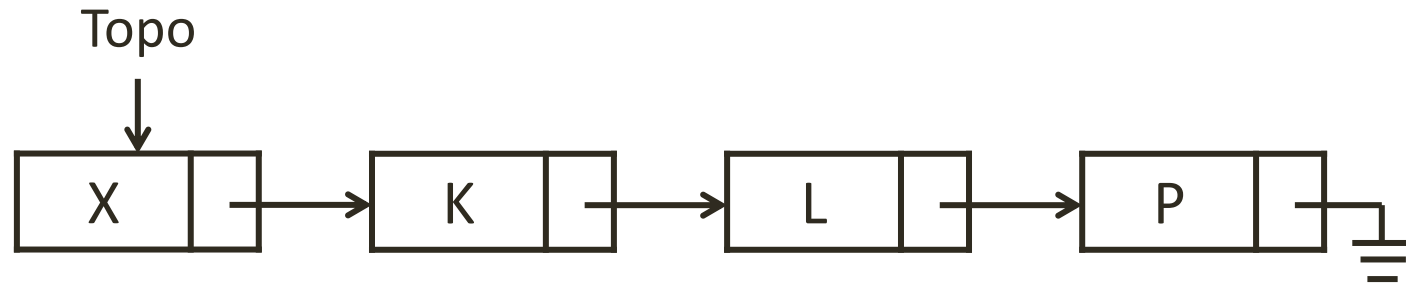
$\Theta(1)$

Pilha (alocação dinâmica)

- Desempilha(P, x);

Complexidade:

Pilha (alocação dinâmica)



Desempilha(P, x);

se NÃO PilhaVazia(P)

então

Q ← P.Topo;

x ← Q^.info;

P.Topo ← P.Topo^.prox;

dispose(Q);

Complexidade:

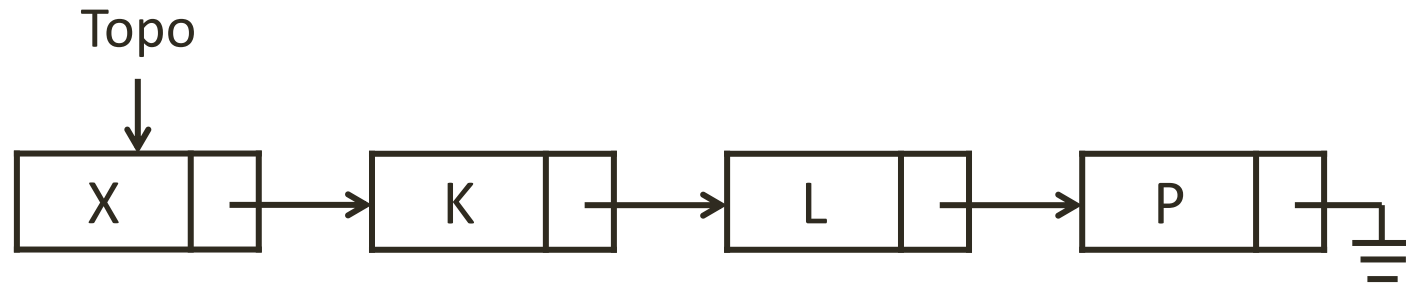
$\Theta(1)$

Pilha (alocação dinâmica)

- ElementoDoTopo(P);

Complexidade:

Pilha (alocação dinâmica)



ElementoDoTopo(P);

se NÃO PilhaVazia(P)

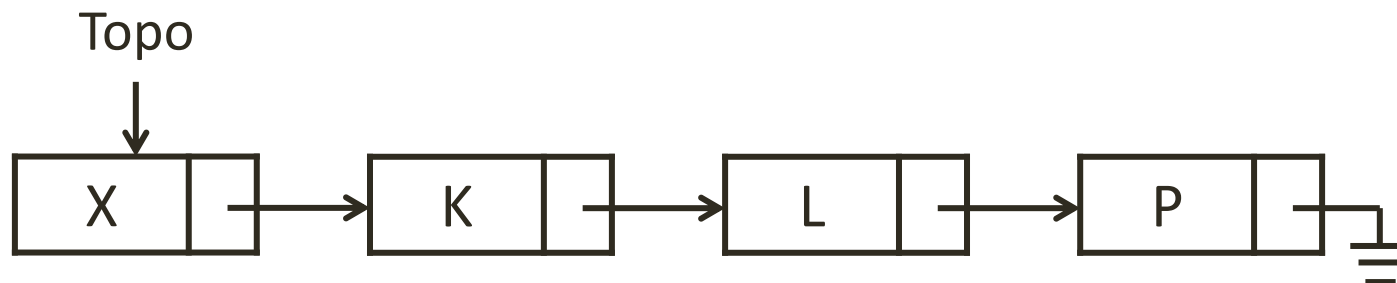
então | ElementoDoTopo $\leftarrow$ P.Topo^.info;

senão | <erro>

Complexidade:

$\Theta(1)$

Pilha (alocação dinâmica)



Função	Complexidade
InicializaPilha(P);	$\Theta(1)$
PilhaVazia(P);	$\Theta(1)$
Empilha(P, x);	$\Theta(1)$
Desempilha(P, x);	$\Theta(1)$
ElementoDoTopo(P);	$\Theta(1)$

Fila (alocação dinâmica)

tipo TpFila = registro

início

Início : ^TpNó;

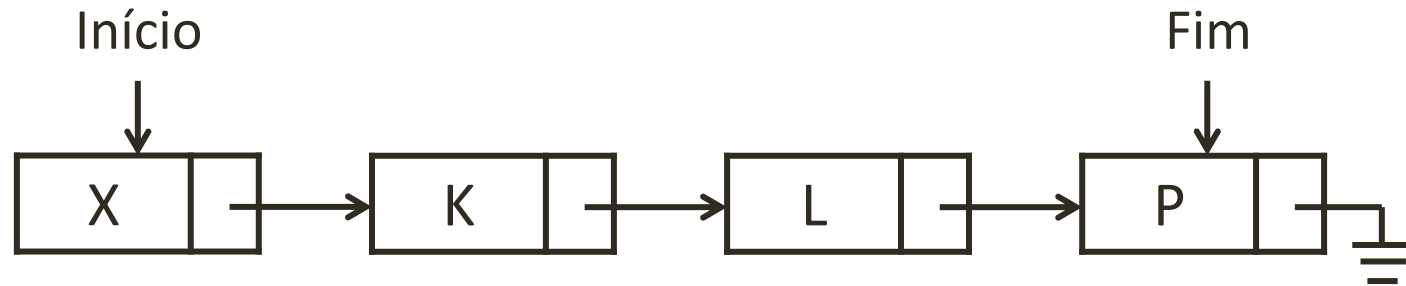
Fim : ^TpNó;

fim;

onde:

- Início : ponteiro que aponta para o primeiro elemento da fila
- Fim : ponteiro que aponta para o último elemento da fila

Fila (alocação dinâmica)



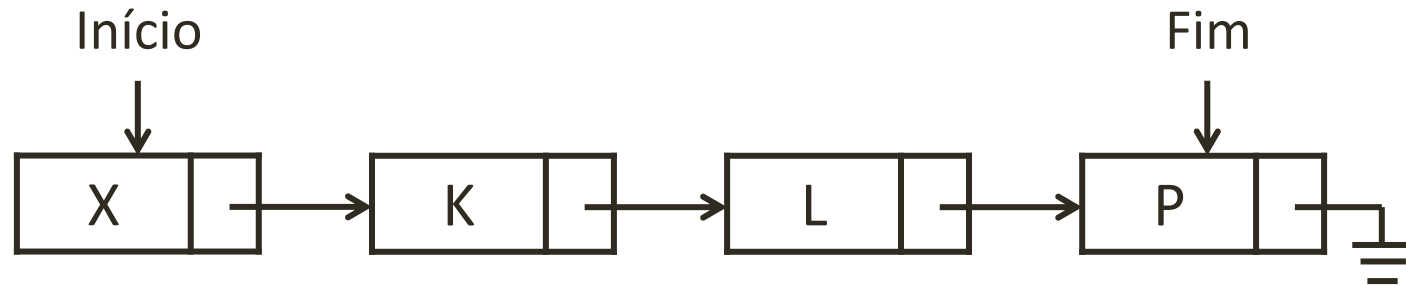
- InicializaFila(Q);
- FilaVazia(Q);
- InsereFila(Q, x);
- RemoveFila(Q, x);
- PrimeiroDaFila(Q);

Fila (alocação dinâmica)

- InicializaFila(Q);

Complexidade:

Fila (alocação dinâmica)



InicializaFila(Q);

Q.Início $\leftarrow$ nil;

Q.Fim $\leftarrow$ nil;

Complexidade:

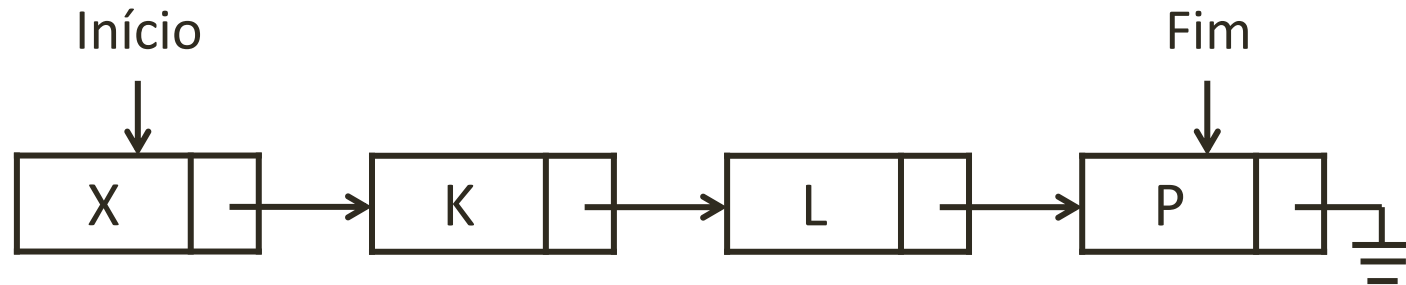
$\Theta(1)$

Fila (alocação dinâmica)

- FilaVazia(Q);

Complexidade:

Fila (alocação dinâmica)



FilaVazia(Q);

FilaVazia $\leftarrow$ Q.Início = nil;

Complexidade:

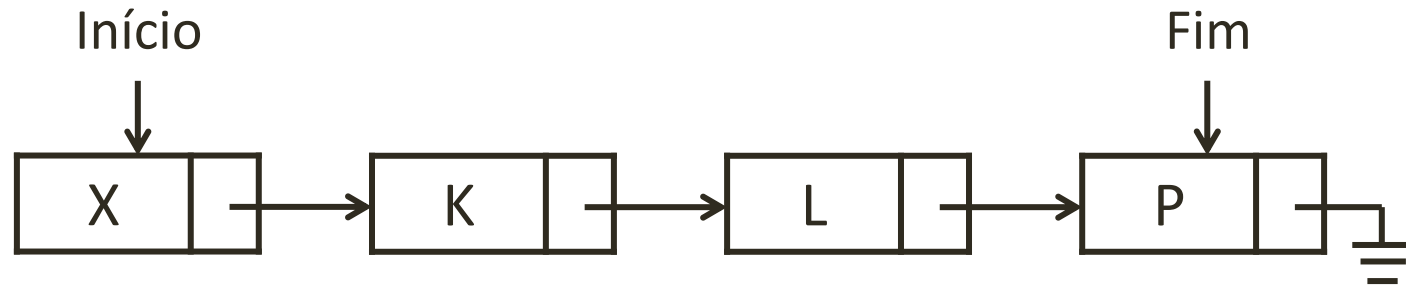
$\Theta(1)$

Fila (alocação dinâmica)

- `InsererFila(Q, x);`

Complexidade:

Fila (alocação dinâmica)



InsererFila(Q, x);

novo(R);

R^.info $\leftarrow$ x;

R^.prox $\leftarrow$ nil;

se FilaVazia(Q)

então | Q.Início $\leftarrow$ R;

senão | Q.Fim^.prox $\leftarrow$ R;

Q.Fim $\leftarrow$ R;

Complexidade:

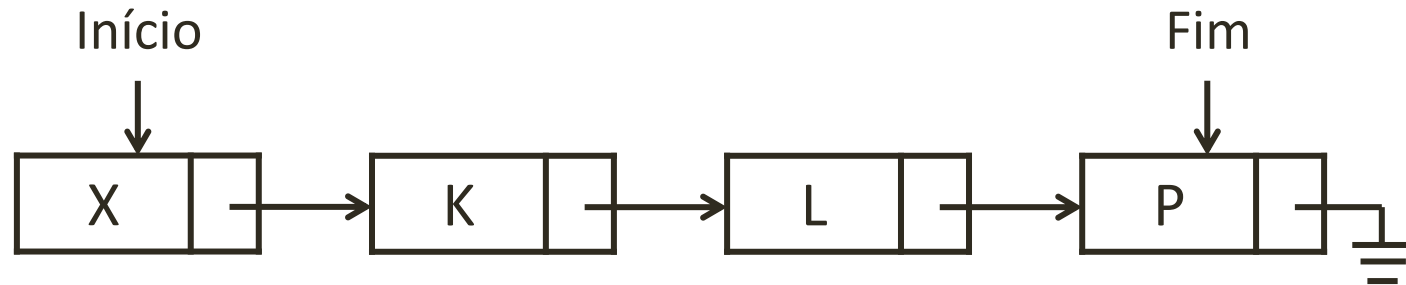
$\Theta(1)$

Fila (alocação dinâmica)

- RemoveFila(Q, x);

Complexidade:

Fila (alocação dinâmica)



RemoveFila(Q, x);

se NÃO FilaVazia(Q)

então

R ← Q.Início;

x ← R^.info;

Q.Início ← Q.Início^.prox;

dispose(R);

se FilaVazia(Q)

então | Q.Fim ← nil;

Complexidade:

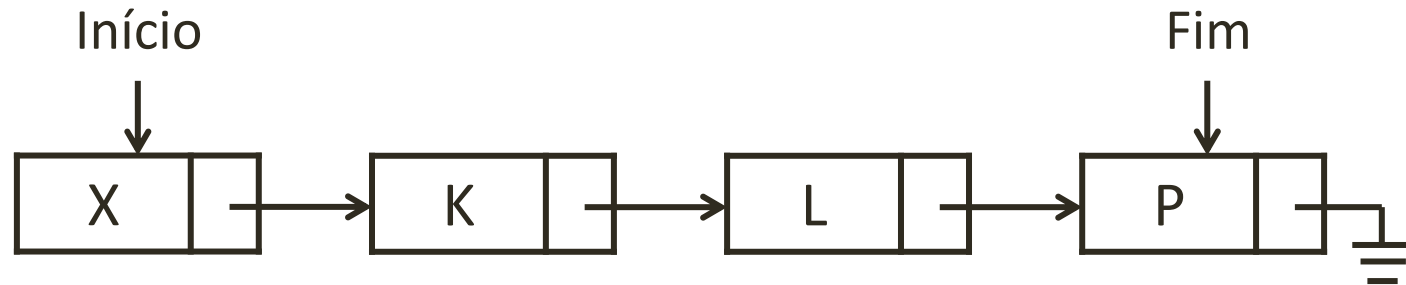
$\Theta(1)$

Fila (alocação dinâmica)

- PrimeiroDaFila(Q);

Complexidade:

Fila (alocação dinâmica)



PrimeiroDaFila(Q);

se NÃO FilaVazia(Q)

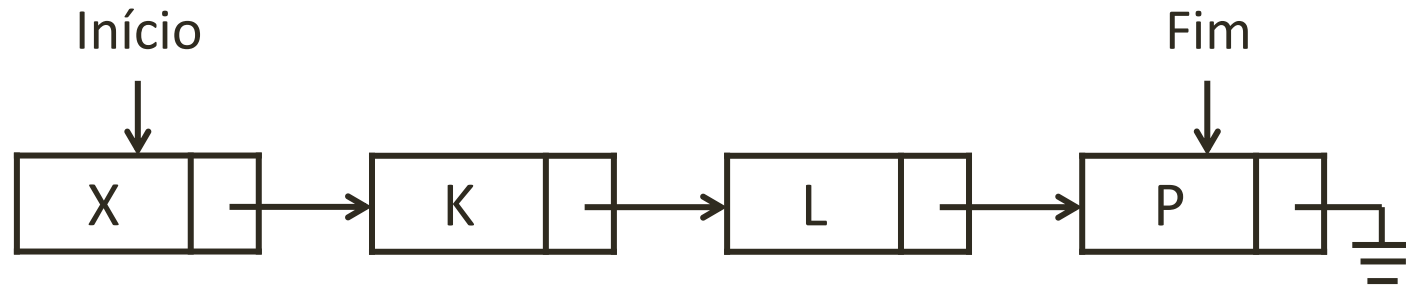
então | PrimeiroDaFila $\leftarrow$ P.Início^.info;

senão | <erro>

Complexidade:

$\Theta(1)$

Fila (alocação dinâmica)



Função	Complexidade
InicializaFila(Q);	$\Theta(1)$
FilaVazia(Q);	$\Theta(1)$
InsereFila(Q, x);	$\Theta(1)$
RemoveFila(Q, x);	$\Theta(1)$
PrimeiroDaFila(Q);	$\Theta(1)$

Fila dupla (alocação dinâmica)

tipo TpFilaDupla = registro

início

Esq : ^TpNó;

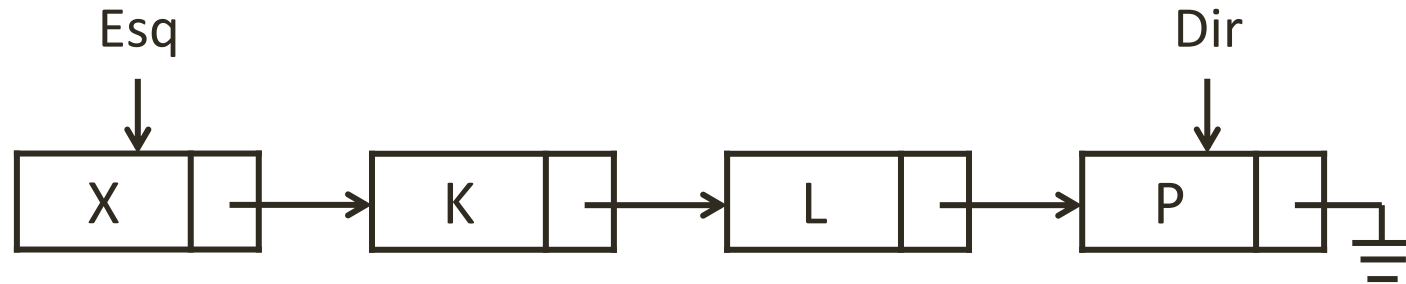
Dir : ^TpNó;

fim;

onde:

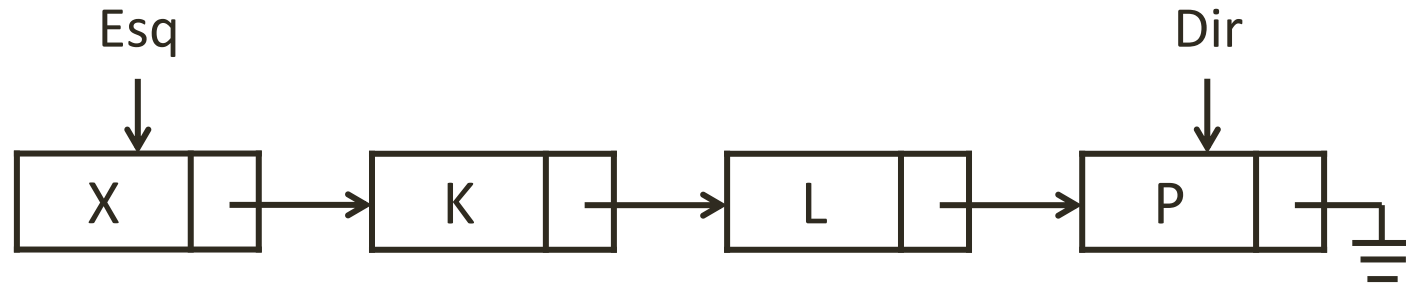
- Esq : ponteiro que aponta para o elemento na extremidade esquerda da fila dupla
- Dir : ponteiro que aponta para o elemento na extremidade direita da fila dupla

Fila dupla (alocação dinâmica)



- InicializaFilaDupla(D);
- FilaDuplaVazia(D);
- InsereEsquerdaFilaDupla(D, x);
- InsereDireitaFilaDupla(D, x);

Fila dupla (alocação dinâmica)



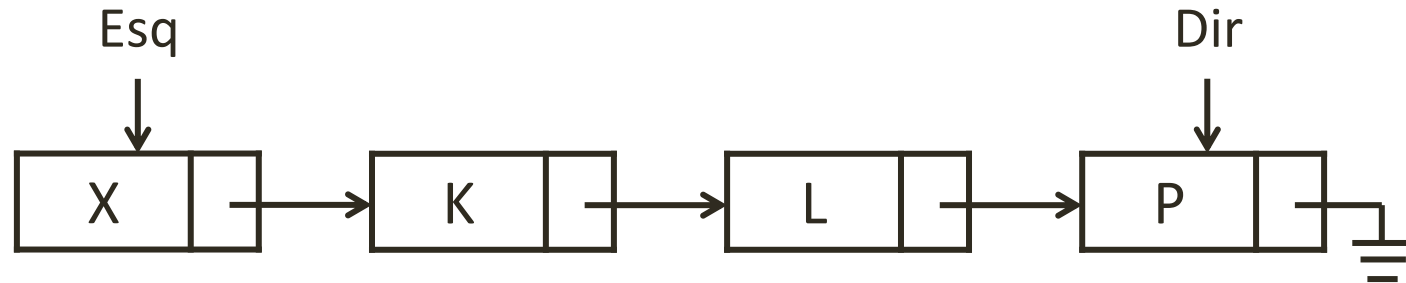
- RemoveEsquerdaFilaDupla(D, x);
- RemoveDireitaFilaDupla(D, x);
- ElementoEsquerdaFilaDupla(D);
- ElementoDireitaFilaDupla(D);

Fila dupla (alocação dinâmica)

- InicializaFilaDupla(D);

Complexidade:

Fila dupla (alocação dinâmica)



InicializaFilaDupla(D);

D.Esq $\leftarrow$ nil;

D.Dir $\leftarrow$ nil;

Complexidade:

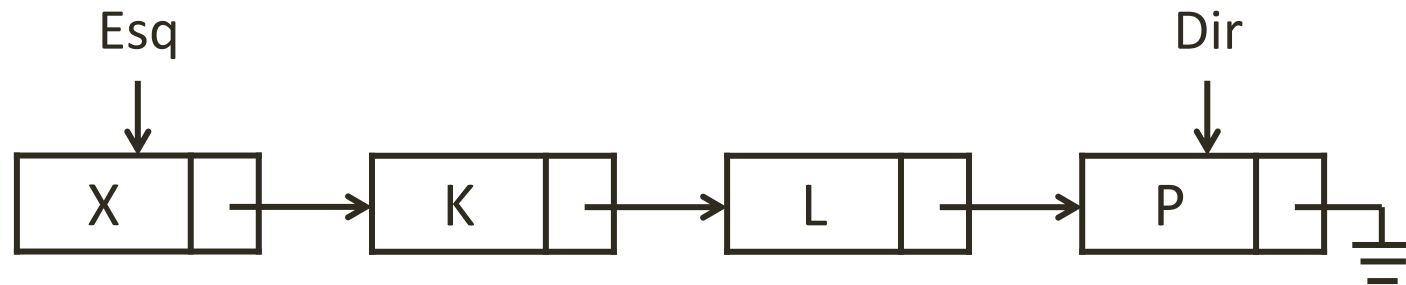
$\Theta(1)$

Fila dupla (alocação dinâmica)

- FilaDuplaVazia(D);

Complexidade:

Fila dupla (alocação dinâmica)



FilaDuplaVazia(D);

FilaDuplaVazia $\leftarrow$ D.Esq = nil;

Complexidade:

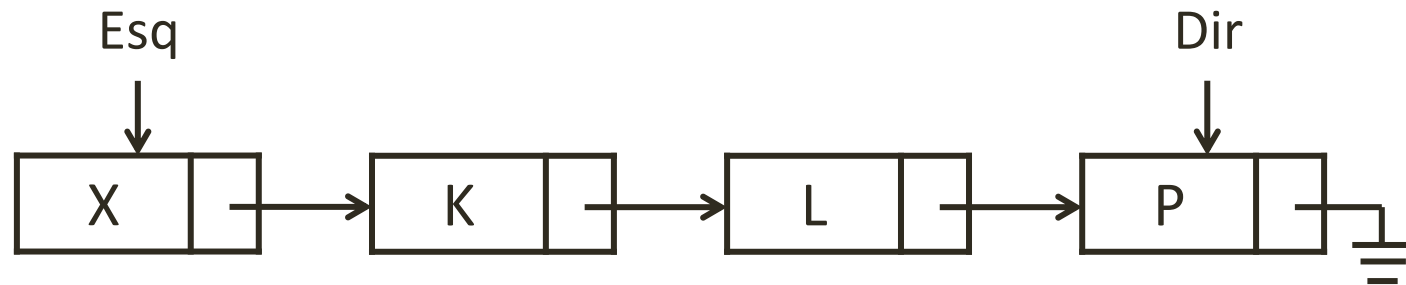
$\Theta(1)$

Fila dupla (alocação dinâmica)

- `InseresquerdaFilaDupla(D, x);`

Complexidade:

Fila dupla (alocação dinâmica)



InserEsquerdaFilaDupla(D, x);

novo(Q);

Q^.info $\leftarrow$ x;

se FilaDuplaVazia(D)

então | D.Dir $\leftarrow$ Q;

Q^.prox $\leftarrow$ D.Esq;

D.Esq $\leftarrow$ Q;

Complexidade:

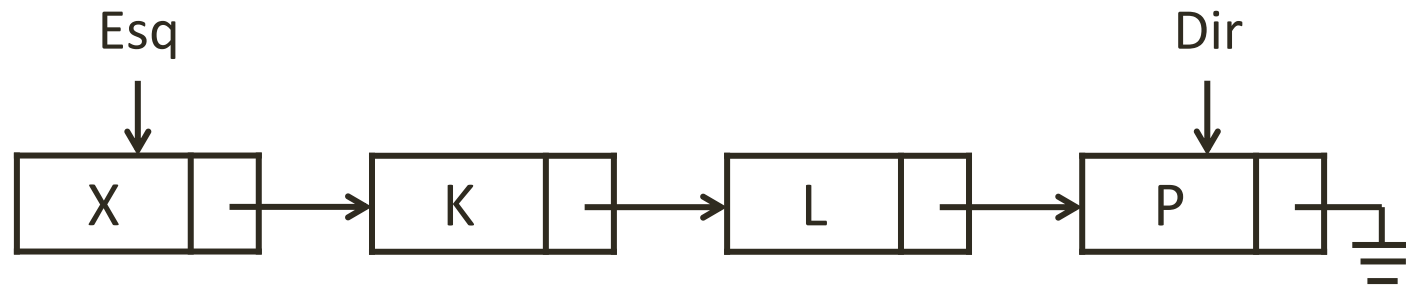
$\Theta(1)$

Fila dupla (alocação dinâmica)

- InsereDireitaFilaDupla(D, x);

Complexidade:

Fila dupla (alocação dinâmica)



InserDireitaFilaDupla(D, x);

se FilaDuplaVazia(D)

então | InserEsquerdaFilaDupla(D, x);

senão | novo(Q);

Q^.info ← x;

Q^.prox ← nil;

D.Dir^.prox ← Q;

D.Dir ← Q;

Complexidade:

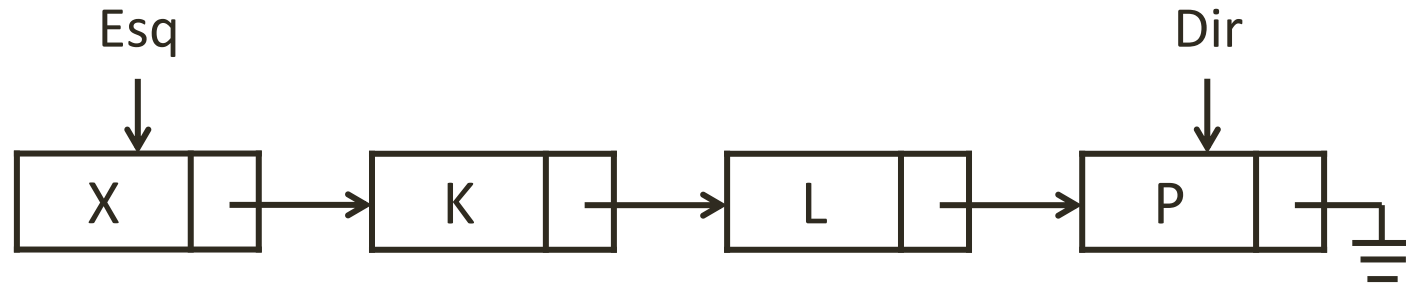
$\Theta(1)$

Fila dupla (alocação dinâmica)

- RemoveEsquerdaFilaDupla(D, x);

Complexidade:

Fila dupla (alocação dinâmica)



RemoveEsquerdaFilaDupla(D, x);

se NÃO FilaDuplaVazia(D)

então

$x \leftarrow D.Esq^.info;$

$Q \leftarrow D.Esq;$

$D.Esq \leftarrow D.Esq^.prox;$

dispose(Q);

se FilaDuplaVazia(D)

então

$D.Dir \leftarrow nil;$

Complexidade:

$\Theta(1)$

Fila dupla (alocação dinâmica)

- RemoveDireitaFilaDupla(D, x);

Complexidade:

Fila dupla (alocação dinâmica)

RemoveDireitaFilaDupla(D, x);

se NÃO FilaDuplaVazia(D)

então

$x \leftarrow D.Dir^{\wedge}.info;$

se $D.Esq = D.Dir$

então

dispose(D.Esq);

InicializaFilaDupla(D);

senão

$Q \leftarrow D.Dir;$

$D.Dir \leftarrow D.Esq;$

enquanto $D.Dir^{\wedge}.prox \neq Q$

faça

$D.Dir \leftarrow D.Dir^{\wedge}.prox;$

$D.Dir^{\wedge}.prox \leftarrow nil;$

dispose(Q);

Complexidade:

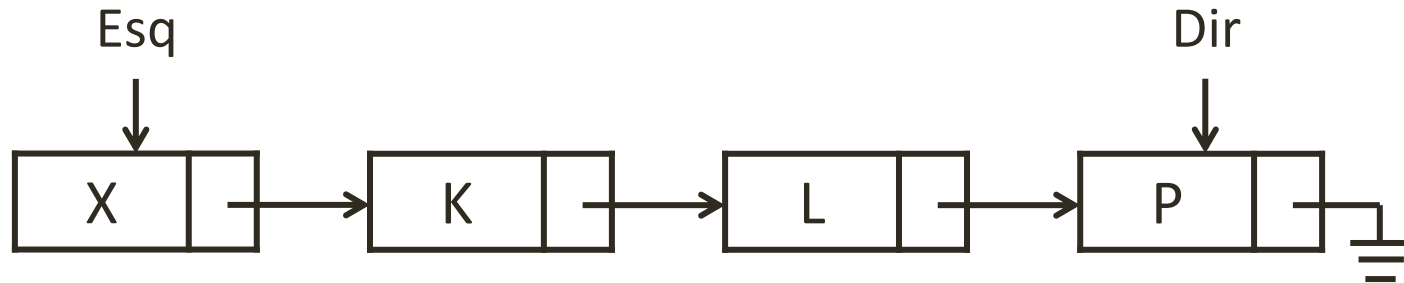
$O(n)$

Fila dupla (alocação dinâmica)

- ElementoEsquerdaFilaDupla(D);

Complexidade:

Fila dupla (alocação dinâmica)



ElementoEsquerdaFilaDupla(D);

se NÃO FilaDuplaVazia(D)

então | ElementoEsquerdaFilaDupla $\leftarrow$ D.Esq^.info;

senão | <erro>

Complexidade:

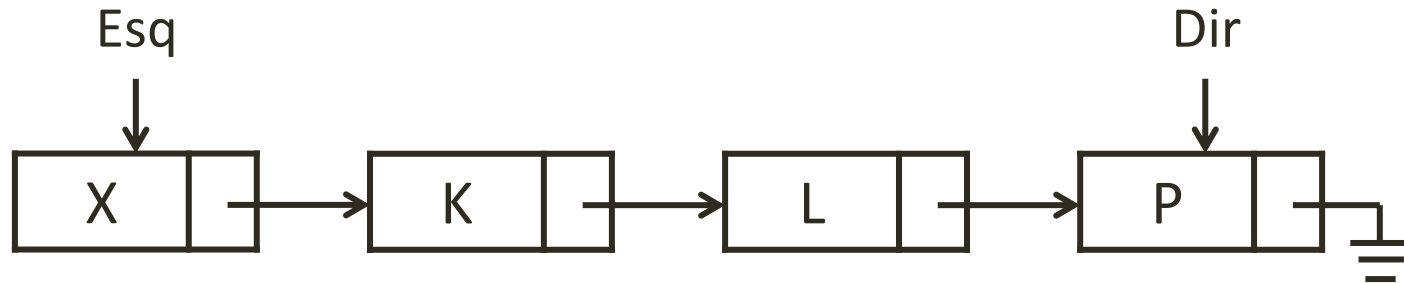
$\Theta(1)$

Fila dupla (alocação dinâmica)

- ElementoDireitaFilaDupla(D);

Complexidade:

Fila dupla (alocação dinâmica)



ElementoDireitaFilaDupla(D);

se NÃO FilaDuplaVazia(D)

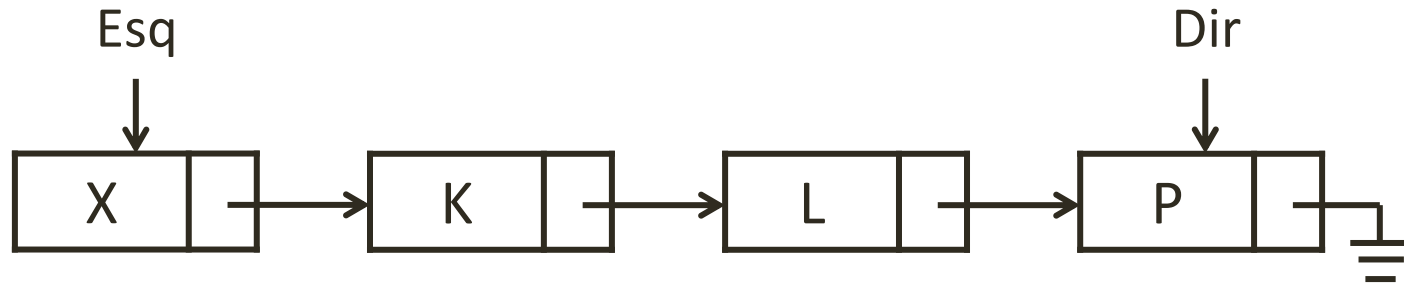
então | ElementoDireitaFilaDupla $\leftarrow$ D.Dir^.info;

senão | <erro>

Complexidade:

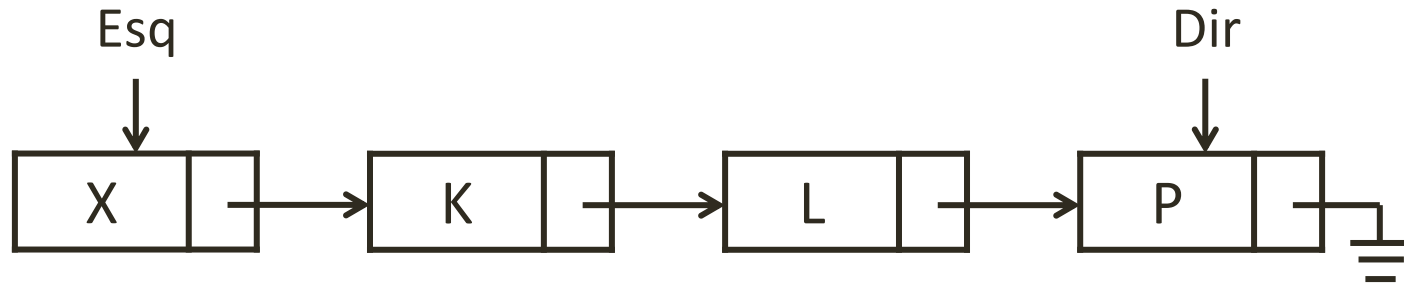
$\Theta(1)$

Fila dupla (alocação dinâmica)



Função	Complexidade
InicializaFilaDupla(D);	$\Theta(1)$
FilaDuplaVazia(D);	$\Theta(1)$
InseresEsquerdaFilaDupla(D, x);	$\Theta(1)$
InseresDireitaFilaDupla(D, x);	$\Theta(1)$

Fila dupla (alocação dinâmica)



Função	Complexidade
RemoveEsquerdaFilaDupla(D, x);	$\Theta(1)$
RemoveDireitaFilaDupla(D, x);	$O(n)$
ElementoEsquerdaFilaDupla(D, x);	$\Theta(1)$
ElementoDireitaFilaDupla(D, x);	$\Theta(1)$

Outras implementações de listas ligadas

Outras implementações de listas ligadas

- Listas ligadas com cabeça de lista
- Listas circulares
- Listas duplamente ligadas

Listas ligadas com cabeça de lista

- A lista ligada com cabeça de lista é uma lista ligada parecida com a que temos estudado até agora
- Na lista ligada que temos estudado (sem cabeça), um ponteiro é necessário para acessar a primeira variável dinâmica da estrutura, que armazena o primeiro elemento da lista
- Na lista ligada com cabeça de lista, a variável dinâmica que contém o primeiro elemento da lista é **precedida por uma outra variável dinâmica extra** que não armazena nenhum elemento da lista
- Este elemento extra é chamado de **cabeça** ou **sentinela**

Listas ligadas com cabeça de lista

- Na lista ligada com cabeça de lista, a variável dinâmica que contém o primeiro elemento da lista é **precedida por uma outra variável dinâmica extra** que não armazena nenhum elemento da lista
- Este elemento extra é chamado de **cabeça** ou **sentinela**
- O campo prox da cabeça da lista aponta para a variável dinâmica que armazena o primeiro elemento da lista
- O objetivo do uso de uma cabeça de lista é facilitar a implementação de alguns algoritmos

Listas ligadas com cabeça de lista

- O tipo TpNó é o mesmo usado nas listas ligadas que vimos anteriormente



tipo TpNó = registro

início

info : tipo;

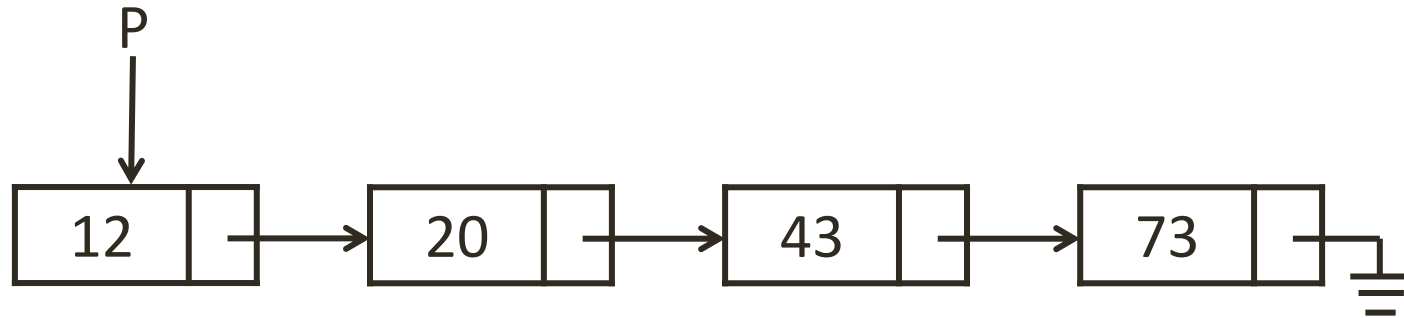
prox : ^TpNó;

fim;

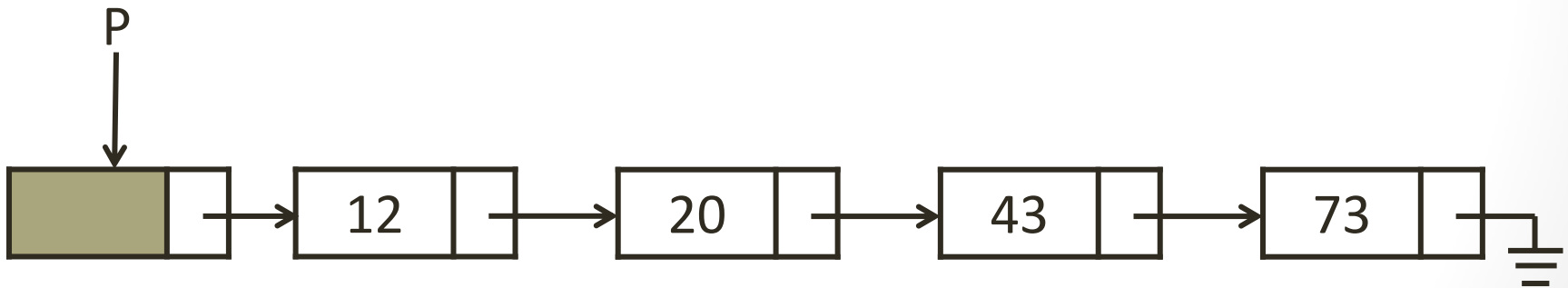
onde:

- info : campo de informação (pode ter um ou mais campos)
- prox : é um ponteiro do tipo TpNó

Listas ligadas com cabeça de lista



(Lista ligada)



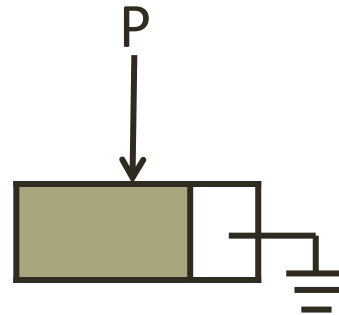
(Lista ligada com cabeça de lista)

Listas ligadas com cabeça de lista

- Diferença entre as duas listas quando estão vazias



(Lista ligada)



(Lista ligada com cabeça de lista)

Listas ligadas com cabeça de lista

- Exemplos de algoritmos para manipulação de listas ligadas com cabeça de lista
- Comparação com algoritmos equivalentes para manipulação de listas ligadas sem cabeça/sentinela
- `InsereOrdenado(P, x);`
- `RemoveElemento(P, x)`

Listas ligadas com cabeça de lista

InserirOrdenado(P, x);

novo(Q);

Q^.info $\leftarrow$ x;

se (P = nil) OU (x $\leq$ P^.info)

então

Q^.prox $\leftarrow$ P;

P $\leftarrow$ Q;

senão

R $\leftarrow$ P;

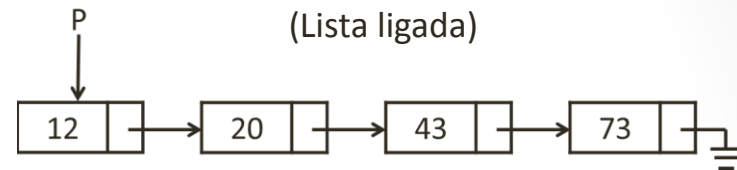
enquanto (R^.prox $\neq$ nil) E (R^.prox^.info < x)

faça

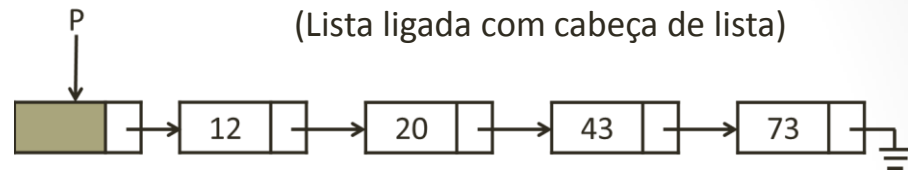
R $\leftarrow$ R^.prox;

Q^.prox $\leftarrow$ R^.prox;

R^.prox $\leftarrow$ Q;



Listas ligadas com cabeça de lista



`InserirOrdenado(P, x);`

`novo(Q);`

`$Q^{\wedge}.info \leftarrow x;$`

`$R \leftarrow P;$`

`enquanto ( $R^{\wedge}.prox \neq nil$ ) E ( $R^{\wedge}.prox^{\wedge}.info < x$ )`

`faça   |    $R \leftarrow R^{\wedge}.prox;$`

`$Q^{\wedge}.prox \leftarrow R^{\wedge}.prox;$`

`$R^{\wedge}.prox \leftarrow Q;$`

Listas ligadas com cabeça de lista

RemoveElemento(P, x);

se $P \neq \text{nil}$

então

se $P^{\wedge}.\text{info} = x$

então

$Q \leftarrow P;$

$P \leftarrow P^{\wedge}.\text{prox};$

dispose(Q);

senão

$R \leftarrow P;$

enquanto $(R^{\wedge}.\text{prox} \neq \text{nil}) \wedge (R^{\wedge}.\text{prox}^{\wedge}.\text{info} \neq x)$

faça

$R \leftarrow R^{\wedge}.\text{prox};$

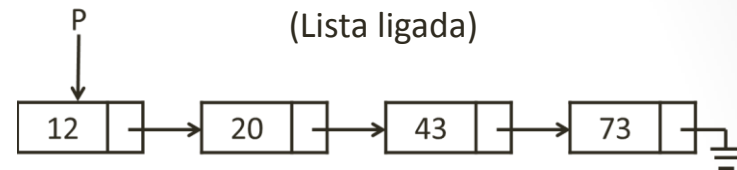
se $R^{\wedge}.\text{prox} \neq \text{nil}$

então

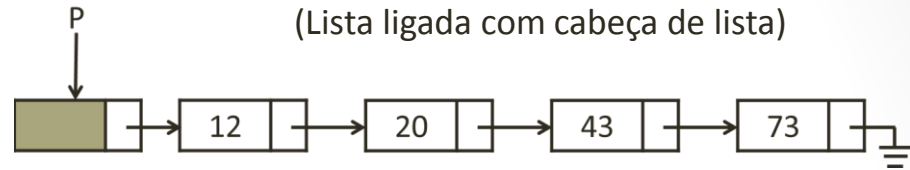
$Q \leftarrow R^{\wedge}.\text{prox};$

$R^{\wedge}.\text{prox} \leftarrow Q^{\wedge}.\text{prox};$

dispose(Q);



Listas ligadas com cabeça de lista



RemoveElemento(P, x);

$R \leftarrow P;$

enquanto ($R^{\wedge}.prox \neq nil$) E ($R^{\wedge}.prox^{\wedge}.info \neq x$)

faça | $R \leftarrow R^{\wedge}.prox;$

se $R^{\wedge}.prox \neq nil$

então | $Q \leftarrow R^{\wedge}.prox;$

$R^{\wedge}.prox \leftarrow Q^{\wedge}.prox;$

dispose(Q);

Listas circulares

tipo TpListaCirc = registro

início

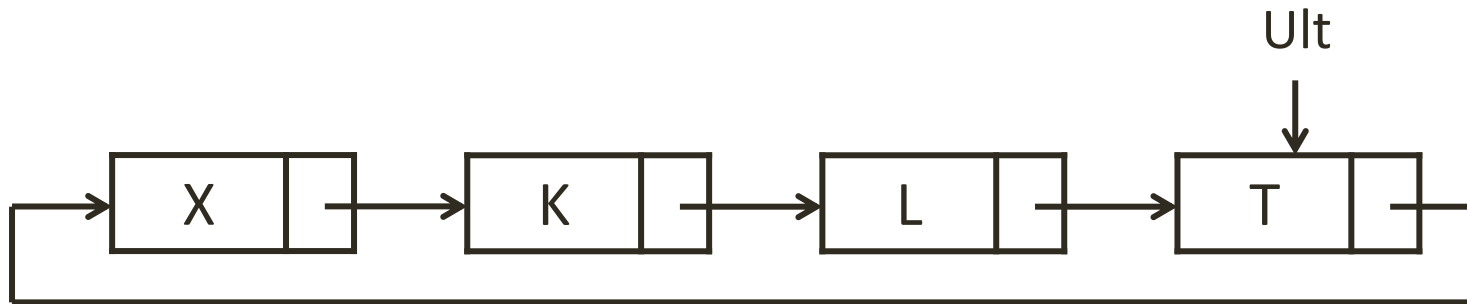
Ult : ^TpNó;

fim;

onde:

- Ult : ponteiro que aponta para o último elemento da lista ligada
- O tipo TpNó é o mesmo usado nas listas ligadas anteriores
- Na lista circular, o primeiro elemento é apontado por Ult^.prox

Listas circulares



tipo TpListaCirc = registro

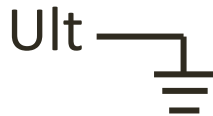
início

Ult : ^TpNó;

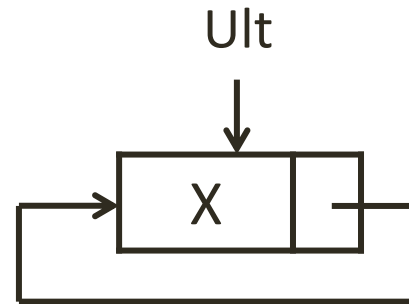
fim;

Listas circulares

- Casos particulares de listas circulares

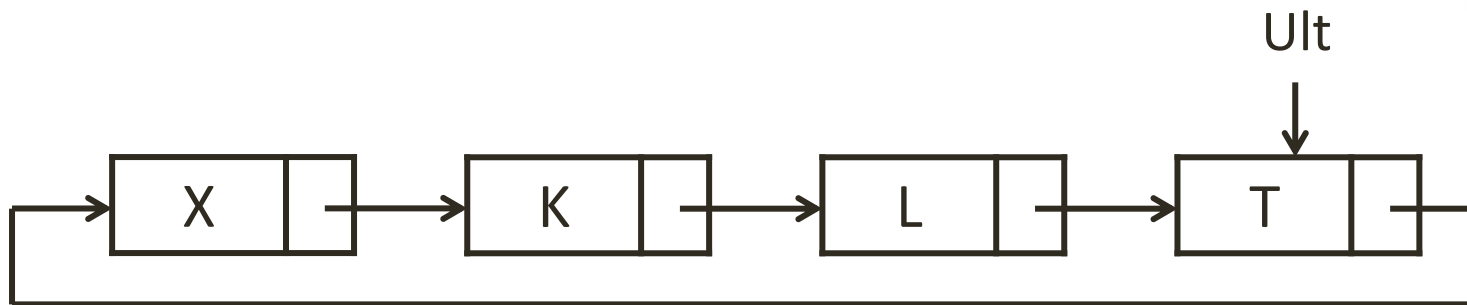


(Lista circular vazia)



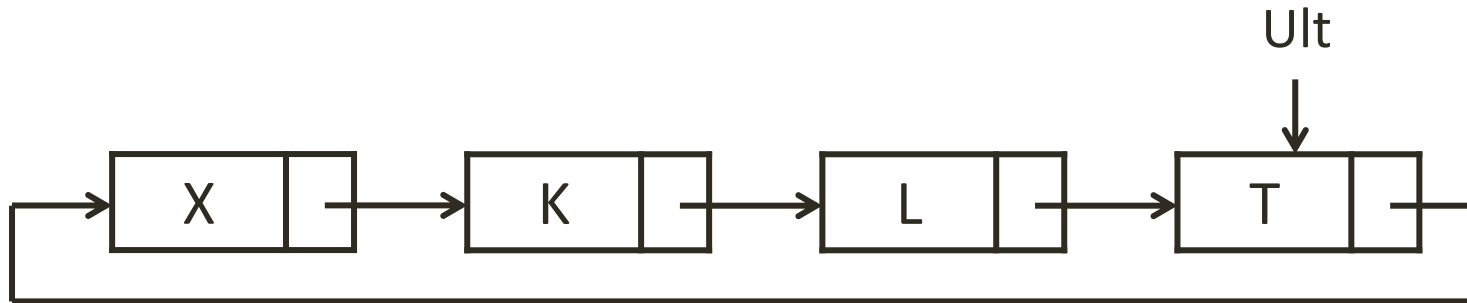
(Lista circular com um único elemento)

Listas circulares



- Fila dupla (implementada com uma lista circular)
- InicializaListaCirc(LC);
- ListaCircVazia(LC);
- InsereEsquerdaListaCirc(LC, x);
- InsereDireitaListaCirc(LC, x);

Listas circulares



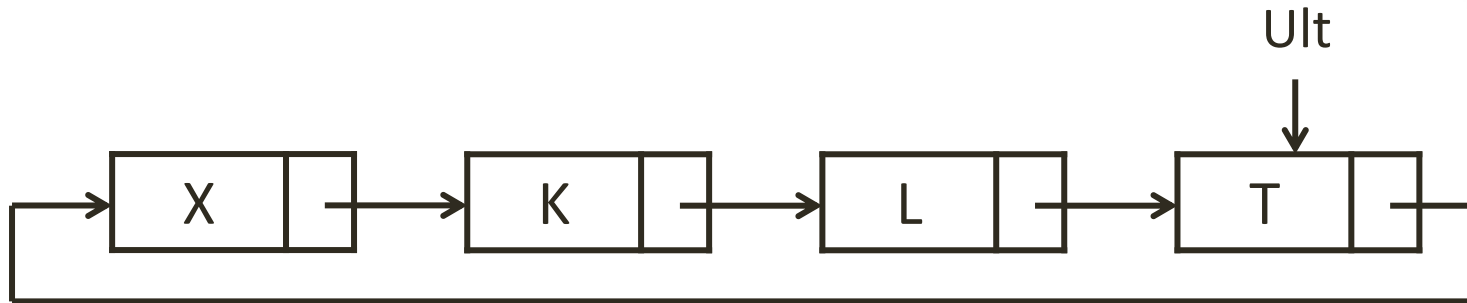
- Fila dupla (implementada com uma lista circular)
- RemoveEsquerdaListaCirc(LC, x);
- RemoveDireitaListaCirc(LC, x);
- ElementoEsquerdaListaCirc(LC);
- ElementoDireitaListaCirc(LC);

Listas circulares

- InicializaListaCirc(LC);

Complexidade:

Listas circulares



InicializaListaCirc(LC);

LC.Ult $\leftarrow$ nil;

Complexidade:

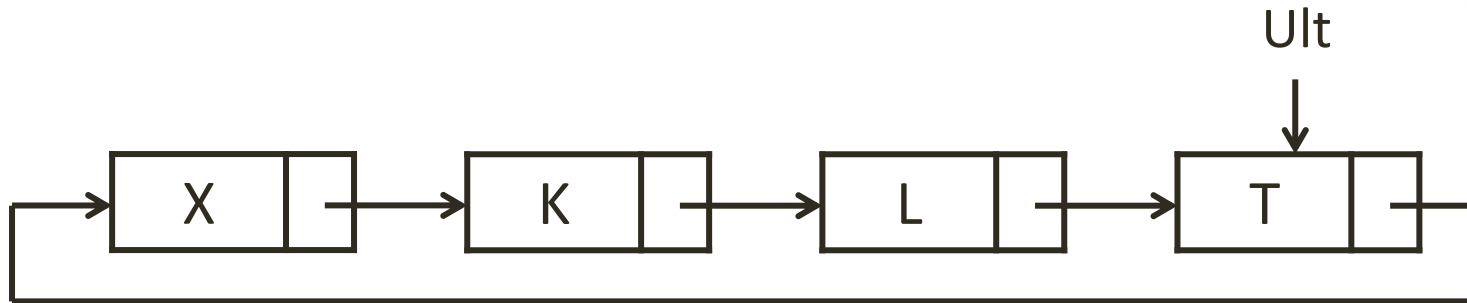
$\Theta(1)$

Listas circulares

- ListaCircVazia(LC);

Complexidade:

Listas circulares



ListaCircVazia(LC);

ListaCircVazia $\leftarrow$ LC.Ult = nil;

Complexidade:

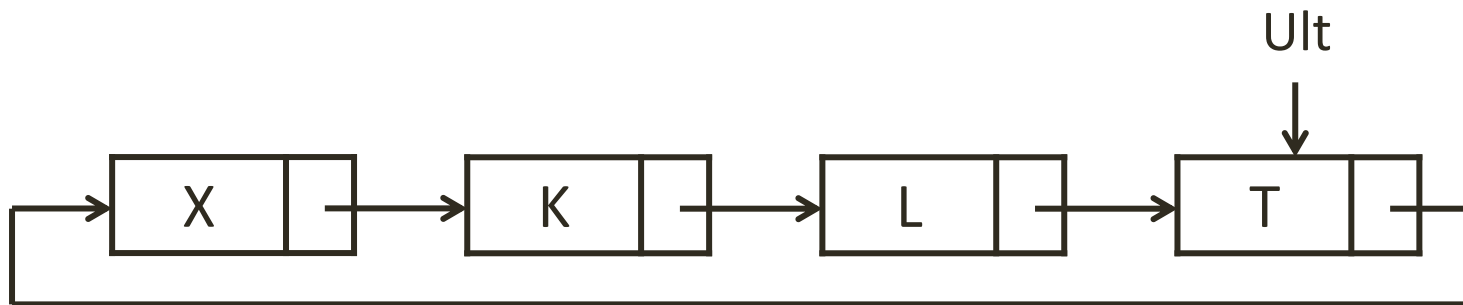
$\Theta(1)$

Listas circulares

- `InseresquerdaListaCirc(LC, x);`

Complexidade:

Listas circulares



InserEsquerdaListaCirc(LC, x);

novo(Q);

Q^.info $\leftarrow$ x;

se ListaCircVazia(LC)

então

LC.Ult $\leftarrow$ Q;

LC.Ult^.prox $\leftarrow$ LC.Ult;

senão

Q^.prox $\leftarrow$ LC.Ult^.prox;

LC.Ult^.prox $\leftarrow$ Q;

Complexidade:

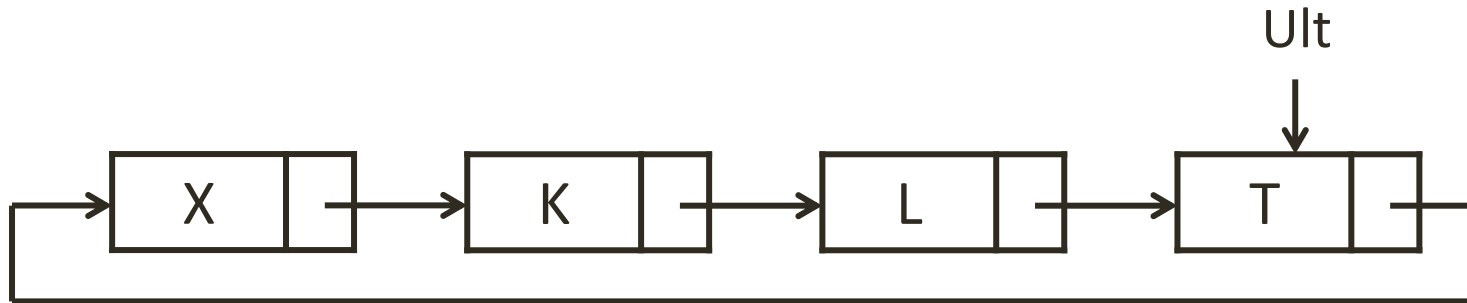
$\Theta(1)$

Listas circulares

- `InserDireitaListaCirc(LC, x);`

Complexidade:

Listas circulares



InserDireitaListaCirc(LC, x);

InserEsquerdaListaCirc(LC, x);

LC.Ult $\leftarrow$ LC.Ult[^].prox;

Complexidade:

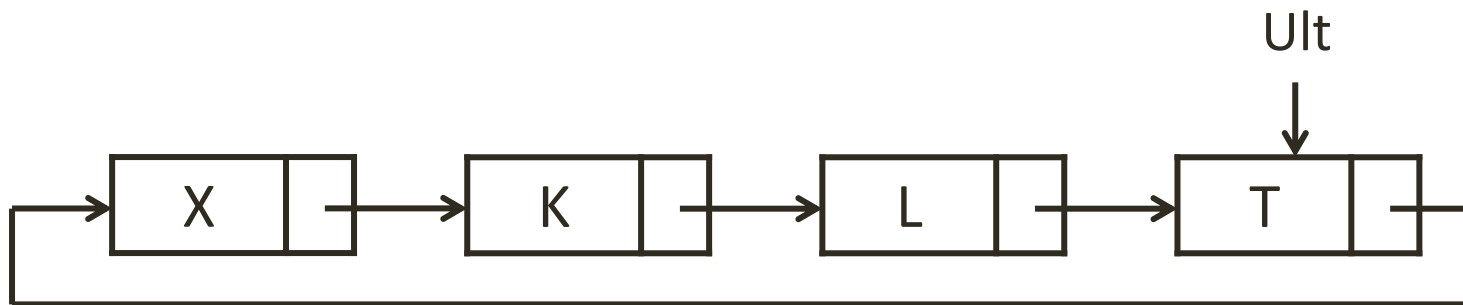
$\Theta(1)$

Listas circulares

- RemoveEsquerdaListaCirc(LC, x);

Complexidade:

Listas circulares



RemoveEsquerdaListaCirc(LC, x);

se NÃO ListaCircVazia(LC)

então

Q ← LC.Ult[^].prox;

x ← Q[^].prox;

se Q = LC.Ult

então | LC.Ult ← nil;

senão | LC.Ult[^].prox ← Q[^].prox;

dispose(Q);

Complexidade:

$\Theta(1)$

Listas circulares

- RemoveDireitaListaCirc(LC, x);

Complexidade:

Listas circulares

RemoveDireitaListaCirc(LC, x);

se NÃO ListaCircVazia(LC)

então

$x \leftarrow LC.Ult^{\wedge}.info;$

se $LC.Ult = LC.Ult^{\wedge}.prox$

então

dispose(LC.Ult);

$LC.Ult \leftarrow nil;$

senão

$Q \leftarrow LC.Ult;$

repita

$LC.Ult \leftarrow LC.Ult^{\wedge}.prox;$

até $LC.Ult^{\wedge}.prox = Q;$

$LC.Ult^{\wedge}.prox \leftarrow Q^{\wedge}.prox;$

dispose(Q);

Complexidade:

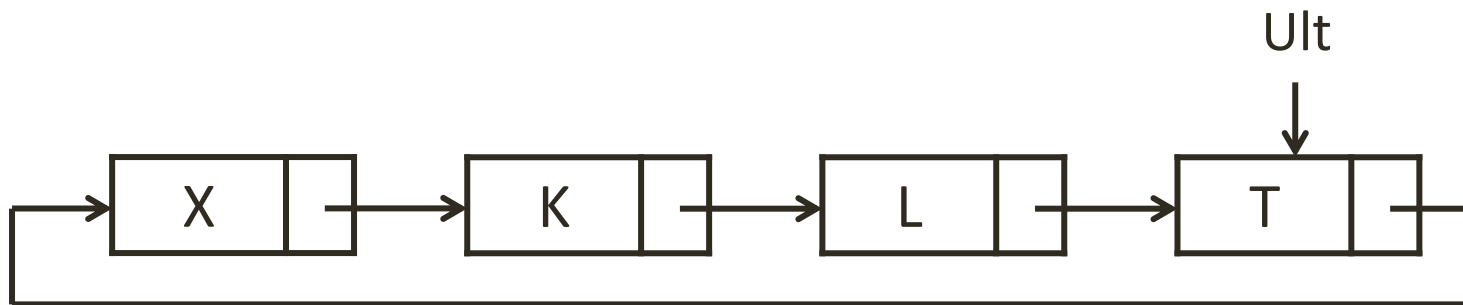
$O(n)$

Listas circulares

- ElementoEsquerdaListaCirc(LC);

Complexidade:

Listas circulares



ElementoEsquerdaListaCirc(LC);

se NÃO ListaCircVazia(LC)

então | ElementoEsquerdaListaCirc ← LC.Ult^.prox^.info;

senão | <erro>

Complexidade:

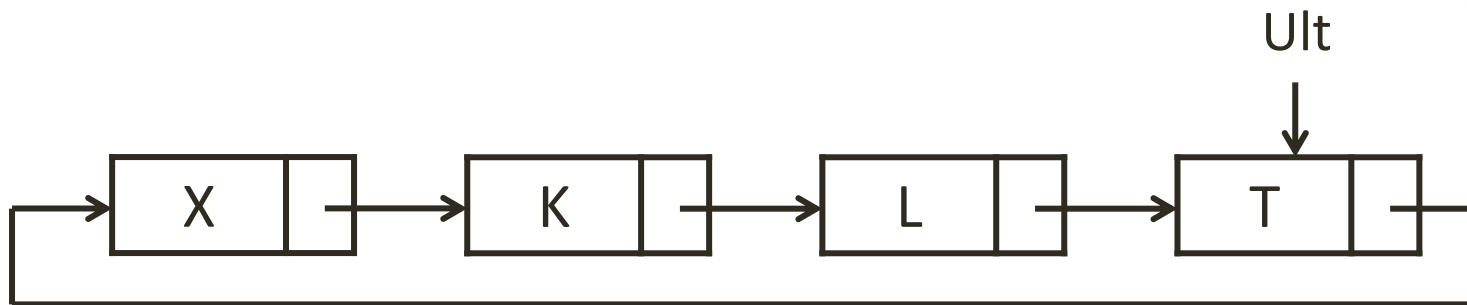
$\Theta(1)$

Listas circulares

- ElementoDireitaListaCirc(LC);

Complexidade:

Listas circulares



ElementoDireitaListaCirc(LC);

se NÃO ListaCircVazia(LC)

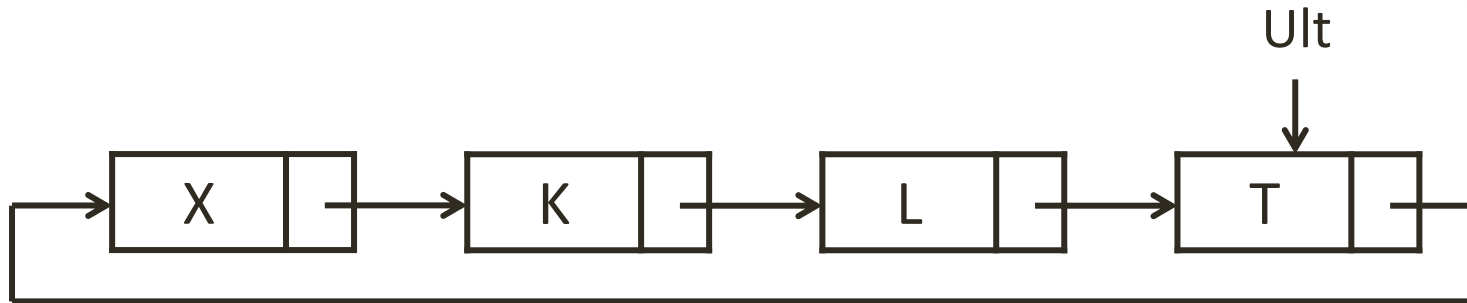
então | ElementoDireitaListaCirc ← LC.Ult^.info;

senão | <erro>

Complexidade:

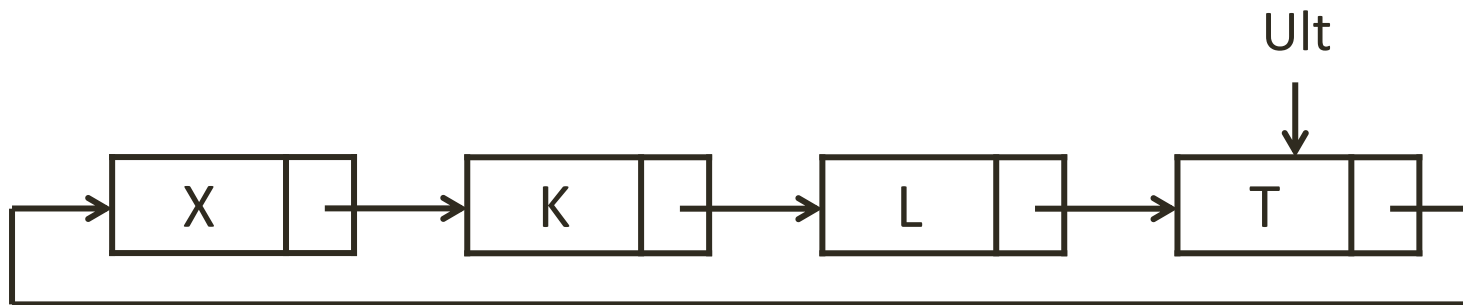
$\Theta(1)$

Listas circulares



Função	Complexidade
InicializaListaCirc(LC);	$\Theta(1)$
ListaCircVazia(LC);	$\Theta(1)$
InseresEsquerdaListaCirc(LC, x);	$\Theta(1)$
InseresDireitaListaCirc(LC, x);	$\Theta(1)$

Listas circulares



Função	Complexidade
RemoveEsquerdaListaCirc(LC, x);	$\Theta(1)$
RemoveDireitaListaCirc(LC, x);	$O(n)$
ElementoEsquerdaListaCirc(LC, x);	$\Theta(1)$
ElementoDireitaListaCirc(LC, x);	$\Theta(1)$

Listas duplamente ligadas

- Registro

tipo TpNóDuplo = registro
início



info : tipo;

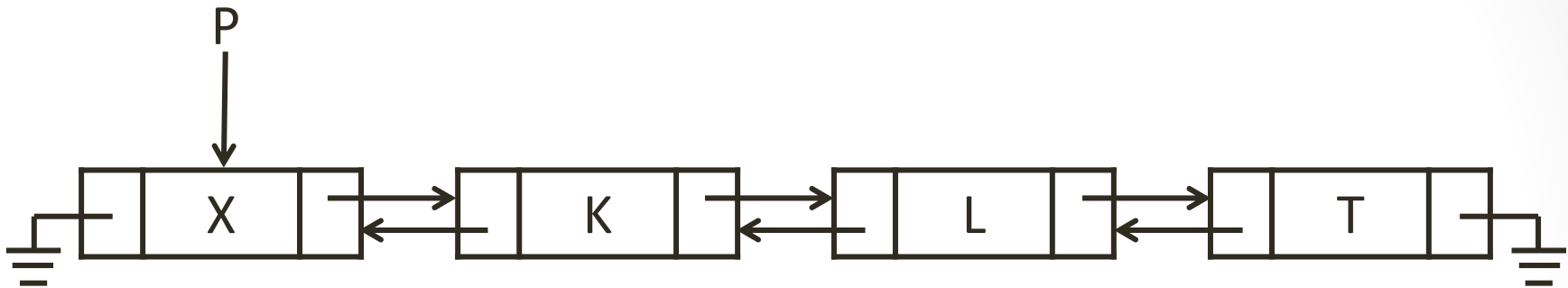
pred, suc : ^TpNóDuplo;

fim;

onde:

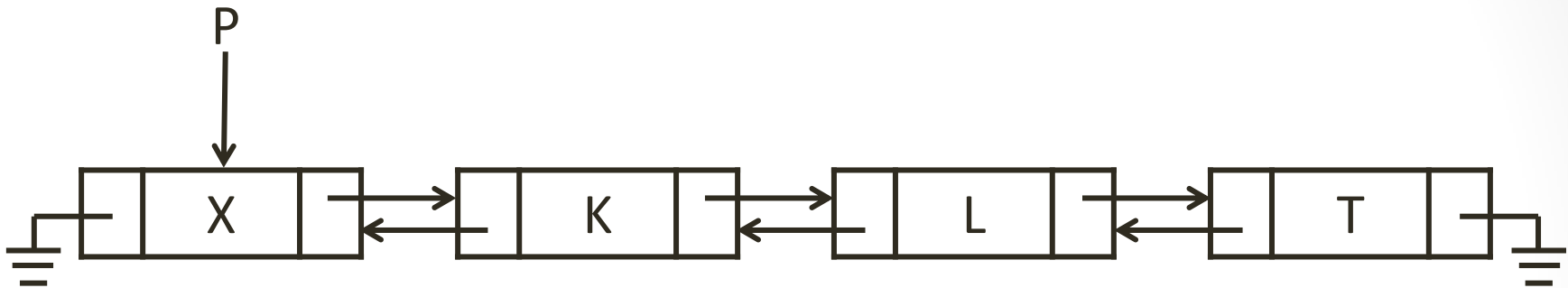
- info : campo de informação (pode ter um ou mais campos)
- pred e suc: são ponteiros do tipo TpNóDuplo

Listas duplamente ligadas



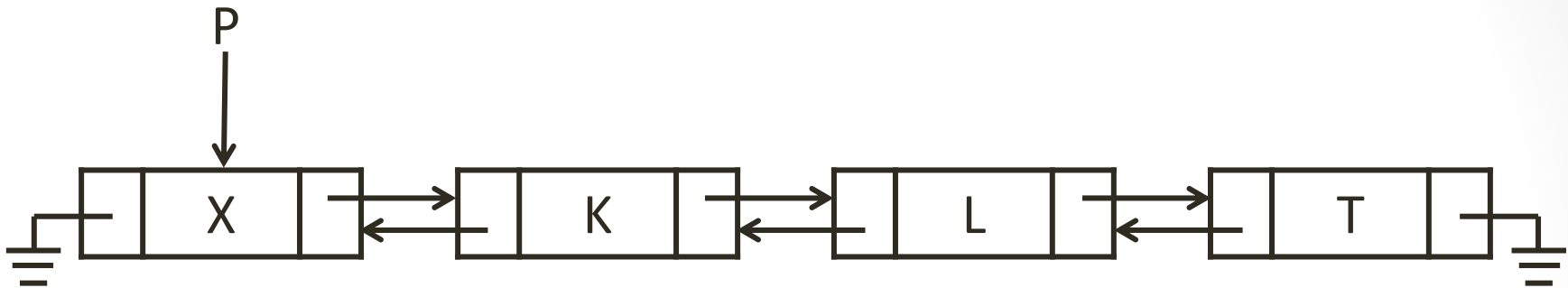
- Em uma lista duplamente ligada, cada variável dinâmica possuirá dois campos do tipo ponteiro:
 - O campo pred de uma variável dinâmica é um ponteiro que aponta para a **variável dinâmica predecessora** na lista duplamente ligada
 - No caso da primeira variável dinâmica da lista duplamente ligada, o campo pred é **aterrado**

Listas duplamente ligadas



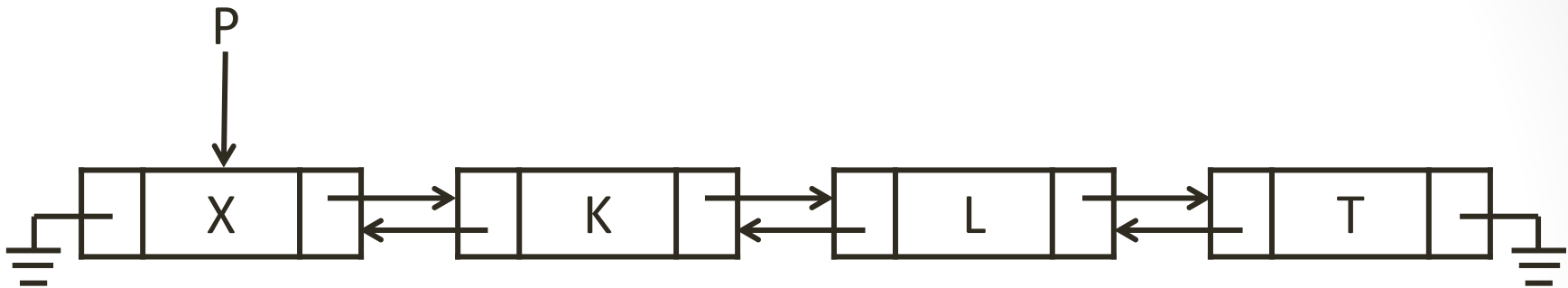
- Em uma lista duplamente ligada, cada variável dinâmica possuirá dois campos do tipo ponteiro:
 - O campo suc de uma variável dinâmica é um ponteiro que aponta para a **variável dinâmica sucessora** na lista duplamente ligada
 - No caso da última variável dinâmica da lista duplamente ligada, o campo suc é **aterrado**

Listas duplamente ligadas



- Propriedade: $Q = Q^{\wedge}.suc^{\wedge}.pred$
- (exceção: o último elemento da lista está aterrado, portanto, não tem sucessor)

Listas duplamente ligadas



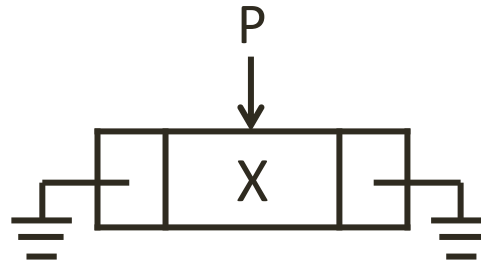
- Propriedade: $Q = Q^{\wedge}.pred^{\wedge}.suc$
- (exceção: o primeiro elemento da lista está aterrado, portanto, não tem predecessor)

Listas duplamente ligadas

- Casos particulares de listas duplamente ligadas

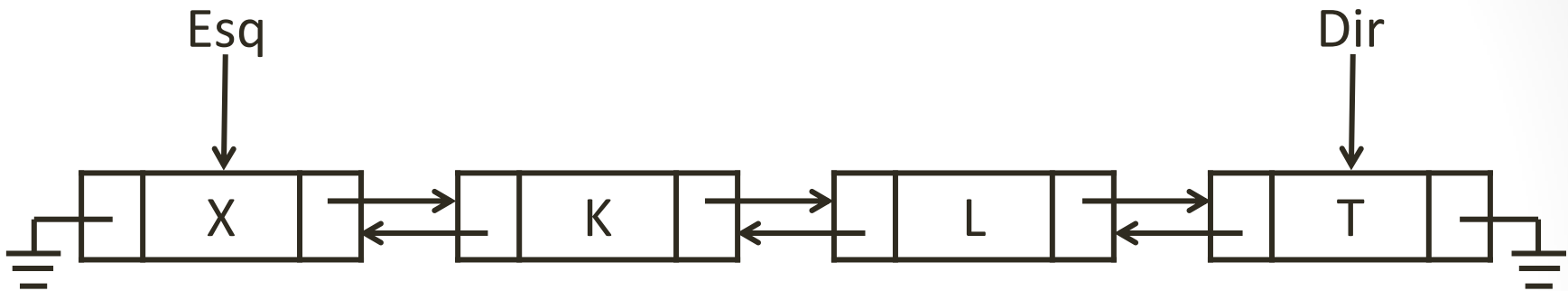


(Lista duplamente ligada vazia)



(Lista duplamente ligada com um único elemento)

Listas duplamente ligadas



tipo TpListaDupla = registro

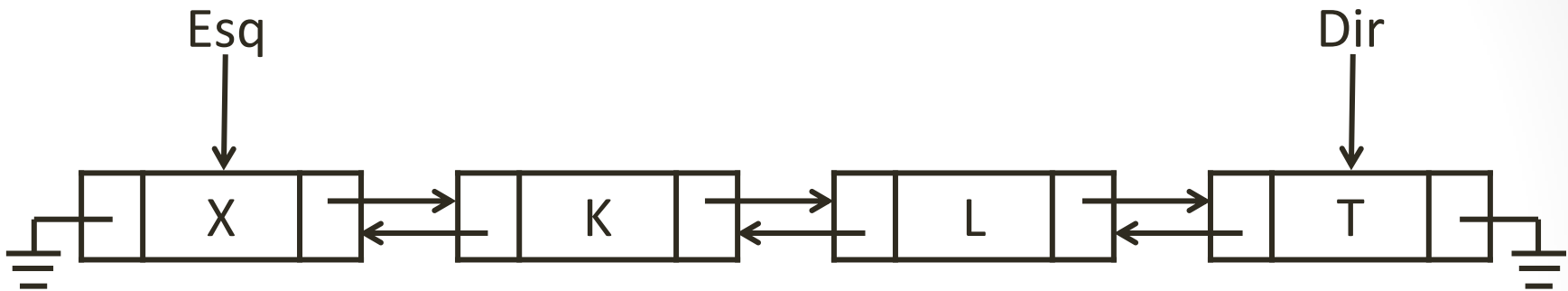
início

Esq : ^TpNóDuplo;

Dir : ^TpNóDuplo;

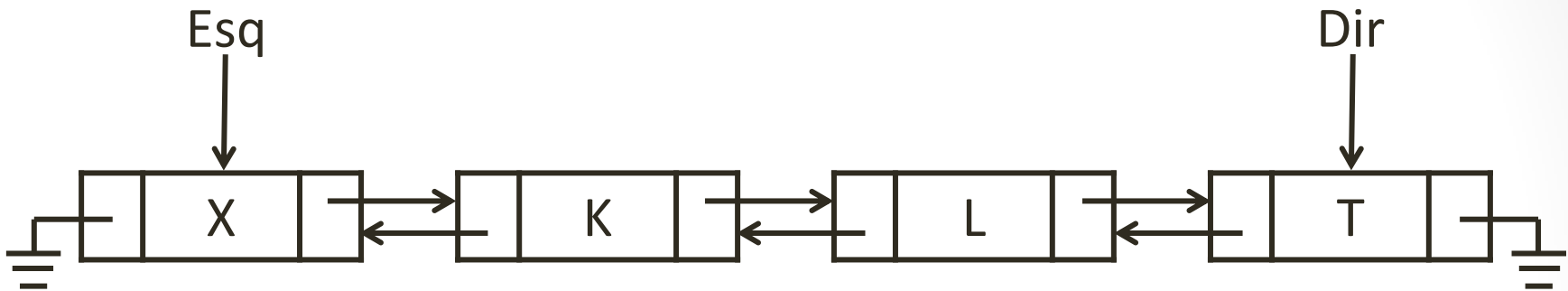
fim;

Listas duplamente ligadas



- Fila dupla (implementada com uma lista duplamente ligada)
- InicializaListaDupla(LD);
- ListaDuplaVazia(LD);
- InsereEsquerdaListaDupla(LD, x);
- InsereDireitaListaDupla(LD, x);

Listas duplamente ligadas



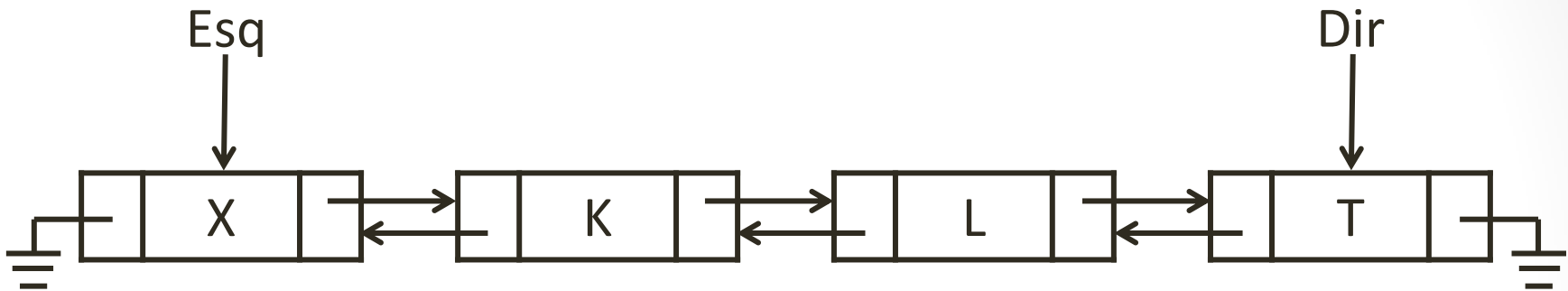
- Fila dupla (implementada com uma lista duplamente ligada)
- RemoveEsquerdaListaDupla(LD, x);
- RemoveDireitaListaDupla(LD, x);
- ElementoEsquerdaListaDupla(LD);
- ElementoDireitaListaDupla(LD);

Listas duplamente ligadas

- InicializaListaDupla(LD);

Complexidade:

Listas duplamente ligadas



InicializaListaDupla(LD);

LD.Esq $\leftarrow$ nil;

LD.Dir $\leftarrow$ nil;

Complexidade:

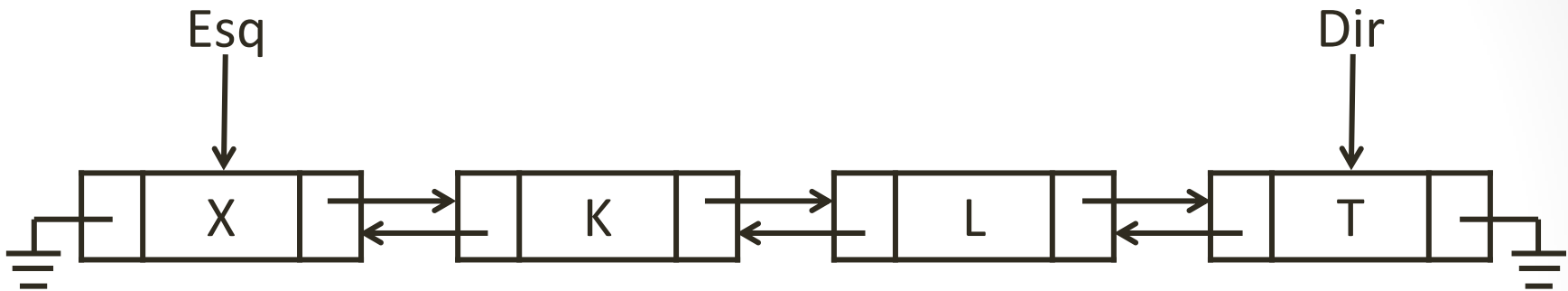
$\Theta(1)$

Listas duplamente ligadas

- ListaDuplaVazia(LD);

Complexidade:

Listas duplamente ligadas



`ListaDuplaVazia(LD);`

`ListaDuplaVazia ← LD.Esq = nil;`

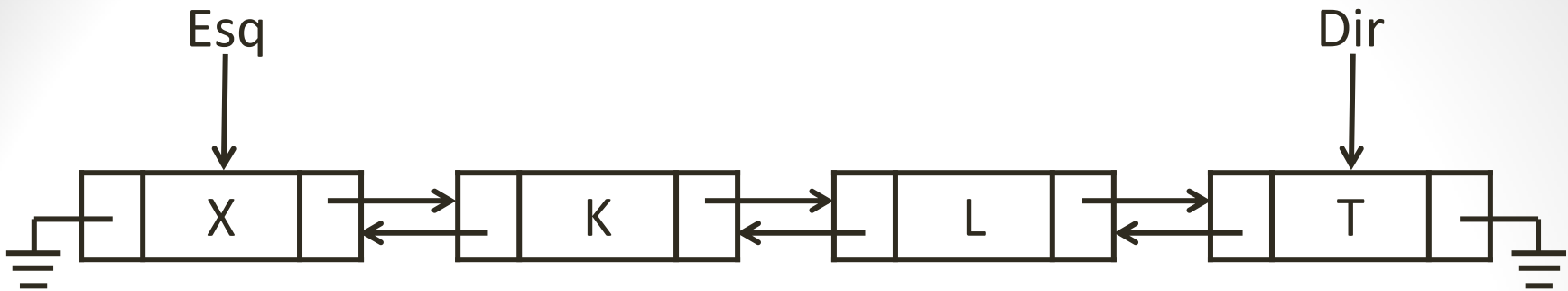
Complexidade:

$\Theta(1)$

Listas duplamente ligadas

- `InserEsquerdaListaDupla(LD, x);`

Complexidade:



InserEsquerdaListaDupla(LD, x);

novo(Q);

Q^.info $\leftarrow$ x;

Q^.pred $\leftarrow$ nil;

Q^.suc $\leftarrow$ LD.Esq;

se ListaDuplaVazia(LD)

então | LD.Dir $\leftarrow$ Q;

senão | LD.Esq^.pred $\leftarrow$ Q;

LD.Esq $\leftarrow$ Q;

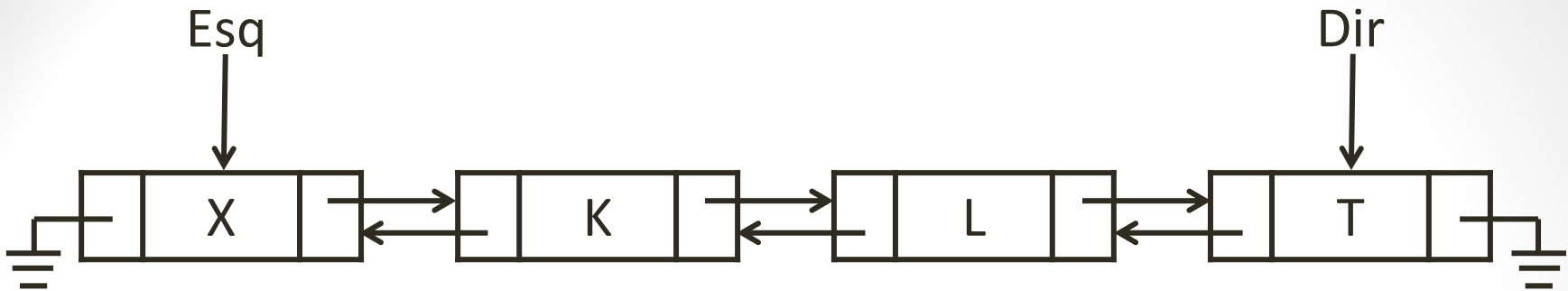
Complexidade:

$\Theta(1)$

Listas duplamente ligadas

- `InserDireitaListaDupla(LD, x);`

Complexidade:



InserDireitaListaDupla(LD, x);

novo(Q);

Q^.info $\leftarrow$ x;

Q^.suc $\leftarrow$ nil;

Q^.pred $\leftarrow$ LD.Dir;

se ListaDuplaVazia(LD)

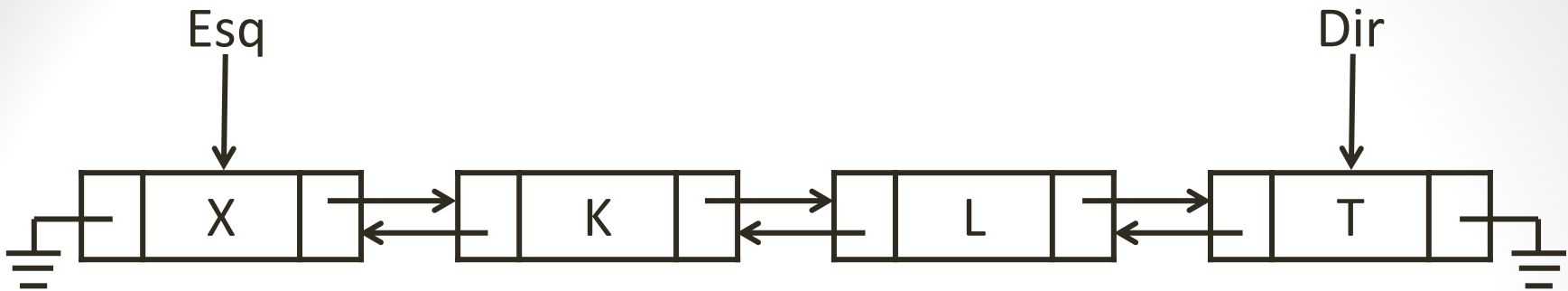
então | LD.Esq $\leftarrow$ Q;

senão | LD.Dir^.suc $\leftarrow$ Q;

LD.Dir $\leftarrow$ Q;

Complexidade:

$\Theta(1)$



InsereEsquerdaListaDupla(LD, x);

novo(Q);

Q^.info $\leftarrow$ x;

Q^.pred $\leftarrow$ nil;

Q^.suc $\leftarrow$ LD.Esq;

se ListaDuplaVazia(LD)

então | LD.Dir $\leftarrow$ Q;

senão | LD.Esq^.pred $\leftarrow$ Q;

LD.Esq $\leftarrow$ Q;

InsereDireitaListaDupla(LD, x);

novo(Q);

Q^.info $\leftarrow$ x;

Q^.suc $\leftarrow$ nil;

Q^.pred $\leftarrow$ LD.Dir;

se ListaDuplaVazia(LD)

então | LD.Esq $\leftarrow$ Q;

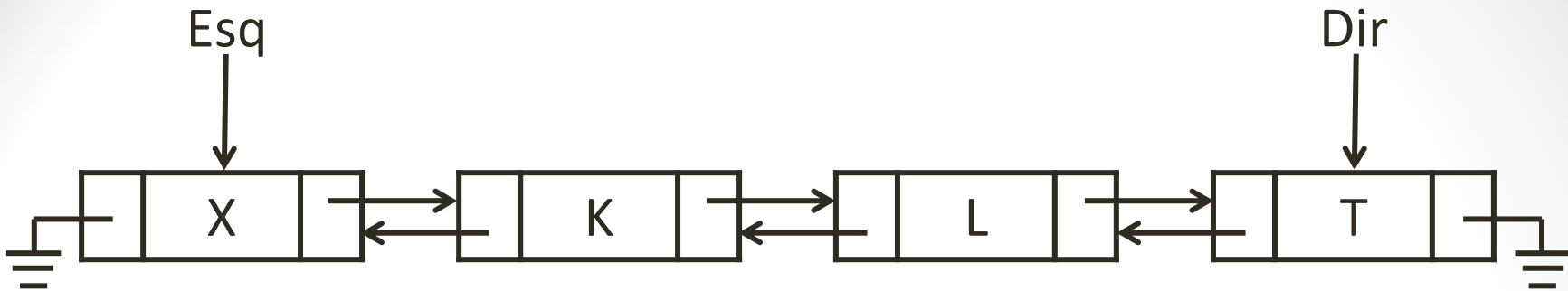
senão | LD.Dir^.suc $\leftarrow$ Q;

LD.Dir $\leftarrow$ Q;

Listas duplamente ligadas

- RemoveEsquerdaListaDupla(LD, x);

Complexidade:



RemoveEsquerdaListaDupla(LD, x);

se NÃO ListaDuplaVazia(D)

então

$x \leftarrow LD.Esq^.info;$

$Q \leftarrow LD.Esq;$

$LD.Esq \leftarrow LD.Esq^.suc;$

se $LD.Esq = nil$

então | $LD.Dir \leftarrow nil;$

senão | $LD.Esq^.pred \leftarrow nil;$

dispose(Q);

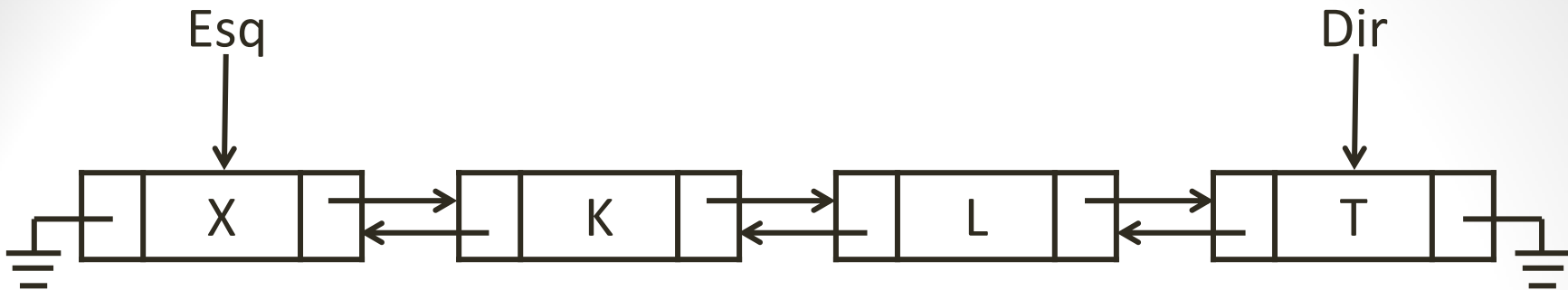
Complexidade:

$\Theta(1)$

Listas duplamente ligadas

- RemoveDireitaListaDupla(LD, x);

Complexidade:



RemoveDireitaListaDupla(LD, x);

se NÃO ListaDuplaVazia(D)

então

$x \leftarrow LD.Dir^{\wedge}.info;$

$Q \leftarrow LD.Dir;$

$LD.Dir \leftarrow LD.Dir^{\wedge}.pred;$

se $LD.Dir = nil$

então

$LD.Esq \leftarrow nil;$

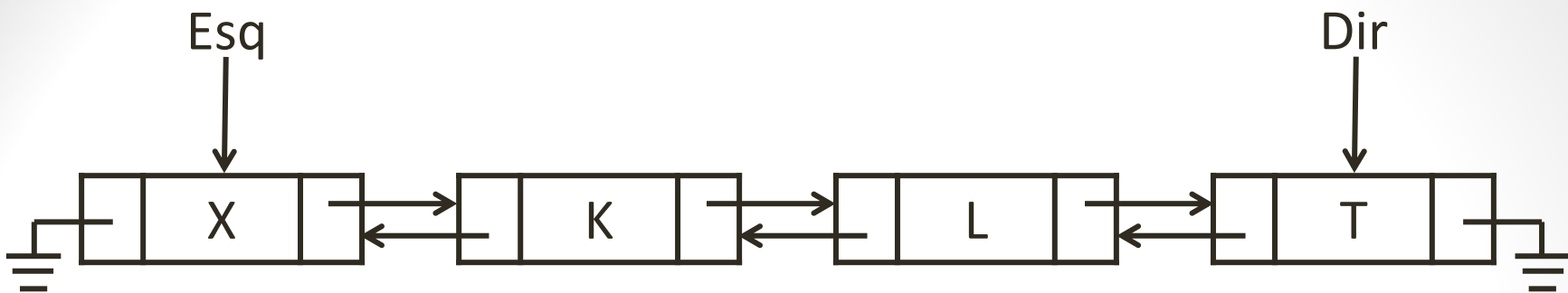
senão

$LD.Dir^{\wedge}.suc \leftarrow nil;$

dispose(Q);

Complexidade:

$\Theta(1)$



RemoveEsquerdaListaDupla(LD, x);

se NÃO ListaDuplaVazia(D)

então

$x \leftarrow LD.Esq^.info;$

$Q \leftarrow LD.Esq;$

$LD.Esq \leftarrow LD.Esq^.suc;$

se $LD.Esq = nil$

então $LD.Dir \leftarrow nil;$

senão $LD.Esq^.pred \leftarrow nil;$

dispose(Q);

RemoveDireitaListaDupla(LD, x);

se NÃO ListaDuplaVazia(D)

então

$x \leftarrow LD.Dir^.info;$

$Q \leftarrow LD.Dir;$

$LD.Dir \leftarrow LD.Dir^.pred;$

se $LD.Dir = nil$

então $LD.Esq \leftarrow nil;$

senão $LD.Dir^.suc \leftarrow nil;$

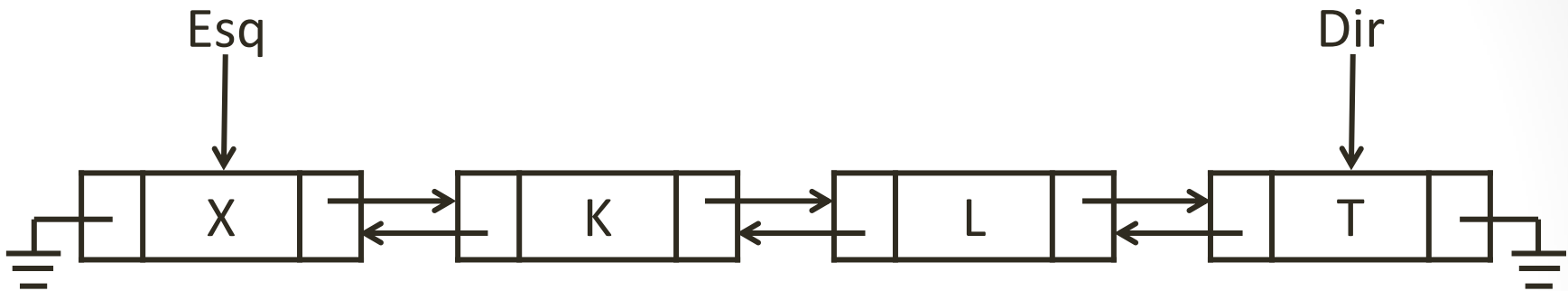
dispose(Q);

Listas duplamente ligadas

- ElementoEsquerdaListaDupla(LD);

Complexidade:

Listas duplamente ligadas



ElementoEsquerdaListaDupla(LD);

se NÃO ListaDuplaVazia(LD)

então | ElementoEsquerdaListaDupla $\leftarrow$ LD.Esq^.info;

senão | <erro>

Complexidade:

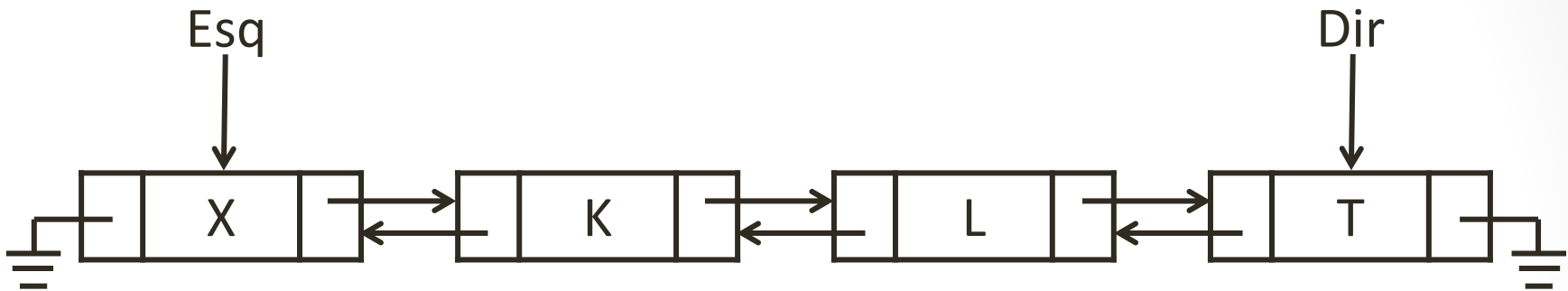
$\Theta(1)$

Listas duplamente ligadas

- ElementoDireitaListaDupla(LD);

Complexidade:

Listas duplamente ligadas



ElementoDireitaListaDupla(LD);

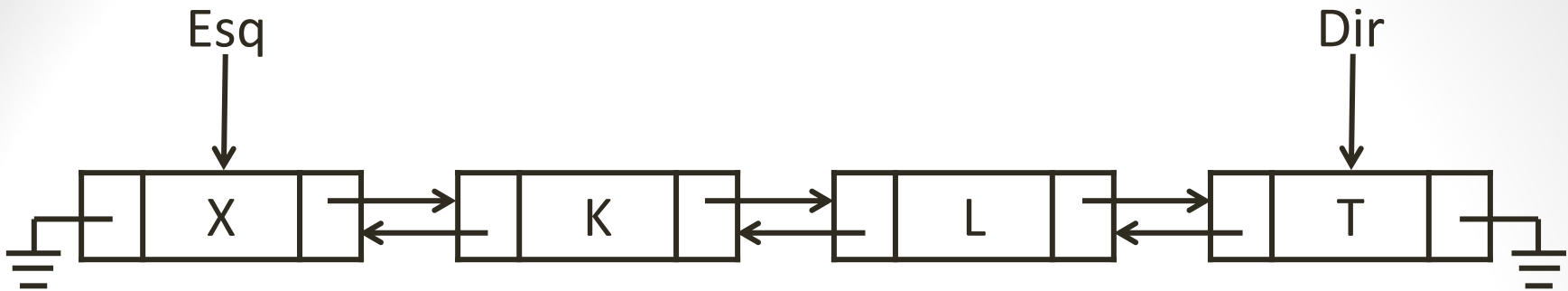
se NÃO ListaDuplaVazia(LD)

então | ElementoDireitaListaDupla $\leftarrow$ LD.Dir^.info;

senão | <erro>

Complexidade:

$\Theta(1)$



ElementoEsquerdaListaDupla(LD);

se NÃO ListaDuplaVazia(LD)

então | ElementoEsquerdaListaDupla $\leftarrow$ LD.Esq^.info;

senão | <erro>

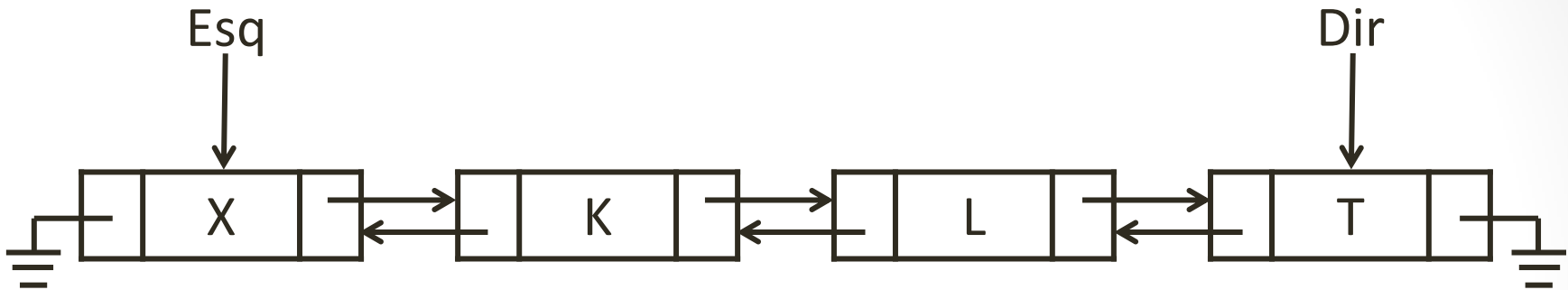
ElementoDireitaListaDupla(LD);

se NÃO ListaDuplaVazia(LD)

então | ElementoDireitaListaDupla $\leftarrow$ LD.Dir^.info;

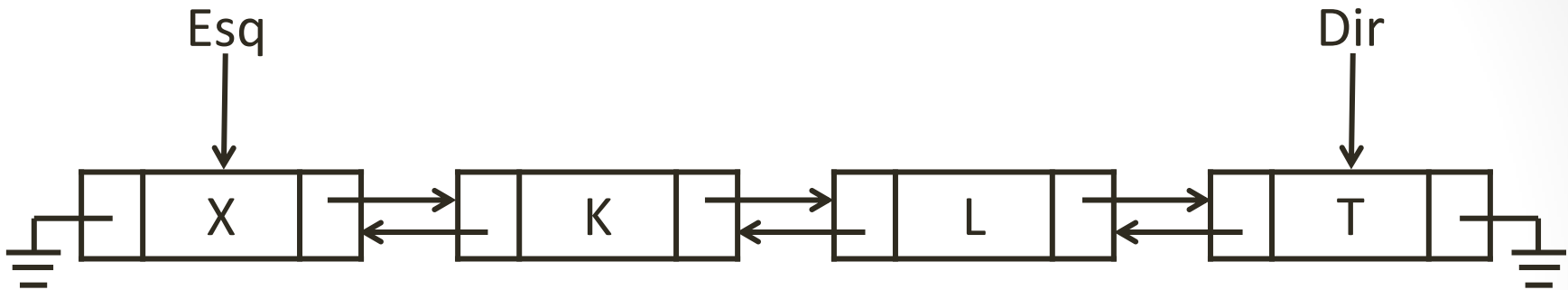
senão | <erro>

Listas duplamente ligadas



Função	Complexidade
InicializaListaDupla(LD);	$\Theta(1)$
ListaDuplaVazia(LD);	$\Theta(1)$
InseresquerdaListaDupla(LD, x);	$\Theta(1)$
InseredireitaListaDupla(LD, x);	$\Theta(1)$

Listas duplamente ligadas



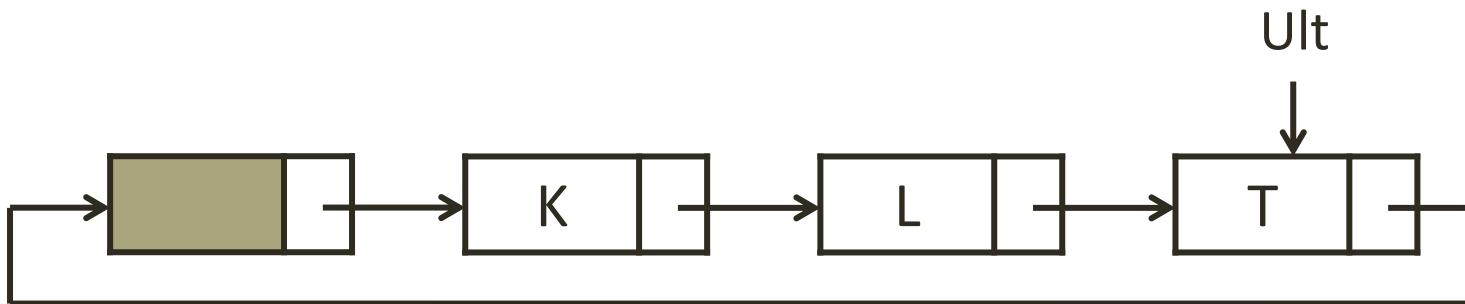
Função	Complexidade
RemoveEsquerdaListaDupla(LD, x);	$\Theta(1)$
RemoveDireitaListaDupla(LD, x);	$\Theta(1)$
ElementoEsquerdaListaDupla(LD, x);	$\Theta(1)$
ElementoDireitaListaDupla(LD, x);	$\Theta(1)$

Outras implementações de listas ligadas

- Listas ligadas com cabeça de lista
 - Listas circulares
 - Listas duplamente ligadas
-
- **Podemos combinar os tipos de listas ligadas apresentados e gerar outros tipos de listas ligadas**

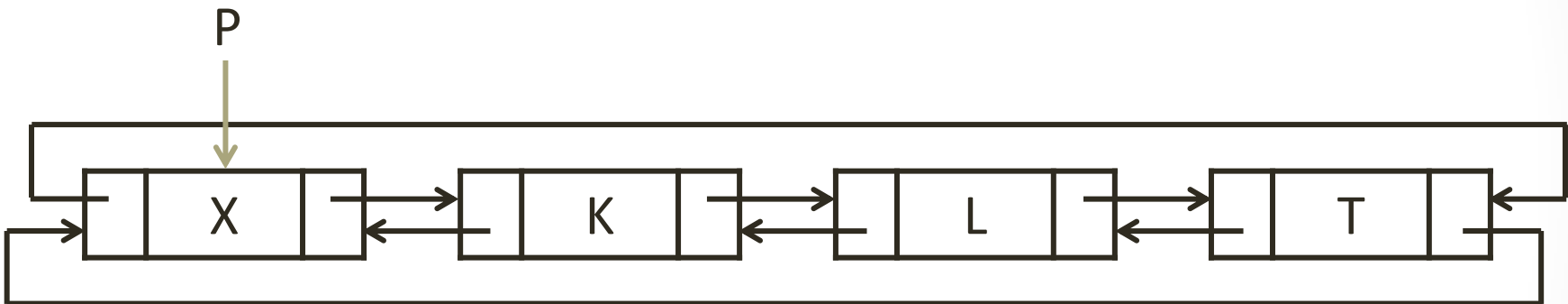
Outras implementações de listas ligadas

- Listas circulares com cabeça de lista



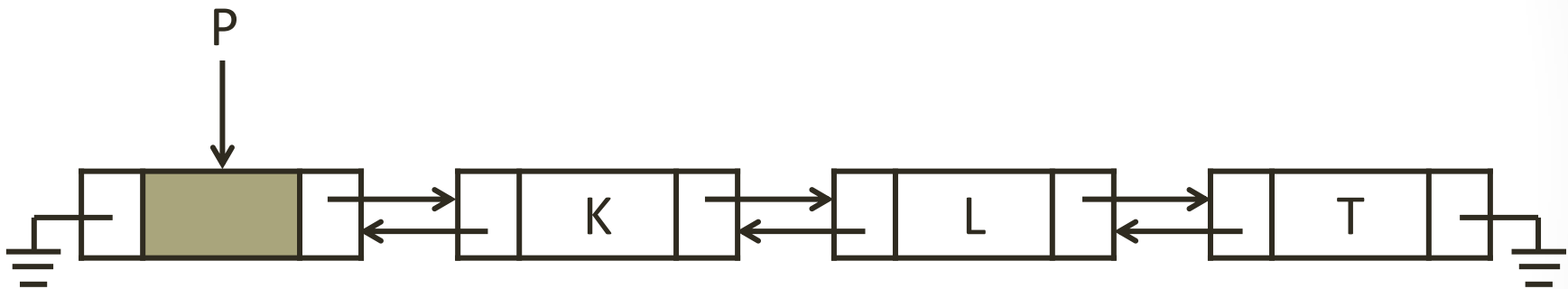
Outras implementações de listas ligadas

- Listas circulares duplamente ligadas



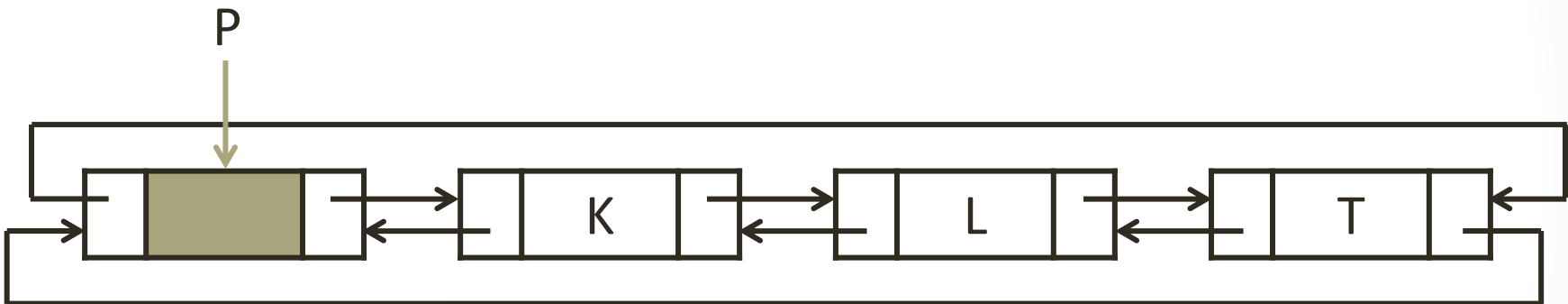
Outras implementações de listas ligadas

- Listas duplamente ligadas com cabeça de lista



Outras implementações de listas ligadas

- Listas circulares duplamente ligadas com cabeça de lista



Outras implementações de listas ligadas

- Aplicação : matrizes esparsas

<https://homepages.dcc.ufmg.br/~nivio/cursos/aed2/tp1/aed01tp1.html>

Exercícios

- Listas lineares
- Considere as seguintes operações com o tipo abstrato de dado lista linear
 1. Criar uma lista linear
 2. Localizar o i -ésimo elemento de uma lista linear
 3. Inserir um elemento na i -ésima posição
 4. Remover o i -ésimo elemento da lista linear
 5. Combinar duas ou mais listas lineares em uma única lista
- Implemente as operações apresentadas utilizando alocação estática (vetores)
- Apresente a complexidade das funções implementadas para o pior caso (em notação assintótica)

Exercícios

- Listas lineares
- Considere as seguintes operações com o tipo abstrato de dado lista linear
 6. Separar uma lista linear em duas ou mais listas
 7. Criar uma cópia dos elementos de uma lista linear (gerando uma outra lista linear)
 8. Informar o número de elementos de uma lista linear
 9. Pesquisar por um elemento em uma lista linear
 10. Ordenar uma lista linear de acordo com alguma característica de seus elementos (ordem crescente ou decrescente)
- Implemente as operações apresentadas utilizando alocação estática (vetores)
- Apresente a complexidade das funções implementadas para o pior caso (em notação assintótica)

Exercícios

- Listas lineares
- Considere as seguintes operações com o tipo abstrato de dado lista linear
 1. Criar uma lista linear
 2. Localizar o i -ésimo elemento de uma lista linear
 3. Inserir um elemento na i -ésima posição
 4. Remover o i -ésimo elemento da lista linear
 5. Combinar duas ou mais listas lineares em uma única lista
- Implemente as operações apresentadas utilizando alocação dinâmica (listas ligadas)
- Apresente a complexidade das funções implementadas para o pior caso (em notação assintótica)

Exercícios

- Listas lineares
- Considere as seguintes operações com o tipo abstrato de dado lista linear
 6. Separar uma lista linear em duas ou mais listas
 7. Criar uma cópia dos elementos de uma lista linear (gerando uma outra lista linear)
 8. Informar o número de elementos de uma lista linear
 9. Pesquisar por um elemento em uma lista linear
 10. Ordenar uma lista linear de acordo com alguma característica de seus elementos (ordem crescente ou decrescente)
- Implemente as operações apresentadas utilizando alocação dinâmica (listas ligadas)
- Apresente a complexidade das funções implementadas para o pior caso (em notação assintótica)

Exercícios

- Listas lineares
- Considere as implementações solicitadas nos slides anteriores utilizando alocações estática e dinâmica
- Compare e comente, para cada um dos itens (1 a 10), as implementações para o caso estático e dinâmico em termos de complexidade para o pior caso (utilize notação assintótica em suas respostas)

Exercícios

- Pilha
- Faça uma função que receba uma cadeia de caracteres contendo uma expressão numérica e que verifique se a parentização da expressão está correta.
- A função deve devolver VERDADEIRO se a parentização estiver correta, e FALSO, caso contrário.
- Utilize pilha na sua solução
 - (a) Considere expressões numéricas que utilizam somente parênteses, como as que usamos em um programa
 - (b) Considere expressões numéricas que utilizam parênteses, colchetes e chaves

<https://brasilecola.uol.com.br/matematica/expresoes-numericas.htm>

Exercícios

- Pilha
- Demonstre a utilização de uma pilha na resolução de uma expressão numérica em notação posfixa.
- Utilize na sua demonstração uma expressão em notação posfixa e a resolução passo-a-passo utilizando uma pilha
- Considere uma partida do jogo Torre de Hanoi com cinco discos no pino inicial.
- Apresente a sequência de operações Empilha / Desempilha executada para passar os cinco discos para uma outra pilha

Exercícios

- Pilha
- Faça uma função que receba uma pilha e que remova um elemento que se encontre na base da estrutura.
- (dica: utilize uma pilha auxiliar na solução proposta)
- Faça uma função que inverta os elementos de uma pilha utilizando uma fila

Exercícios

- Pilha
- Implemente a função `InicializaPilha(P)` considerando uma pilha armazenada em lista ligada. Esta função deve inicializar a pilha devolvendo todas as células utilizadas para a memória

Exercícios

- Pilha
- Faça uma função que implemente duas pilhas em um único vetor.
- Deve ser proposta uma estrutura de dados TpPilhas que seja um registro contendo o vetor e as variáveis de controle dos topos de cada pilha
- Implemente as operações de manipulação destas pilhas (InicializaPilha1, InicializaPilha2, PilhaVazia1, PilhaVazia2, PilhaCheia1, PilhaCheia2, Empilha1, Empilha2, Desempilha1, Desempilha2, ElementoDoTopo1 e ElementoDoTopo2)

Exercícios

- Pilha
- Implemente uma pilha utilizando duas filas.
- Deve ser proposta uma estrutura de dados TpPilha que seja um registro onde as duas filas sejam campos da estrutura
- Implemente as operações de manipulação desta pilha

Exercícios

- Fila
- Faça uma função que inverta os elementos de uma fila utilizando uma pilha
- Faça uma função que receba duas filas F1 e F2 ordenadas em ordem crescente e que gere uma única fila ordenada. As filas F1 e F2 ficarão vazias no processo

Exercícios

- Fila
- Faça uma função que receba uma fila de números inteiros e que devolva a mesma fila com todos os elementos positivos posicionados antes de todos os elementos negativos
- Implemente a função InicializaFila(Q) considerando uma fila armazenada em lista ligada. Esta função deve inicializar a fila devolvendo todas as células utilizadas para a memória

Exercícios

- Fila
- Faça uma função que receba uma fila de números inteiros e que esvazie esta lista da seguinte forma:
 - remova o primeiro elemento
 - se o número removido for positivo, decremente o número e o insira novamente na fila
 - Se o número removido for negativo ou zero, descarte este número

Exercícios

- Fila
- Implemente uma fila utilizando duas pilhas.
- Deve ser proposta uma estrutura de dados TpFila que seja um registro onde as duas pilhas sejam campos da estrutura
- Implemente as operações de manipulação desta fila

Exercícios

- Fila dupla
- Faça uma função que receba uma cadeia de caracteres e que verifique se esta cadeia é um palíndromo
- Utilize fila dupla na solução proposta
 - (a) Considere o problema de avaliar uma única palavra como palíndromo ou não
 - (b) Considere o problema de avaliar uma frase, ignorando espaços e pontuação

Exercícios

- Outras implementações de listas ligadas
- Faça uma função que receba uma lista linear implementada com lista circular e que mova o primeiro elemento da lista linear para a última posição
- Faça uma função que receba uma lista linear implementada com lista circular e que mova o último elemento da lista linear para a primeira posição

Exercícios

- Outras implementações de listas ligadas
- Faça uma função que receba duas listas ligadas circulares P1 e P2 e que realize a concatenação destas duas listas. A lista ligada P2 deve ser concatenada no final da lista ligada P1.
 - (A lista de saída também deve ser uma lista ligada circular)
 - (A lista ligada P2 deve ser concatenada no final da lista ligada P1)

Exercícios

- Outras implementações de listas ligadas
- Faça uma função que receba duas listas ligadas com cabeça de lista P1 e P2 e que realize a concatenação destas duas listas. A lista ligada P2 deve ser concatenada no final da lista ligada P1.
 - (A lista de saída também deve ser uma lista ligada com cabeça de lista)
 - (A lista ligada P2 deve ser concatenada no final da lista ligada P1)

Exercícios

- Outras implementações de listas ligadas
- Faça uma função que remova um elemento x de uma lista duplamente ligada P , caso x esteja nesta lista (assuma que a lista ligada P não tenha elementos repetidos)
- Faça uma função que receba uma lista duplamente ligada P ordenada e um elemento x . Esta função deve inserir o elemento x na lista P (a lista P deve continuar ordenada após a inserção de x)

Exercícios

- Outras implementações de listas ligadas
- Escreva todas as operações de gerenciamento de um fila dupla armazenada em uma lista circular duplamente ligada

Bibliografia

- ZIVIANI, N.; *Projeto de Algoritmos com Implementações em Pascal e C*, 3ª Edição, Livraria Pioneira Editora, 1996.
- CORMEN, T. H. et al – *Algoritmos* , Trad. Segunda Edição Americana. Editora Campus, 2002.
- WIRTH, N. – *Algoritmos e Estruturas de Dados*. Editora Prentice-Hall, 1989.